

22春A卷答案

2026年5月27日 20:33



22春A卷答
案



华中科技大学计算机科学与技术学院 2021~2022 第二学期

“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试日期 2022-05-21 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四		总分	核对人
分值	40	20	20	20		100	
得分							

分 数	
评卷人	

一、程序运行的基本原理问答（共 40 分）

解答内容不得超过装订线

对如下 C 语言程序，用 VS2019（Intel CPU）编译、链接、调试运行。

```
#include <stdio.h>
int fmax(int x, int y)
{
    int u;
    if (x > y) u = x;
    else u = y;
    return u;
}
int main(int argc, char* argv[])
{
    int a = 100; // 0x64
    int b = 0x12345678;
    int r = 0;
    char msg[6] = "abcde"; // 'a'的ASCII是 0x61
    r = fmax(a, b);
    msg[17] = '1'; // '1'的ASCII是 0x31
    r = *(int*)(msg+1);
    return 0;
}
```

在调试时，在监视窗口中看到 变量 a 的地址（即&a）为 0x010ffe98；变量 b 的地址（即&b）为 0x010ffe94；变量 r 的地址为 0x010ffe90， 数组 msg 的起始地址为 0x010ffe88。

1、填写执行到“r=fmax(a,b);”时，内存中数据存放的结果。要求以字节为单位、用 16 进制数的形式填空，最左边是内存窗口显示的内存地址。（10 分）

0x010ffe88	<u>61</u>	<u>62</u>	<u>63</u>	<u>64</u>	<u>65</u>	<u>00</u>	XX	XX
0x010ffe90	<u>00</u>	<u>00</u>	<u>00</u>	<u>00</u>	<u>78</u>	<u>56</u>	<u>34</u>	<u>12</u>
0x010ffe98	<u>64</u>	<u>00</u>	<u>00</u>	<u>00</u>	XX	XX	XX	XX

2、数据传送指令解读 （10 分，每空 2 分）

语句 “int a = 100;” 的反汇编指令为： mov dword ptr [ebp-4], 64h

根据观察到的 a 的地址，推断执行该语句时，(ebp)= 0x 10ffe9c，该指令的机器码为 C7 45 FC 64 00 00 00，其中 FC 对应的是指令中 -4 的编码。

根据观察到的 a 与 b 的地址关系，推断出 “int b = 0x12345678; ” 对应的反汇编指令为：
mov dword ptr [ebp-8], 12345678h

执行 “msg[17]='1’; ” 后，变量 a 中的值为 0x 3164 。

执行 “r = *(int *)(msg+1);” 后， r 中的值为 0x 65646362 。

3、函数调用语句解读 （10 分，每空 2 分）

对于语句 “r=fmax(a,b);” 对应的反汇编代码（最左边的是机器指令的地址）如下。

0072162B	mov	eax, dword ptr [ebp-8]		0x010ffe28
0072162E	push	eax		0x010ffe2c
0072162F	mov	ecx, dword ptr [ebp-4]	0x00721632	0x010ffe30
00721632	push	ecx	0x00000064	0x010ffe34
00721633	call	007212B2	0x12345678	0x010ffe38
00721638	add	esp, 8	XXXXXXXX	0x010ffe3c
0072163B	mov	dword ptr [ebp-0Ch], eax		

设执行 “r=fmax(a,b);” 之前，(esp) = 0x010ffe3c

- ① 在表格的适当位置填写 刚进入函数 fmax 内部时，堆栈中存放的相关数据。
- ② 刚进入函数 fmax 内部时，(esp)= 0x 010ff330
- ③ 执行 “add esp, 8” 之后，(esp) = 0x 0x10ffe3c 。

4. 函数体的指令解读 (10 分，每小题 2 分)

函数体对应的反汇编代码有：

```
push    ebp
mov     ebp, esp    .....
if (x > y) u= x;
007215C3 mov     eax, dword ptr [ebp+8]
007215C6 cmp     eax, dword ptr [ebp+0Ch]
007215C9 jle     007215D3
007215CB mov     eax, dword ptr [ebp+8]
007215CE mov     dword ptr [ebp-4], eax
007215D1 jmp     007215D9
else u = y;
007215D3 mov     eax, dword ptr [ebp+0Ch]
007215D6 mov     dword ptr [ebp-4], eax
return u;
007215D9 mov     eax, dword ptr [ebp-4] .....
pop     ebp
ret
```

解答内容不得超过装订线

- ① 执行到 “cmp eax, dword ptr [ebp+0Ch]” 指令时, (ebp)=0x__010ff32c__。
- ② 函数参数 x 的地址 (即&x) 是 0x__010ff334__。
- ③ 执行 “cmp eax, dword ptr [ebp+0Ch] ” 后, CF=__1__, SF=__1__, ZF=__0__, OF=__0__。
- ④ 执行 RET 指令时, CPU 所做的工作是__将栈顶双字出栈到 EIP__。
- ⑤ 函数返回结果的方法是__使用 EAX 传递返回整数值__。

分 数	
评卷人	

二、程序阅读与理解 （共 20 分）

1、阅读下面的程序，回答问题 （10 分）。

```
.686P
.model flat, c
includelib kernel32.lib
includelib libcmtd.lib
includelib legacy_stdio_definitions.lib
printf proto :ptr sbyte, :vararg
ExitProcess proto stdcall :dword
.data
    nums sdword 1234, -1, 100, -20, -2
    len = ($-nums)/4
    fmt db "%d ", 0dh, 0ah, 0
.code
main proc c
    mov edx, 0
    mov ecx, len ; ①
    lea esi, nums ; ②
process:
    cmp dword ptr [esi], 0
    jge next ; ③
    inc edx
next:
    add esi, 4
    dec ecx
    jne process
    invoke printf, offset fmt, edx
    invoke ExitProcess, 0
main endp
end
```

- 1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？（2 分）
统计 nums 中，负数的个数，保存到 edx 中，并显示 edx 的值。显示 3

2) 若将 ③ 处的语句改为 “jae next”, 程序运行的结果是什么? (2 分)

显示 0

3) 若标号 process 写到 ②处语句前, 程序运行的结果是什么? 为什么? (3 分)

显示 0, 始终比较第一个元素

4) 若标号 ①处语句写到标号 process 之后, 程序运行会出现什么现象? 为什么? (3 分)

```
process: mov ecx, len
        cmp dword ptr [esi], 0 .....
```

死循环。每次循环未修改 ecx 的值。

2. 程序填空 (10 分, 每空 1 分)

在一个以 0 结束的字符串中, 将所有的大写字母转换为对应的小写字母, 并将转换结果、以及修改的字母个数输出。

```
.....
.data
    buf db "abcDEFgh", 0
    fmt db "%s %d", 0

.code
main proc c
    mov esi, _offset buf ; esi 存放待访问字符的地址
    mov ecx, 0 ; ecx 存放修改字母的个数

next:
    mov al, [esi]
    cmp al, 0
    je exit
    cmp al, 'A'
    jb cont
    cmp al, 'Z'
    ja cont
    add al, 'a' - 'A'
    mov [esi], al
    inc ecx

cont:
    inc esi
    jmp next

exit:
    invoke printf, offset fmt, offset buf, ecx
    invoke ExitProcess, 0

main endp
end
```

分 数	
评卷人	

三、程序优化 （共 20 分）。

1、对 C 程序进行编译生成 Release 版程序时，编译器会做一些优化工作。请举出 5 个优化场景，简要说明做了什么优化，以及为什么能加快执行速度。（10 分）

- 使用寄存器作为循环计数器；
- 使用寄存器作为数组下标，进行变址寻址；
- 使用寄存器传递参数；
- 使用带比例因子的变址寻址操作数，获取 EA，代替算术运算；
- 使用宽字节类型的寄存器，进行数组元素的操作，减少循环次数；
- 使用顺序访问数组元素的方式，代替循环访问数组元素；

解答内容不得超过装订线

2、如下的 C 语言程序段（32 位段）实现了“统计一个字符串 buf 中小写字母的个数，并将其放入 count 中”的功能。其编译后调试版本的汇编语言代码如下(注：斜体部分为 C 语句；在反汇编时，勾选显示符号名)。（10 分）

```
int count = 0;
00A616A0  mov     dword ptr [count],0
int i;
for (i = 0; buf[i] != 0;i++)
00A616A7  mov     dword ptr [i],0
00A616AE  jmp     fcount+59h (0A616B9h)
00A616B0  mov     eax,dword ptr [i]
00A616B3  add     eax,1
00A616B6  mov     dword ptr [i],eax
00A616B9  mov     eax,dword ptr [i]
00A616BC  movsx   ecx,byte ptr buf[eax]
00A616C1  test    ecx,ecx
00A616C3  je      fcount+8Ah (0A616EAh)
if (buf[i] >= 'a' && buf[i] <= 'z')
00A616C5  mov     eax,dword ptr [i]
00A616C8  movsx   ecx,byte ptr buf[eax]
```

```

00A616CD  cmp      ecx,61h
00A616D0  jl      fcount+88h (0A616E8h)
00A616D2  mov      eax,dword ptr [i]
00A616D5  movsx    ecx,byte ptr buf[ecx]
00A616DA  cmp      ecx,7Ah
00A616DD  jg      fcount+88h (0A616E8h)
        count++;
00A616DF  mov      eax,dword ptr [count]
00A616E2  add      eax,1
00A616E5  mov      dword ptr [count],eax
00A616E8  jmp      fcount+50h (0A616B0h)

```

(1) 指出该段程序执行效率不高的原因 (2 分)。

使用局部变量作为循环计数器；使用局部变量作为数组下标，进行变址寻址；
在条件表达式中，对同一个变量 buf[i]，进行了两次赋值

(2) “00A616D0 jl fcount+88h (0A616E8h)” 处指令的机器码为 7CH 16H，解释 16H 代表的含义 (2 分)

为跳转目的地址到该指令的距离：0A616E8h - (0A616D0h + 2)

(3) 改编相应的汇编语言程序，以提高程序的执行效率。(6 分)

要求写出采取的优化方法，在优化程序段中，可以用标号来代替转移指令的目的地址。

```

count: edx
i: ecx
buf[i]: al

mov edx, 0
mov ecx, 0
next: mov al, buf[ecx]
      cmp al, 0
      je exit
      cmp al, 'a'
      jb cont
      cmp al, 'z'
      ja cont
      inc edx
cont: jmp next
exit: mov count, edx

```

解答内容不得超过装订线

分 数	
评卷人	

四、设计题（20 分）

在 BUF1 为首址的字类型存储区中，存放了 M 个互不相同的数据；在 BUF2 为首址的字类型存储区中存放了 N 个数据。用汇编语言编写完整的程序，统计在两个存储区中同时出现的数据个数，存放到字类型变量 COUNT 中，最后在屏幕上输出 COUNT 的值。

要求：

- (1) 简要描述设计思想，给出寄存器分配方案。
- (2) 用子程序 FIND 判断某个字类型数据是否在某存储区中出现；描述其入口参数、出口参数。
- (3) 画出程序的流程图。
- (4) 程序完整（包括堆栈段、数据段、代码段定义等），至少给出 4 条必要的注释。
- (5) BUF1、BUF2 中的数据在定义时直接给出初值，无需输入；M、N 自己给定；子程序 FIND 中不允许使用全局变量。

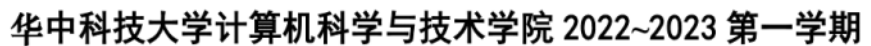
(1)
esi: BUF1 寻址、BUF2 寻址
ecx: 外循环、内循环
(2) FIND 入口参数：
ax: 待查找字
ebx:存储区首址
edx: 存储区长度
出口参数：
ax: 1 表示找到；0 表示未找到

22秋A卷

2026年5月27日 20:36



22秋A卷



考试方式	闭卷	考试日期	2022-12-04	考试时长	150 分钟
------	----	------	------------	------	--------

题号	一	二	三	四	五	总分	核对人
分值	20	30	20	20	10	100	
得分							

分 数	
评卷人	

解答内容不得超过装订线

① 在取出指令、对指令进行译码后, EIP 会自动加上_____ ; 若该指令不涉及转移,

② 若指令是条件转移指令, 如 LP (LP 为一个标号), 在 ZF=1 时, 会执行 (EIP) +

③ 若指令是无条件转移指令 **JMP**，除了直接跳转外，还可以进行间接跳转。借助于寄存器



⑥在程序运行过程中，出现中断信号时，CPU 会根据中断类型号，利用 找到

第 1 页 共 9 页

2、机器指令一般都有操作对象，指令中可以直接存放常量操作数或者操作数的地址。操作数的地址有多种表达方式。下面一段代码给出了访问同一单元的多种 C 语句。请回答 C 语句访问方式与机器指令的对应关系。

```
int global[5] = { 10,20,30,40,50 }; // 全局变量

void func ()
{
    int local[5] = { 10,20,30,40,50 };
    int i = 2;
    int *p;
    global[i] = 25;
    *(global + 2) = 25;
    p = &global[2];
    *p = 25;
    local[i] = 25;
}
```

① 在反汇编窗口看到 `global[i]=25` 对应如下两条指令：

```
mov    eax, dword ptr [ebp-18h]    ; 显示符号名时的语句为 mov eax, dword ptr [i]
mov    dword ptr [eax*4+00419000h],19h
```

第一条指令中变量 `i` 的地址表达式为 _____；第二条指令中目的操作数的寻址方式是 _____，数组 `global` 的起始地址是 _____，目的操作数寻址中采用比例因子 4 的原因是 _____。

② `*(global + 2) = 25`；仅对应一条机器指令，指令为 _____

其目的操作数的寻址方式是 _____。

③ `*p = 25`；如果变量 `p` 的地址表达式为 `[ebp-1Ch]`，访问存储单元(`*p`)时，使用的是寄存器间接寻址方式（使用寄存器 `eax`），则对应的两条汇编指令为：

```
_____  
_____
```

④ `local[i]=25`；访问 `local[i]` 采用的是基址加变址寻址方式，对应的汇编指令如下：

```
mov    _____, dword ptr [ebp-18h]
mov    dword ptr [ebp+eax*4-14h],19h
```

与全局变量 `global` 空间分配位置不同，`local` 的空间分配在栈上。

数组 `local` 的起始地址的表达式为：_____。

分 数	
评卷人	

二、数据存储及 C 语句转换填空 (共 30 分)

在Windows + VS2019 (Intel CPU) 环境下, 对一个C语言程序进行编译、链接、调试运行, 程序片段如下。

```
int fmin(int x, int y)
{
    int t;
    if (x > y) t= y;
        else t = x;
    return t;
}

void fmain( )
{
```

```
char buf[6] = "12345";      // '1' 的 ASCII 是 0x31
int u = 0x20221204;         // mov dword ptr [ebp-0Ch], 20221204h
int v = -32;                // mov dword ptr [ebp-10h], 0FFFFFFE0h
int w = 0;                  // mov dword ptr [ebp-14h], 0
w = fmin(u, v);
w = *(int *) (buf+1);
```

在调试时，在监视窗口中看到数组 buf 的起始地址为 0x00ffdb8，变量 u 的地址（即&u）为 0x00ffdb4；v 的地址（即&v）为 0x00ffdb0。

- 1、填写执行到“w=fmin(u,v);”时，内存中数据存放的结果。要求以字节为单位、用 16 进制数的形式填空，最左边是内存窗口显示的内存地址。（6 分）

2、 0x00dffd0

3、 0x00dffdb8	XX	XX
---------------	----	----

- 4、 数据传送指令解读 (6 分, 每空 2 分)

语句 “int u = 0x20221204;” 的反汇编指令为: `mov dword ptr [ebp-0Ch], 20221204h`

根据观察到的 u 的地址，推断执行该语句时，(ebp)= 0x _____，该指令的机器码为 C7

45 F4 04 12 22 20, 其中 F4 对应的是指令中的 的编码。

执行 “w = *(int *) (buf+1);” 后, w 中的值为 0x 00000000。

- 5、函数调用语句解读 (10 分, 每空 2 分)

对于语句 “w=fmin(u, v);” 对应的反汇编代码（最左边的是机器指令的地址）如下。

```

009A204B  mov  eax,dword ptr [ebp-10h]
009A204E  push  eax
009A204F  mov  ecx,dword ptr [ebp-0Ch]
009A2052  push  ecx
009A2053  call  009A125D
009A2058  add   esp,8
009A205B  mov  dword ptr [ebp-14h],eax

```

	0x00dffd4c
	0x00dffd50
	0x00dffd54
	0x00dffd58
	0x00dffd5c
XXXXXXXX	0x00dffd60

设执行 “w=fmin(u,v);” 之前, (esp)=0x00dffd60

- ① 在表格的适当位置填写 刚进入函数 fmin 内部时, 堆栈中存放的相关数据。
- ② 刚进入函数 fmin 内部时, (esp)= 0x_____。
- ③ 执行 “add esp, 8” 之后, (esp)= 0x_____。

4. 函数 fmin 的指令解读 (8 分, 每小题 2 分)

函数体对应的反汇编代码有:

```

        push    ebp
        mov     ebp, esp
        ..... 此处有一些堆栈操作 push 指令, 但无改变 ebp的指令
        if (x > y)  t = y;
009A20A3  mov  eax,dword ptr [ebp+8]
009A20A6  cmp  eax,dword ptr [ebp+0Ch]
009A20A9  jle  009A20B3
009A20AB  mov  eax,dword ptr [ebp+0Ch]
009A20AE  mov  dword ptr [ebp-4],eax
009A20B1  jmp  009A20B9
        else  t = x;
009A20B3  .....
009A20B9  .....

```

- ① 函数参数 x 的地址 (即&x) 是 0x_____。
- ② 局部变量 t 的地址 (即&t) 是 0x_____。
- ③ 执行 “cmp eax, dword ptr [ebp+0Ch] ” 后, CF=____, SF=____, ZF=____, OF=____。
- ④ 在函数的结束处, 有程序段

```

_____
_____
ret

```

执行后可返回到调用函数的断点处。

分 数	
评卷人	

三、程序阅读与理解 （共 20 分）

1、阅读下面的程序，回答问题 （10 分）。

```
.686P
.model      flat, c
includelib  libcmnt.lib
includelib  legacy_stdio_definitions.lib
printf      proto  :ptr sbyte, :vararg
ExitProcess proto stdcall :dword
.data
    buf  sdword  -100, 2022, -32, 90, -30
    len  = ($-buf)/4
    fmt  db  "%d ", 0dh, 0ah, 0
.code
main proc c
    mov  eax, 0
    mov  ecx, len          ; ①
    mov  edi, offset buf   ; ②
LP_START:
    cmp  dword ptr [edi], 0
    jle  LP_NEXT          ; ③
    inc  eax
LP_NEXT:
    add  edi, 4
    sub  ecx, 1
    jne  LP_START
    invoke printf, offset fmt, eax
    invoke ExitProcess, 0
main endp
end
```

- 1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？（2 分）
统计 buf 数组中正数的个数到 eax 中，然后显示正数的个数。运行后显示 2。
- 2) 若将 ③ 处的语句改为 “jbe LP_NEXT”，程序运行的结果是什么？(2 分)
显示 5
- 3) 若标号 LP_START 写到 ②处语句前，程序运行的结果是什么？为什么？（3 分）
显示 0。循环中始终比较 buf 的开头元素（0 号元素）是否小于等于 0，该值小于 0。
- 4) 若标号 ①处语句写到标号 LP_START 之后，程序运行会出现什么现象？为什么？（3 分）
LP_START: mov ecx, len cmp dword ptr [edi], 0 ……

程序运行异常。表面上是死循环，每执行一次循环，ecx 都恢复成了 len。但是，由于循环中, edi 在不断地增加，到 edi 指向的地址超出程序空间范围时，执行 “cmp dword ptr [edi],

解答内容不得超过装订线

0”会出现访问异常，引起程序异常终止。

2. 程序填空（10 分，每空 1 分）

将一个以 0 结束的字符串中的所有小写字母转换为对应的大写字母，并将转换后的字符串、以及修改的字母个数输出。

```
.....  
.data  
    fmt db "%s %d", 0  
    buf db "Hello, 123, Good", 0  
.code  
main proc c  
    mov esi, 0 ; esi 存放 buf 数组元素的下标  
    mov ecx, 0 ; ecx 存放修改字母的个数  
start:  
    mov al, buf[esi]  
    cmp al, 0  
    je over / jz over         
    cmp al, 'a'  
    jl next / jb next         
    cmp al, 'z'  
    jg next / ja next         
    add al, 'A' - 'a'  
    mov buf[esi], al  
    inc ecx         
next:  
    inc esi         
    jmp start  
over:  
    invoke printf, offset fmt, offset buf, ecx  
    invoke ExitProcess, 0  
main endp  
end
```

分 数	
评卷人	

四、程序优化 （共 20 分）。

1、对 C 程序进行编译时，编译器可以做优化工作。请举出 5 个不同的优化场景，说明做了什么优化，以及能加快执行速度的原因。（10 分）

① 访问二维数组时，将列序优先调整为行序优先。二维数组是按行序存放的，在访问数据时，数据会成块地调入 CPU 的 cache 中，行序访问能够提高 cache 的命中率，避免不停地在内存和 cache 之间交换数据，从而提高执行速度。

② 用循环访问一维数组时，将循环展开，即变成无循环的语句，或者循环次数少的语句。因为 CPU 中采用指令流水线的方式，转移指令会导致流水线上的一些指令加工工作被废弃。若是顺序执行指令，则会充分发挥指令流水线的作用，提高速度。

③ 分支语句向无分支语句转换，例如使用条件传送指令代替转移指令。原理同②。

④ 用执行快的指令代替执行慢的指令。例如 $x*2$ ，用 $x<<1$ 来代替。

CPU 中的串操作指令、SIMD 指令，都可以用来加快程序的执行速度。

⑤ 执行算法的优化，例如 $3*x$ ，可以变成 $2*x+x$ ；而 $2*x$ 又可以使用逻辑左移 1 位指令来实现。虽然，指令条数变多，但速度会更快；原理就是 CPU 执行指令的速度不一样，一条慢的指令比几条快的指令耗时更多。

上述优化是与计算机硬件设备相关的。其他优化包括软件层面的优化，如编译器就可以计算出值的表达式，可以直接得到结果，而不需要生成相应的机器指令序列。

2、如下的 C 语言程序段的功能，其编译后调试版本的汇编语言代码如下(注：在反汇编时，勾选显示符号名)。（10 分）

```

for (i = 0; i < 5; i++)    sum += array[i];
00862129  mov     dword ptr [i],0
00862130  jmp     foft+5Bh (086213Bh)
00862132  mov     eax,dword ptr [i]
00862135  add     eax,1
00862138  mov     dword ptr [i],eax
0086213B  cmp     dword ptr [i],5
0086213F  jge     foft+70h (0862150h)
00862141  mov     eax,dword ptr [i]
00862144  mov     ecx,dword ptr [sum]
00862147  add     ecx,dword ptr array[eax*4]
0086214B  mov     dword ptr [sum],ecx
0086214E  jmp     foft+52h (0862132h)

```

(1) 编写相应的优化汇编语言程序段，以提高程序的执行效率。（6 分）

要求优化时保留循环结构，在优化程序段中可以用标号来代替转移指令的目的地址，要求写出寄存器的分配（即与变量的绑定关系）。

```

Ebx 存放变量 I 的值;    eax 存放变量 sum 的值    (1 分)
mov  eax, sum            (sum 的值赋给对应的寄存器 1 分)
mov  ebx, 0
loop_p:
  cmp  ebx, 5            循环控制 1 分; 加法 1 分
  jge  loop_over        寄存器修改 1 分
  add  eax, dword ptr array[ebx*4]
  inc  ebx
  jmp  loop_p
loop_over:
  mov  dword ptr [sum], eax    (寄存器的值送给 sum 1 分)
  mov  i, ebx

```

(2) 用循环展开的方法（即去除循环）优化程序段（4 分）。

```

eax 存放变量 sum 的值
mov  eax, sum            sum 与寄存器交换数据正确 1 分
                                累加求和正确 2 分; 最简表达形式 1 分

add  eax, array[0]
add  eax, array[4]
add  eax, array[8]
add  eax, array[12]
add  eax, array[16]
mov  sum, eax

```

分 数	
评卷人	

五、程序的链接问答（10 分）

- 1、在对一个 C 程序文件进行编译和汇编后，生成的可重定位目标文件(obj 文件，或者 .o 文件)，一般由哪些节组成（至少说出 6 个组成部分的名字）？（3 分）
文件头、节头表、代码节 .text, 已初始化数据节 .data, 未初始化数据节 .bss, 只读数据节 .rodata, 符号表节 .symtab, 字符串表节 .strtab、代码节的重定位节(.rela.text)、数据节的重定位节(.rela.data) 等等。
- 2、C 程序中，什么符号需要重定位？什么符号不需要重定位？（2 分）
函数参数、非静态的局部变量不需要重定位；静态的局部变量、全局变量、函数需要重定位。

- 3、设有一个函数中，有语句 `int x=g;` 其中 `g` 是一个初值为 20 的 `int` 类型全局变量。编译器对 `g` 的定义(`int g=20;`)和 `x=g` 编译时，在可重定位目标文件中会生成哪些信息？（5 分）
- 对于 `int g=20;` 在字符串节中，要存放变量的名字(ASCII 串)；在符号表节要存放与 `g` 相关的信息（如 `g` 的值占存储空间的大小<4>；定义 `g` 的节<数据节>；`g` 的类型<变量>；绑定方式<global>等等）；在数据节要存放 `g` 的初值 20。
- 对于 `int x=g;` 在代码节要生成相应的指令，其中 对于 `g` 的引用（即 `g` 的地址），先使用占位符（00 00 00 00）来填充。在代码节的重定位节，要产生一条相应的信息，表明代码节的什么位置要被替换成一个什么类型的值（重定位方式），该值来源于哪一个变量（符号表的哪一项）等。

22秋A卷答案

2026年5月27日 20:36



22秋A卷答
案



华中科技大学计算机科学与技术学院 2022~2023 第一学期

“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试日期 2022-12-04 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四	五	总分	核对人
分值	20	30	20	20	10	100	
得分							

分 数	
评卷人	

一、计算机工作基本原理填空（共 20 分，每空 1 分）

解
答
内
容
不
得
超
过
装
订
线

1、计算机采用存储程序的工作方式，自动逐条取出和执行指令。指令在内存中的存储地址由程序计数器指明（Intel CPU 中为指令指示器 EIP 或者 RIP，下面以 EIP 为例）。EIP 自动变化的规则和变化方式如下：

① 在取出指令、对指令进行译码后，EIP 会自动加上 当前指令的字节长度；若该指令不涉及转移，则可以实现程序的顺序执行；

② 若指令是条件转移指令，如 JE LP (LP 为一个标号)，在 ZF=1 时，会执行 (EIP) +位移量 → EIP，其实质操作是将 LP 的偏移量 送到 EIP 中；

③ 若指令是无条件转移指令 JMP，除了直接跳转外，还可以进行间接跳转。借助于寄存器 EAX，通过间接跳转实现与“JMP LP”相同的功能，可以使用语句：

JMPEAX

④ 借助堆栈段和子程序返回指令，实现与“JMP LP”相同的功能，可以使用语句：

PUSH LP
RET

⑤ 执行 CALL 指令时，CPU 会 将返回地址（EIP 的下一条指令地址）压栈，并跳转到目标地址；

⑥ 在程序运行过程中，出现中断信号时，CPU 会根据中断类型号，利用 IDT 找到中断处理程序的入口地址。中断处理程序中有 IRET 指令，从栈中取出进入中断处理程

序前保存的断点地址送给 EIP，从而实现被中断程序的继续执行。

2、机器指令一般都有操作对象，指令中可以直接存放常量操作数或者操作数的地址。操作数的地址有多种表达方式。下面一段代码给出了访问同一单元的多种 C 语句。请回答 C 语句访问方式与机器指令的对应关系。

```
int global[5] = { 10,20,30,40,50 }; // 全局变量

void func ()
{
    int local[5] = { 10,20,30,40,50 };
    int i = 2;
    int *p;
    global[i] = 25;
    *(global + 2) = 25;
    p = &global[2];
    *p = 25;
    local[i] = 25;
}
```

① 在反汇编窗口看到 global[i]=25 对应如下两条指令：

```
mov    eax, dword ptr [ebp-18h]    ; 显示符号名时的语句为 mov eax, dword ptr [i]
mov    dword ptr [eax*4+00419000h],19h
```

第一条指令中变量 i 的地址表达式为 _____ ；第二条指令中目的操作数的寻址方式是 _____ ，数组 global 的起始地址是 _____ ，目的操作数寻址中采用比例因子 4 的原因是 _____ 。

② $*(global + 2) = 25$; 仅对应一条机器指令，指令为 _____
其目的操作数的寻址方式是 _____ 。

③ $*p = 25$; 如果变量 p 的地址表达式为 [ebp-1Ch]，访问存储单元(*p)时，使用的是寄存器间接寻址方式（使用寄存器 eax），则对应的两条汇编指令为：

```
_____  
_____
```

④ local[i]=25; 访问 local[i] 采用的是基址加变址寻址方式，对应的汇编指令如下：

```
mov    _____, dword ptr [ebp-18h]
mov    dword ptr [ebp+eax*4-14h],19h
```

与全局变量 global 空间分配位置不同，local 的空间分配在栈上。

数组 local 的起始地址的表达式为：_____。

分 数	
评卷人	

二、数据存储及 C 语句转换填空（共 30 分）

在Windows + VS2019（Intel CPU）环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fmin(int x, int y)
{
    int t;
    if (x > y) t= y;
    else t = x;
    return t;
}

void fmain( )
{
    char buf[6] = "12345";    // '1'的 ASCII 是 0x31
    int u = 0x20221204;        // mov dword ptr [ebp-0Ch], 20221204h
    int v = -32;                // mov dword ptr [ebp-10h], 0FFFFFFE0h
    int w = 0;                  // mov dword ptr [ebp-14h], 0
    w = fmin(u, v);
    w = *(int *) (buf+1);
}
```

在调试时，在监视窗口中看到数组 buf 的起始地址为 0x00dffdb8，变量 u 的地址（即&u）为 0x00dffdb4； v 的地址(即&v) 为 0x00dffdb0。

1、填写执行到“w=fmin(u,v);”时，内存中数据存放的结果。要求以字节为单位、用 16 进制数的形式填空，最左边是内存窗口显示的内存地址。（6 分）

2、0x00dffdb0 _____

3、0x00dffdb8 _____ XX XX

4、数据传送指令解读（6 分，每空 2 分）

语句“int u = 0x20221204;”的反汇编指令为：mov dword ptr [ebp-0Ch], 20221204h

根据观察到的 u 的地址，推断执行该语句时，(ebp)= 0x _____，该指令的机器码为 C7

45 F4 04 12 22 20，其中 F4 对应的是指令中_____的编码。

执行“w = *(int *) (buf+1);”后，w 中的值为 0x_____。

5、函数调用语句解读（10 分，每空 2 分）

对于语句“w=fmin(u, v);”对应的反汇编代码（最左边的是机器指令的地址）如下。

```

009A204B  mov  eax,dword ptr [ebp-10h]
009A204E  push  eax
009A204F  mov  ecx,dword ptr [ebp-0Ch]
009A2052  push  ecx
009A2053  call  009A125D
009A2058  add   esp,8
009A205B  mov  dword ptr [ebp-14h],eax

```

	0x00dffd4c
	0x00dffd50
	0x00dffd54
	0x00dffd58
	0x00dffd5c
XXXXXXXX	0x00dffd60

设执行 “w=fmin(u,v);” 之前, (esp)=0x00dffd60

- ① 在表格的适当位置填写 刚进入函数 fmin 内部时, 堆栈中存放的相关数据。
- ② 刚进入函数 fmin 内部时, (esp)= 0x_____。
- ③ 执行 “add esp, 8” 之后, (esp)= 0x_____。

4. 函数 fmin 的指令解读 (8 分, 每小题 2 分)

函数体对应的反汇编代码有:

```

        push  ebp
        mov   ebp, esp
        ..... 此处有一些堆栈操作 push 指令, 但无改变 ebp的指令
        if (x > y)  t = y;
009A20A3  mov  eax,dword ptr [ebp+8]
009A20A6  cmp  eax,dword ptr [ebp+0Ch]
009A20A9  jle  009A20B3
009A20AB  mov  eax,dword ptr [ebp+0Ch]
009A20AE  mov  dword ptr [ebp-4],eax
009A20B1  jmp  009A20B9
        else  t = x;
009A20B3  .....
009A20B9  .....

```

- ① 函数参数 x 的地址 (即&x) 是 0x_____。
- ② 局部变量 t 的地址 (即&t) 是 0x_____。
- ③ 执行 “cmp eax,dword ptr [ebp+0Ch] ” 后, CF=____,SF=____,ZF=____,OF=_____。
- ④ 在函数的结束处, 有程序段

```

_____
_____
ret

```

执行后可返回到调用函数的断点处。

分 数	
评卷人	

三、程序阅读与理解 （共 20 分）

1、阅读下面的程序，回答问题 （10 分）。

```
.686P
.model      flat, c
includelib  libcmnt.lib
includelib  legacy_stdio_definitions.lib
printf      proto  :ptr sbyte, :vararg
ExitProcess proto stdcall :dword
.data
    buf  sdword  -100, 2022, -32, 90, -30
    len  = ($-buf)/4
    fmt  db  "%d ", 0dh, 0ah, 0
.code
main proc c
    mov  eax, 0
    mov  ecx, len          ; ①
    mov  edi, offset buf   ; ②
LP_START:
    cmp  dword ptr [edi], 0
    jle  LP_NEXT          ; ③
    inc  eax
LP_NEXT:
    add  edi, 4
    sub  ecx, 1
    jne  LP_START
    invoke printf, offset fmt, eax
    invoke ExitProcess, 0
main endp
end
```

- 1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？（2 分）
统计 buf 数组中正数的个数到 eax 中，然后显示正数的个数。运行后显示 2。
- 2) 若将 ③ 处的语句改为 “jbe LP_NEXT”，程序运行的结果是什么？(2 分)
显示 5
- 3) 若标号 LP_START 写到 ②处语句前，程序运行的结果是什么？为什么？（3 分）
显示 0。循环中始终比较 buf 的开头元素（0 号元素）是否小于等于 0，该值小于 0。
- 4) 若标号 ①处语句写到标号 LP_START 之后，程序运行会出现什么现象？为什么？（3 分）
LP_START: mov ecx, len cmp dword ptr [edi], 0 ……
程序运行异常。表面上是死循环，每执行一次循环，ecx 都恢复成了 len。但是，由于循环中, edi 在不断地增加，到 edi 指向的地址超出程序空间范围时，执行 “cmp dword ptr [edi],

0”会出现访问异常，引起程序异常终止。

2. 程序填空（10 分，每空 1 分）

将一个以 0 结束的字符串中的所有的小写字母转换为对应的大写字母，并将转换后的字符串、以及修改的字母个数输出。

```
.....  
.data  
    fmt db "%s %d", 0  
    buf db "Hello, 123, Good", 0  
.code  
main proc c  
    mov esi, 0 ; esi 存放 buf 数组元素的下标  
    mov ecx, 0 ; ecx 存放修改字母的个数  
start:  
    mov al, buf[esi]  
    cmp al, 0  
    je over / jz over         
    cmp al, 'a'  
    jl next / jb next         
    cmp al, 'z'  
    jg next / ja next         
    add al, 'A' - 'a'  
    mov buf[esi], al  
    inc ecx         
next:  
    inc esi         
    jmp start  
over:  
    invoke printf, offset fmt, offset buf, ecx  
    invoke ExitProcess, 0  
main endp  
end
```

分 数	
评卷人	

四、程序优化 （共 20 分）。

1、对 C 程序进行编译时，编译器可以做优化工作。请举出 5 个不同的优化场景，说明做了什么优化，以及能加快执行速度的原因。（10 分）

① 访问二维数组时，将列序优先调整为行序优先。二维数组是按行序存放的，在访问数据时，数据会成块地调入 CPU 的 cache 中，行序访问能够提高 cache 的命中率，避免不停地在内存和 cache 之间交换数据，从而提高执行速度。

② 用循环访问一维数组时，将循环展开，即变成无循环的语句，或者循环次数少的语句。因为 CPU 中采用指令流水线的方式，转移指令会导致流水线上的一些指令加工工作被废弃。若是顺序执行指令，则会充分发挥指令流水线的作用，提高速度。

③ 分支语句向无分支语句转换，例如使用条件传送指令代替转移指令。原理同②。

④ 用执行快的指令代替执行慢的指令。例如 $x*2$ ，用 $x<<1$ 来代替。

CPU 中的串操作指令、SIMD 指令，都可以用来加快程序的执行速度。

⑤ 执行算法的优化，例如 $3*x$ ，可以变成 $2*x+x$ ；而 $2*x$ 又可以使用逻辑左移 1 位指令来实现。虽然，指令条数变多，但速度会更快；原理就是 CPU 执行指令的速度不一样，一条慢的指令比几条快的指令耗时更多。

上述优化是与计算机硬件设备相关的。其他优化包括软件层面的优化，如编译器就可以计算出值的表达式，可以直接得到结果，而不需要生成相应的机器指令序列。

2、如下的 C 语言程序段的功能，其编译后调试版本的汇编语言代码如下(注：在反汇编时，勾选显示符号名)。（10 分）

```

for (i = 0; i < 5; i++)    sum += array[i];
00862129  mov     dword ptr [i],0
00862130  jmp     foft+5Bh (086213Bh)
00862132  mov     eax,dword ptr [i]
00862135  add     eax,1
00862138  mov     dword ptr [i],eax
0086213B  cmp     dword ptr [i],5
0086213F  jge     foft+70h (0862150h)
00862141  mov     eax,dword ptr [i]
00862144  mov     ecx,dword ptr [sum]
00862147  add     ecx,dword ptr array[eax*4]
0086214B  mov     dword ptr [sum],ecx
0086214E  jmp     foft+52h (0862132h)

```

(1) 编写相应的优化汇编语言程序段，以提高程序的执行效率。（6 分）

要求优化时保留循环结构，在优化程序段中可以用标号来代替转移指令的目的地址，要求写出寄存器的分配（即与变量的绑定关系）。

```

Ebx 存放变量 I 的值；    eax 存放变量 sum 的值    (1 分)
mov  eax, sum            (sum 的值赋给对应的寄存器 1 分)
mov  ebx, 0
loop_p:
  cmp  ebx, 5            循环控制 1 分；加法 1 分
  jge  loop_over         寄存器修改 1 分
  add  eax, dword ptr array[ebx*4]
  inc  ebx
  jmp  loop_p
loop_over:
  mov  dword ptr [sum], eax    (寄存器的值送给 sum 1 分)
  mov  i, ebx

```

(2) 用循环展开的方法（即去除循环）优化程序段（4 分）。

```

eax 存放变量 sum 的值
mov  eax, sum            sum 与寄存器交换数据正确 1 分
                                累加求和正确 2 分； 最简表达形式 1 分

add  eax, array[0]
add  eax, array[4]
add  eax, array[8]
add  eax, array[12]
add  eax, array[16]
mov  sum, eax

```

分 数	
评卷人	

五、程序的链接问答（10 分）

- 1、在对一个 C 程序文件进行编译和汇编后，生成的可重定位目标文件(obj 文件，或者 .o 文件)，一般由哪些节组成（至少说出 6 个组成部分的名字）？（3 分）
文件头、节头表、代码节 .text, 已初始化数据节 .data, 未初始化数据节 .bss, 只读数据节 .rodata, 符号表节 .symtab, 字符串表节 .strtab、代码节的重定位节(.rela.text)、数据节的重定位节(.rela.data) 等等。
- 2、C 程序中，什么符号需要重定位？什么符号不需要重定位？（2 分）
函数参数、非静态的局部变量不需要重定位；静态的局部变量、全局变量、函数需要重定位。

- 3、设有一个函数中，有语句 `int x=g;` 其中 `g` 是一个初值为 20 的 `int` 类型全局变量。编译器对 `g` 的定义(`int g=20;`)和 `x=g` 编译时，在可重定位目标文件中会生成哪些信息？（5 分）
- 对于 `int g =20;` 在字符串节中，要存放变量的名字(ASCII 串)；在符号表节要存放与 `g` 相关的信息（如 `g` 的值占存储空间的大小<4>；定义 `g` 的节<数据节>；`g` 的类型<变量>；绑定方式<global>等等）；在数据节要存放 `g` 的初值 20。
- 对于 `int x=g;` 在代码节要生成相应的指令，其中 对于 `g` 的引用（即 `g` 的地址），先使用占位符（00 00 00 00）来填充。在代码节的重定位节，要产生一条相应的信息，表明代码节的什么位置要被替换成一个什么类型的值（重定位方式），该值来源于哪一个变量（符号表的哪一项）等。

23秋A卷

2026年5月28日 9:58



23秋A卷



华中科技大学计算机科学与技术学院 2023~2024 第一学期

“ 计算机系统基础 ” 考试试卷(A 卷)

考试方式 闭卷 考试日期 2023-12-03 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四	五	总分	核对人
分值	20	30	20	10	20	100	
得分							

分 数	
评卷人	

一、计算机工作基本原理填空（共 20 分）

解
答
内
容
不
得
超
过
装
订
线

1、在对某程序进行调试时，看到如下一段信息（10 分）：

```
004116DA 83 7D FC 00      cmpl .....
004116DE 75 08          jne  004116E8
004116E0 8B 45 FC          mov  .....
004116E3 89 45 F8          mov  .....
004116E6 EB 07          jmp  004116EF
004116E8 C7 45 F8 01 00 00 00 movl .....
004116EF E8 2D FB FF FF    callq func (00411221) ; func 为函数名
004116F4 C7 45 F4 21 12 41 00 movl $0x411221, -0x0c(%ebp)
004116FB FF 55 F4          callq -0x0c(%ebp)
.....
```

① 观察指令在内存中的存放形式(每个空对应一个 16 进制字节数据)（2 分）：

004116DA _____

004116E2 _____

② 设当前 eip 为 0x004116DE，在取出 eip 指向的指令并进行译码后，eip = _____；

③ 在 0x004116DE 处的指令为： 75 08 jne 004116E8，直观上是在 zf = _____ 时，会将 0x004116E8 → eip，计算出该地址值的方法 _____；当转移条件不成立时，则该指令执行完成， eip _____（会、不会）改变。

④ 004116EF 处的指令中有 0xFFFFFB2D，计算出该值的方法是_____；

⑤ 004116FB 处的子程序调用指令，调用的子程序的入口地址是 _____，执行该处语句时，CPU 会将 0x_____压入堆栈中。

⑥ 执行子程序中的 RET 指令时，CPU 会_____ → eip。

2、设一个 C 语言程序中定义有全局数组 int array[5]； array 的起始地址(即&array[0])为 0x420100，每个元素占 4 个字节，现要将数组的第 2 个元素(即 array[2])置为 20。按指定的寻址方式写出实现该功能的机器指令段(汇编语句段)。（10 分，每空 1 分）

注：完成该功能的 C 语句有：“array[2]=20; *(array +2) =20; int i=2; array[i]=20; int *p=array; p[i] = 20;”等等。不同的 C 语句写法编译后，生成的机器指令不同，会出现多种寻址方式访问同一单元。

① 使用直接寻址方式，机器指令中含有操作数的偏移地址：

② 使用寄存器间接寻址方式，先将操作数的地址送 `ebx`，再访存：
_____；给 `ebx` 赋值

③ 使用变址寻址方式，先将元素的下标（即 2）送 `eax`，再访存：
_____；给 `eax` 赋值

④ 使用基址加变址寻址方式，先将数组的起始地址送 `eax`，第 2 个元素在数组中的偏移字节数送 `ebx`，再访存：

_____；给 `eax` 赋值
_____；给 `ebx` 赋值

⑤ 对于全局变量，其空间分配在_____。而对于非静态的局部变量，其空间分配在_____，对应的地址表达式一般为 `disp(%ebp)`、`disp(%esp)` 等。

分 数	
评卷人	

二、数据存储及 C 语句转换填空（共 30 分）

在Linux环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fadd(int a, int b)
{
    int temp;
    temp = a + b;
    return temp;
}

void main( )
{
    int x = 0x1234;
    int y = -32;
    int result = 0;
    char msg[6] = "abc12"; // '1'的ASCII是 0x31, 'a'的ASCII是 0x61
    result = fadd(x, y);
    result = *(int *) (msg+1);
}
```

调试时，设变量 `x` 的地址（即`&x`）为 `0xffffd508`；`y` 的地址（即`&y`）为 `0xffffd50c`，`result` 的地址为 `0xffffd510`，数组 `msg` 的起始地址为 `0xffffb516`。

1、执行到“`result=fadd(x,y);`”时，以字节为单位观察内存内容（用 16 进制数的形式填空，最左边是内存地址）。（6 分）

<code>0xffffd508</code>	_____	_____	_____	_____	_____	_____	_____
<code>0xffffd510</code>	_____	_____	_____	_____	XX	XX	_____
<code>0xffffd518</code>	_____	_____	_____	_____	XX	XX	XX XX

2、数据传送指令解读（6 分，每空 2 分）

“`int x = 0x1234;`”对应的机器指令为：`movl $0x1234, -0x20(%ebp)`，执行该语句时，`ebp=0x_____`。

“`int result = 0;`”对应的反汇编指令为_____。

执行“`result = *(int *) (msg+1);`”后，`result` 中的值为 `0x_____`。

3、函数调用语句解读（10 分，每空 2 分）

语句“`result = fadd(x, y);`”对应的反汇编代码（最左边的是机器指令的地址）如下。

```

0x56556210 <+72>: push -0x1c(%ebp)
0x56556213 <+75>: push -0x20(%ebp)
0x56556216 <+78>: call 0x5655619d <fadd>
0x5655621b <+83>: add $0x8, %esp
0x5655621e <+86>: mov %eax, -0x18(%ebp)
    
```

	0xffffd4ec
	0xffffd4f0
	0xffffd4f4
0x5655621b	0xffffd4f8
-0x20(%ebp)	0xffffd4fc
-0x1c(%ebp)	0xffffd500
XXXXXXXXXX	

设执行“result = fadd(x, y);”之前，esp 的值为 0xffffd500。

- ① 在表格的适当位置填写刚进入函数 fadd 内部时，堆栈中存放的相关数据（6 分）；
- ② 刚进入函数 fadd 内部时，esp = 0x ffffd4f0；
- ③ 执行“add \$0x8, %esp”之后，esp = 0x f8 (?)

4. 函数 fadd 的指令解读（8 分，每小题 2 分）

函数体对应的反汇编代码有：

```

0x5655619d <+0>: push %ebp
0x5655619e <+1>: mov %esp, %ebp
0x565561a0 <+3>: sub $0x10, %esp
0x565561a3 <+6>: mov 0x8(%ebp), %edx
0x565561a6 <+9>: mov 0xc(%ebp), %eax
0x565561a9 <+12>: add %edx, %eax
0x565561ab <+14>: mov %eax, -0x4(%ebp)
0x565561ae <+17>: mov -0x4(%ebp), %eax
0x565561b1 <+20>: .....
    
```

- ① 函数参数 a 的地址（即&a）是 0x_____。
- ② 局部变量 temp 的地址（即&temp）是 0x_____。
- ③ 执行“add %edx, %eax”后，CF=____, SF=____, ZF=____, OF=_____。
- ④ 在函数的结束处，有程序段

ret

执行后可返回到调用函数的断点处。

分 数	
评卷人	

三、程序阅读与理解（共 20 分）

1、阅读下面的程序，回答问题（10 分，每小题 2 分）。

```

void func ()
{
    unsigned short us = 65535;
    unsigned int ui;
    short s = -1;
    int i;
    int x = 0, y = 0; ; ①
    ui = us;
    i = s; ; ②
    if (ui > 0) x = 1;
    if (i > 0) y = 1; ; ③
    ui = ui >> 1; ; >>1 右移一个二进制位
    i = i >> 1; ; ④
    ui = ~ui; ; 按位取反
    i = -i; ; ⑤
}
    
```

(1) 执行完 ① 处语句后,

ui = 0x _____ s = 0x _____ (2 个字节的 16 进制数)

(2) 执行完 ② 处语句后,

ui = 0x _____ i = 0x _____ (4 个字节的 16 进制数)

ui=us; 对应的机器指令: _____ us, %eax 、 mov %eax, ui

i=s; 对应的机器指令: _____ s, %eax 、 mov %eax, i

(3) 执行完 ③ 处语句后,

x = _____ y = _____

if (ui>0) x=1; 对应的机器指令: cmp \$0, ui、 _____ lp1、 mov \$1, x、 lp1:...

if (i >0) y=1; 对应的机器指令: cmp \$0, i、 _____ lp2、 mov \$1, y、 lp2:...

(4) 执行完 ④ 处语句后,

ui = 0x _____ i = 0x _____ (4 个字节的 16 进制数)

ui = ui >> 1; 对应的机器指令: _____ \$1, ui

i = i >> 1; 对应的机器指令: _____ \$1, i

(5) 执行完 ⑤ 处语句后,

ui = 0x _____ i = 0x _____ (4 个字节的 16 进制数)

ui = ~ui; 对应的机器指令: _____ ui

i = -i; 对应的机器指令: _____ i

2、阅读下面的程序, 回答问题 (10 分)。

```
.section .data
    array: .long 10, -20, 30, -40, 50
    length = (. -array)/4          # length 为array中元数的个数, = 5
    format: .ascii "%d\n\0"

.section .text
.global _start
_start:
    mov $0, %eax
    mov $length, %ecx
    lea array, %edi                # ①
lp_1:
    cmpl $0, (%edi)
    jl lp_2                        # ②
    inc %eax
lp_2:
    add $4, %edi
    sub $1, %ecx                   # ③
    jne lp_1
    push %eax
    push $format
    call printf
```

解答内容不得超过装订线

```
mov $1, %eax    # 程序正常退出
mov $0, %ebx
int $0x80
```

- 1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？（2分）
- 2) 若标号 lp_1 写到 ①处语句前，程序运行的结果是什么？为什么？（3分）
- 3) 若将 ② 处的语句改为 “jb lp_2”, 程序运行的结果是什么？(2分)
- 4) 若漏写了 ③ 处的语句，程序运行会出现什么现象？为什么？（3分）

分 数	
评卷人	

四、程序优化（共 10 分）。

1、举例说明编写 C 程序或者编译器优化时利用相应原则进行优化的做法（包括优化前的方法，优化后的方法）。（10 分，每小题 2 分)

- ① 提高 CPU 中 cache 的命中率
- ② 提高 CPU 中指令流水线的利用率
- ③ 使用 CPU 中处理速度更快的指令
- ④ 使用 CPU 中单指令多数据流指令或串操作指令
- ⑤ 使用多核 CPU 中多线程处理能力

分 数	
评卷人	

五、链接和异常控制流问答（20 分）

1、设一个函数中有语句 int temp=global; 其中 global 是一个初值为 35 的 int 类型全局变量。编译器对 global 的定义(int global=35;)和 temp=global 编译时，分别会在可重定位目标文件中的哪些节生成哪些信息？（10 分）

2、什么是中断和异常？两者有何差别？什么是中断描述符表？中断和异常的响应过程是什么？（10 分）

23秋A卷答案

2026年5月28日 9:58



23秋A卷答
案



华中科技大学计算机科学与技术学院 2023~2024 第一学期

“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试日期 2023-12-03 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四	五	总分	核对人
分值	20	30	20	10	20	100	
得分							

分 数	
评卷人	

一、计算机工作基本原理填空（共 20 分）

1、在对某程序进行调试时，看到如下一段信息（10 分）：

```
004116DA  83 7D FC 00          cmpl  .....
004116DE  75 08                  jne   004116E8
004116E0  8B 45 FC              mov   .....
004116E3  89 45 F8              mov   .....
004116E6  EB 07                 jmp   004116EF
004116E8  C7 45 F8 01 00 00 00  movl  .....
004116EF  E8 2D FB FF FF       callq func (00411221) ; func 为函数名
004116F4  C7 45 F4 21 12 41 00  movl  $0x411221, -0x0c(%ebp)
004116FB  FF 55 F4              callq -0x0c(%ebp)
.....
```

① 观察指令在内存中的存放形式(每个空对应一个 16 进制字节数据)（2 分）：

004116DA 83 7D FC 00 75 08 8B 45
004116E2

② 设当前 eip 为 0x004116DE，在取出 eip 指向的指令并进行译码后，eip = 0x004116E0；

③ 在 0x004116DE 处的指令为：75 08 jne 004116E8，直观上是在 zf = 0 时，会将 0x004116E8 → eip，计算出该地址值的方法 0x004116E0(下一条指令的地址) + 08；当

解
答
内
容
不
得
超
过
装
订
线

转移条件不成立时，则该指令执行完成，eip 不会（会、不会）改变。

④ 004116EF 处的指令中有 0xFFFFFB2D，计算出该值的方法是 00411221 - 004116F4；

⑤ 004116FB 处的子程序调用指令，调用的子程序的入口地址是 0x411221，执行该处语句时，CPU 会将 0x 004116FE 压入堆栈中。

⑥ 执行子程序中的 RET 指令时，CPU 会 从栈顶弹出一个双字 → eip。

2、设一个 C 语言程序中定义有全局数组 int array[5]； array 的起始地址(即 &array[0])为 0x420100，每个元素占 4 个字节，现要将数组的第 2 个元素(即 array[2])置为 20。按指定的寻址方式写出实现该功能的机器指令段（汇编语句段）。（10 分，每空 1 分）

注：完成该功能的 C 语句有：“array[2]=20; *(array +2) =20; int i=2; array[i]=20; int *p=array; p[i] = 20;”等等。不同的 C 语句写法编译后，生成的机器指令不同，会出现多种寻址方式访问同一单元。

① 使用直接寻址方式，机器指令中含有操作数的偏移地址：

movl \$20, 0x420108 或 mov dword ptr [420108h], 20

② 使用寄存器间接寻址方式，先将操作数的地址送 ebx，再访存：

lea 0x420108, %ebx 或 lea ebx, dword ptr [420108h]；给 ebx 赋值

movl \$20, (%ebx) 或 mov dword ptr [ebx], 20

③ 使用变址寻址方式，先将元素的下标（即 2）送 eax，再访存：

movl \$2, %eax 或 mov eax, 2；给 eax 赋值

movl \$20, 0x420100(, %eax, 4) 或 mov dword ptr [420100h+eax*4], 20

④ 使用基址加变址寻址方式，先将数组的起始地址送 eax，第 2 个元素在数组中的偏移字节数送 ebx，再访存：

lea 0x420100, %eax 或者 lea eax, array；给 eax 赋值

movl \$8, %ebx 或者 mov ebx, 8；给 ebx 赋值

movl \$20, (%eax, %ebx, 1) 或者 mov dword ptr [eax+ebx], 20

⑤ 对于全局变量，其空间分配在 数据段。而对于非静态的局部变量，其空间分配在 堆栈段，对应的地址表达方式一般为 disp(%ebp)、disp(%esp) 等。

分 数	
评卷人	

二、数据存储及 C 语句转换填空（共 30 分）

在Linux环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fadd(int a, int b)
{
    int temp;

    temp = a + b;

    return temp;
}

void main( )
{
    int x = 0x1234;

    int y = -32;

    int result = 0;

    char msg[6] = "abc12";    // '1'的 ASCII 是 0x31, 'a'的 ASCII 是 0x61

    result = fadd(x, y);

    result = *(int *) (msg+1);
}
```

解答内容不得超过装订线

调试时，设变量 x 的地址（即&x）为 0xffffd508； y 的地址(即&y) 为 0xffffd50c, result 的地址为 0xffffd510，数组 msg 的起始地址为 0xffffb516。

1、执行到“result=fadd(x,y);”时，以字节为单位观察内存内容（用 16 进制数的形式填空，最左边是内存地址）。（6 分）

0xffffd508	<u>34</u>	<u>12</u>	<u>00</u>	<u>00</u>	<u>E0</u>	<u>FF</u>	<u>FF</u>	<u>FF</u>
0xffffd510	<u>00</u>	<u>00</u>	<u>00</u>	<u>00</u>	XX	XX	<u>61</u>	<u>62</u>
0xffffd518	<u>63</u>	<u>31</u>	<u>32</u>	<u>00</u>	XX	XX	XX	XX

2、数据传送指令解读 （6 分，每空 2 分）

“int x = 0x1234;”对应的机器指令为： movl \$0x1234, -0x20(%ebp)，执行该语句时， ebp= 0x ffffd528。

“int result = 0;”对应的反汇编指令为 movl \$0, -0x18(%ebp)。

执行 “result = *(int *) (msg+1);”后， result 中的值为 0x 32316362。

3、函数调用语句解读 （10 分，每空 2 分）

语句 “result = fadd(x, y);” 对应的反汇编代码（最左边的是机器指令的地址）如下。

0x56556210 <+72>:	push -0xc(%ebp)		0xffffd4ec
0x56556213 <+75>:	push -0x20(%ebp)		0xffffd4f0
0x56556216 <+78>:	call 0x5655619d <fadd>	0x5655621b	0xffffd4f4
0x5655621b <+83>:	add \$0x8, %esp	0x00001234	0xffffd4f8
0x5655621e <+86>:	mov %eax, -0x18(%ebp)	0xfffffe0 或 -32	0xffffd4fc
		XXXXXXXX	0xffffd500

设执行 “result = fadd(x, y);” 之前，esp 的值为 0xffffd500。

- ① 在表格的适当位置填写刚进入函数 fadd 内部时，堆栈中存放的相关数据（6 分）；
 - ② 刚进入函数 fadd 内部时，esp = 0x ffffd4f4 ；
 - ③ 执行 “add \$0x8, %esp” 之后，esp = 0x ffffd500 。
4. 函数 fadd 的指令解读（8 分，每小题 2 分）

函数体对应的反汇编代码有：

```
0x5655619d <+0>:    push %ebp
0x5655619e <+1>:    mov %esp, %ebp
0x565561a0 <+3>:    sub $0x10, %esp
0x565561a3 <+6>:    mov 0x8(%ebp), %edx
0x565561a6 <+9>:    mov 0xc(%ebp), %eax
0x565561a9 <+12>:   add %edx, %eax
0x565561ab <+14>:   mov %eax, -0x4(%ebp)
0x565561ae <+17>:   mov -0x4(%ebp), %eax
0x565561b1 <+20>:   .....
```

- ① 函数参数 a 的地址（即&a）是 0x ffffd4f8 。
- ② 局部变量 temp 的地址（即&temp）是 0x ffffd4ec 。
- ③ 执行 “add %edx, %eax” 后，CF= 1 , SF= 0 , ZF= 0 , OF= 0 。
- ④ 在函数的结束处，有程序段

```
mov %ebp, %esp
pop %ebp
ret
```

执行后可返回到调用函数的断点处。

分 数	
评卷人	

三、程序阅读与理解 （共 20 分）

1、阅读下面的程序，回答问题 （10 分，每小题 2 分）。

```
void func ()
{
    unsigned short us = 65535;
    unsigned int ui;
    short s = -1;
    int i;
    int x = 0, y = 0;           ; ①
    ui = us;
    i = s;                       ; ②
    if (ui > 0) x = 1;
    if (i > 0) y = 1;           ; ③
    ui = ui >> 1;               ; >>1 右移一个二进制位
    i = i >> 1;                 ; ④
    ui = ~ui;                   ; 按位取反
    i = -i;                     ; ⑤
}
```

(1) 执行完 ① 处语句后，

us = 0x ffff s = 0x ffff （2 个字节的 16 进制数）

(2) 执行完 ② 处语句后，

ui = 0x 0000ffff i = 0x ffffff （4 个字节的 16 进制数）

ui=us; 对应的机器指令: movzx 或 movz us,%eax 、 mov %eax, ui

i=s; 对应的机器指令: movsx 或 movs s,%eax 、 mov %eax, i

(3) 执行完 ③ 处语句后，

x = 1 y = 0

if (ui>0) x=1; 对应的机器指令: cmp \$0, ui、 jbe lp1、 mov \$1, x、 lp1:...

if (i >0) y=1; 对应的机器指令: cmp \$0, i、 jle lp2、 mov \$1, y、 lp2:...

(4) 执行完 ④ 处语句后，

ui = 0x 00007fff i = 0x ffffff （4 个字节的 16 进制数）

ui = ui >> 1; 对应的机器指令: shr \$1, ui

i = i >> 1; 对应的机器指令: sar \$1, i

(5) 执行完 ⑤ 处语句后，

ui = 0x ffff8000 i = 0x 00000001 （4 个字节的 16 进制数）

ui = ~ui; 对应的机器指令: not ui

i = -i; 对应的机器指令: neg i

2、阅读下面的程序，回答问题（10分）。

```
.section .data
    array: .long 10, -20, 30, -40, 50
    length = (. -array)/4          # length 为array中元数的个数, = 5
    format: .ascii "%d\n\0"
.section .text
.global _start
_start:
    mov $0, %eax
    mov $length, %ecx
    lea array, %edi                # ①
lp_1:
    cmpl $0, (%edi)
    jl lp_2                        # ②
    inc %eax
lp_2:
    add $4, %edi
    sub $1, %ecx                   # ③
    jne lp_1
    push %eax
    push $format
    call printf
    mov $1, %eax                  # 程序正常退出
    mov $0, %ebx
    int $0x80
```

1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？（2分）

统计 array 数组中非负数的个数并显示。 显示 3

2) 若标号 lp_1 写到 ①处语句前，程序运行的结果是什么？为什么？（3分）

显示 5。每次循环都将 array 的地址送 edi，每次循环都是判断数组的第一个元素是否为负数。

3) 若将 ② 处的语句改为 “jb lp_2”，程序运行的结果是什么？（2分）

显示 5。jb 是无符号数比较转移，任何无符号数都不低于 0。

4) 若漏写了 ③ 处的语句，程序运行会出现什么现象？为什么？（3分）

程序运行异常终止。表面上，%edi 在循环中不断加4，加到值为0时，循环终止。但是随着edi的增加，cmpl \$0, (%edi), 访问的内存单元超出程序空间范围，引起异常。

分 数	
评卷人	

四、程序优化 （共 10 分）。

1、举例说明编写 C 程序或者编译器优化时利用相应原则进行优化的做法（包括优化前的方法，优化后的方法）。(10 分，每小题 2 分)

① 提高 CPU 中 cache 的命中率

任务：二维数组求累加和

优化前：二重循环，按列序优先访问

优化后：二重循环，按行序优先

② 提高 CPU 中指令流水线的利用率

任务：一维数组求累加和。

优化前：使用循环，逐个元素相加

优化后：循环展开，消除循环，变成一系列加法语句

③ 使用 CPU 中处理速度更快的指令

任务：表达式计算，将一个数 *9

优化前：用乘法运算，乘 9 运算

优化后：左移 3 个二进制位运算（相当于乘 8），再加原来的操作数

④ 使用 CPU 中单指令多数据流指令或串操作指令

任务：两个数组相加，得到一个新数组

优化前：用循环的方法，逐个对应元素相加

优化后：成组运算，一次多对数据同时运算，减少了循环次数

另一个例子：将一个数组中所有元素置 0。优化前，用循环的方法，逐个元素置 0。优化后，用 memset 函数实现，该函数的实现封装了串操作指令。

⑤ 使用多核 CPU 中多线程处理能力

任务：两个数组相加，得到一个新数组

优化前：逐个对应元素相加

优化后：创建两个或者多个线程，每个线程完成数组中部分元素（一些行）的求和，即按行将数据分成几块，每个线程完成一个块的相加操作。

分 数	
-----	--

- 1、设一个函数中有语句 `int temp=global;` 其中 `global` 是一个初值为 35 的 `int` 类型全局变量。编译器对 `global` 的定义(`int global=35;`)和 `temp=global` 编译时，分别会在可重定位目标文件中的哪些节生成哪些信息？（10 分）

对于 `int global=35;` 在 数据节(.data) 节存放 35 (0x00000023)；在 字符串节(.strtab)，存放字符串“global”；在符号表节(.symtab)，存放有关符号 `global` 的信息，包括定义该符号的节号、在相应节(.data) 节的地址、数据长度（4 个字节）、属性信息等。

对于 `int temp=global;` 在代码节(.text)中，生成相应的语句，其中 `global` 的地址（偏移量）用占位符 `0x00000000`。在代码节的重定位节(.rel.text)，要记录 代码节的相应位置（`global` 占位符的起始地址）要被什么符号（符号表的哪一项）、用什么定位方式（地址相对程序计算器 PC 的 32 位偏移）所代替，还包括计算地址时的附加项等。

- 2、什么是中断和异常？两者有何差别？什么是中断描述符表？中断和异常的响应过程是什么？（10 分）

中断和异常：一个进程在执行过程中，正常的逻辑控制流被特殊的事件所打断，CPU 转到处理这些事件的内核程序去执行，从而引发一个异常控制流。

中断一般是指 CPU 之外的事件发生，如按键盘，鼠标等，是与当前正在执行的指令无关的异步事件。异常是 CPU 正在执行的指令引发的事件，如除 0、访问数据的地址超出程序空间范围等等，是与正在执行的指令相关的同步事件。异常分为故障、陷阱和终止。

中断描述表（中断矢量表）：中断服务程序和异常处理程序的入口地址构成的一个表，每个表项占 8 个字节，存放入口偏移地址及相应代码段的描述信息；这些表项是按照中断和异常的编号（即中断和异常的类型号）顺序排列的。

中断和异常的响应过程：CPU 的控制逻辑确定检测到的中断和异常类型号，从中断描述表取出对应的表项，计算相应的处理程序的入口地址，保存当前的 CS、EIP、EFLAGS 等信息，然后转到处理程序的入口地址对应的位置去执行。在中断处理程序中有 `IRET` 指令，执行该指令从堆栈中恢复 CS、EIP、EFLAGS 等，从而回到被中断的位置继续执行。对于三类异常：故障、陷阱和终止的处理策略有所差异。故障表示能够修复，引起故障的指令会被再次执行；陷阱则是执行引发陷阱的下一条指令；而终止就要结束程序的运行。



华中科技大学计算机科学与技术学院 2023~2024 第二学期

“ 计算机系统基础 ” 考试试卷 (B 卷)

考试方式 闭卷 考试日期 2024-05-25 考试时长 150 分钟

专业班级 启明2401 学号 1202414801 姓名 郭志辉

题号	一	二	三	四	五	总分	总分人	核对人
分值	20	20	20	20	20	100		
得分								

分 数	
评卷人	

一 (20 分) 数据的存储和访问

1、 设有如下程序, 调试时在函数 f 结束前观察了变量的地址以及内存单元内容, 请将程序, 以及内存单元中的内容补充完整。(10 分)。

```
struct person {
    char id[6];
    int height;
    double salary;
    char grade;
};

void f()
{
    int x=25; // &x 为 0xffffd4e0
    int *px = &x; // &px 为 0xffffd4e4
    short sa[2]={-2, 5}; // &sa (即 &sa[0]) 为 0xffffd4e8
    char msg[6]="abcd"; // &msg[0] 为 0xffffd506, 'a' 的 ASCII 为 0x61
    struct person p; // &p 为 0xffffd4ec, &(p.salary) 为 0xffffd4f8
    strcpy(p.id, "abc"); // p 中各字段的地址均按类型长度整除
    p.height = $$; // ① ** 应被替换为 0x00000000
    p.grade='00'; // ② $$ 应被替换为 0x00000000
    p.salary = 12000; // ③ 00 应被替换为 0x00000000
    // ④ p.salary 的表示为 0x00000000
    // ⑤ person 的大小为 24 字节
}
```

下面是以 16 进制数的形式显示内存内容, 每空都需要填一个字节, 最左边是内存地址。
注: 数据是小端存储方式, 部分内存单元内容跟结构变量中的字段无关, 具有随机性。

0xffffd4e0 00 00 00 00 00 00 00 00
0xffffd4e8 00 00 00 00 00 00 00
0xffffd4f0 00 00 00 00 00 00 00
0xffffd4f8 00 00 00 00 00 00 00
0xffffd500 00 00 00 00 00 00 00
0xffffd508 00 00 00 00 00 00 00

第 1 页 共 8 页

float: 1+8+23
double: 1+11+52

12000=2

2 ¹	2	5	32	9	512	13	8192
2 ²	4	6	64	10	1024	14	
3	8	7	128	11	2048		
4	16	8	256	12	4096		

12000 = 2¹³ +

```

global [5]; // 全局变量
void data_access()
{
    ..... // 此处有一些机器指令设置了 %esp, %eax 等

    int i = 0;
    movl $0x0, (%esp)
    int local[5];
    int *p = NULL;
    movl $0x0, 0x4(%esp)
    global [0] = 1;
    movl movl 0x1, 0x34(%eax)
    global [1] = 2;
    movl movl 0x2, 0x38(%eax)
    i = 2;
    movl movl 0x2, (%esp)
    i += 1;
    addl addl $0x1, (%esp)
    global [2] = 3;
    movl movl 0x3, (%esp), %edx
    movl movl $0x3, 0x34(%eax, %edx, 4)
    *(global + 3) = 10;
    movl movl $0xa, 0x40(%eax)
    local[0] = 1;
    movl movl $0x1, 0x8(%esp)
    local[1] = 2;
    movl movl $0x2, 0x14, 0x14(%esp)
    p = local[2];
    lea lea 0x8(%esp), %eax
    movl movl %eax, 0x4(%esp)
    local[2] = 3;
    addl addl $0x10, 0x4(%esp)
    local[2] = 3;
    movl movl 0x4(%esp), %eax
    movl movl $0x32, (%eax)
    i = 1;
    movl movl 0x10(%esp), %eax
    movl movl %eax, (%esp)
}

```

分 数	
评卷人	

二、(20 分) C 语言常用运算及函数调用的机器级实现

1、写出各条 C 语句执行后相应变量的值，并将机器指令补充完整 (10 分)。

```

void myfunc()
{
    .....
    int    x, y;
    unsigned int  u, v;
    unsigned short s = 0xfffe;
    0x0000119d :   movw  $0xfffe, -0x14(%ebp)
    short t = -1;
    0x000011a3 :   movw  $0xffff, -0x12(%ebp)
    x = s; // ① x = 0x 0000ffff
    0x000011a9 :   movzwl -0x14(%ebp), %eax
    0x000011ad :   mov  %eax, -0x10(%ebp)
    y = t; // ② y = 0x ffffffff
    0x000011b0 :   movzwl -0x12(%ebp), %eax
    0x000011b4 :   mov  %eax, -0xc(%ebp)
    u = s; // ③ u = 0x fffff1fe 0000ffff
    0x000011b7 :   movzwl -0x14(%ebp), %eax
    0x000011bb :   mov  %eax, -0x8(%ebp)
    v = t; // ④ v = 0x ffff ffff
    0x000011be :   movzwl -0x10(%ebp), %eax
    0x000011c2 :   mov  %eax, -0x4(%ebp)
    if (x > 0) t = 1; // ⑤ t = 0x 1
    0x000011c5 :   cmpl  $0x0, -0x10(%ebp)
    0x000011c9 :   jg     0x11d1 <myfunc+68>
    0x000011cb :   movw  $0x1, -0x12(%ebp)
    if (u > 0) t = 2; // ⑥ t = 0x 2
    0x000011d1 <+68>: cmpl  $0x0, -0x8(%ebp)
    0x000011d5 :   jg     0x11dd <myfunc+80>
    0x000011d7 :   movw  $0x2, -0x12(%ebp)
    u = u >> 2; // ⑦ u = 0x shr
    0x000011dd <+80>: shr  $0x2, -0x8(%ebp)
    x = x >> 2; // ⑧ x = 0x sar, 算术右移
    0x000011e1 <+84>: shr  $0x2, -0x10(%ebp)
    y = -y; // ⑨ y = 0x neg
    0x000011e5 <+88>: neg  -0xc(%ebp)
    v = ~v; // ⑩ 按位取反: v = 0x not
    0x000011e8 <+91>: not  -0x4(%ebp)
    .....
}

```

2、函数调用的机器级实现（10分）

语句“int r = fsub(25,10);”对应的反汇编代码（最左边的是机器指令的地址）如下。

0x565561bb: 6a 0a push \$0xa	0x0P	0xffffd4e4
0x565561bd: 6a 19 push \$0x19	0xffffd508	0xffffd4e8
0x565561bf: e8 e9 ff ff call 0x5655618d <fsub>	0xffffd50c	0xffffd4ec
0x565561c4: 83 c4 08 add \$0x8,%esp	0x19	0xffffd4f0
0x565561c7: 89 45 fc mov %eax,-0x4(%ebp)	0x0	0xffffd4f4
	XXXXXXXX	0xffffd4f8

设执行“int r = fsub(25,10);”之前，esp 的值为 0xffffd4f8，ebp 值为 0xffffd508。

函数 fsub 的反汇编代码如下：

```
int fsub(int x, int y)
{
    0x5655618d <+0>: 55          push    %ebp
    0x5655618e <+1>: 89 e5       mov     %esp,%ebp
    0x56556190 <+3>: 83 ec 10    sub     $0x10,%esp
    0x56556193 <+6>: e8 35 00 00 call    0x565561cd <__x86.get_pc_thunk.ax>
    0x56556198 <+11>: 05 44 2e 00 add     $0x2e44,%eax

    int result;
    result = x - y;

    0x5655619d <+16>: 8b 45 08    mov     0x8(%ebp),%eax
    0x565561a0 <+19>: 2b 45 0c    sub     0xc(%ebp),%eax
    0x565561a3 <+22>: 89 45 fc    mov     %eax,-0x4(%ebp)

    return result;

    0x565561a6 <+25>: 8b 45 fc    mov     -0x4(%ebp),%eax
}
0x565561a9 <+28>: c9          leave   %eax
0x565561aa <+29>: c3          ret     0
```

- 在上面的表格中，填写执行到 leave 时，相应内存单元中的值。
- ① 函数参数 x 的地址（即&x）是 0x_____。
- ② 局部变量 result 的地址（即&result）是 0x_____。
- ③ 执行“sub 0xc(%ebp), %eax”后，CF=0, SF=0, ZF=0, OF=0。
- ④ 执行“ret”后，%esp=0x_____。

分 数	
评卷人	

三、(20 分) 计算机系统的基本原理

1、程序计数器 (eip) 的自动变化 (10 分)

设有 void myfunc() { }

int main() { }

int x = -20;

0x565561ca <+29>: c7 45 f0 ec ff ff ff movl \$0xffffffff,-0x10(%ebp)

// 在取出该指令并进行译码后, eip = 0x_____

int y;

void (*fp)();

if (x >= 0) y = 1;

0x565561d1 <+36>: 83 7d f0 00 cmpl \$0x0,-0x10(%ebp)

0x565561d5 <+40>: 78 09 js 0x565561e0 <main+51>

// 上面语句中的 0x565561e0, 是 直接寻址, 计算得到的

// 当 js 转移条件成立, 即 sf = 1, 执行 eip + 指令中位移量 → eip

// 若 js 转移条件不成立, 该指令也就执行完成, 此时 eip = 0x 565561d7.

0x565561d7 <+42>: c7 45 ec 01 00 00 00 movl \$0x1,-0x14(%ebp)

0x565561de <+49>: eb 07 jmp 0x565561e7 <main+58>

// 上面语句中的机器码 07, 表示的含义是 直接寻址.

else y = -1;

0x565561e0 <+51>: c7 45 ee ff ff ff ff movl \$0xffffffff,-0x14(%ebp)

myfunc();

0x565561e7 <+58>: e8 b1 ff ff ff call 0x5655619d <myfunc>

// 执行 call 指令时, 会将 地址, 将 地址 → eip

fp = myfunc;

0x565561ec <+63>: 8d 83 c5 d1 ff ff lea -0x2e3b(%ebx),%eax

// 执行 lea 指令时, ebx = 0x_____

0x565561f2 <+69>: 89 45 f4 mov %eax,-0xc(%ebp)

fp();

0x565561f5 <+72>: 8b 45 f4 mov -0xc(%ebp),%eax

0x565561f8 <+75>: ff d0 call *%eax

// 执行 call 指令时, 采用 寄存器寻址方式, 得到函数的入口地址

0x565561fa <+77>:

}
执行函数 myfunc 中的 ret 指令时, CPU 会 返回 → eip。

2、程序执行的基本原理（10分）。

- (1) 程序运行的基本过程是什么？
- (2) 举例说明什么是地址类型转换？什么是数据类型转换？
- (3) 全局变量与非静态的局部变量各分配在什么段中？ 为什么一个函数不应返回局部变量的地址？
- (4) 指令集体系结构与计算机系统的什么部件有关？它是哪两个部分之间的接口？
- (5) 什么是缓冲区溢出？缓冲区溢出会有哪些危害？

分 数	
评卷人	

四、（20分）程序编译及优化

1、举例说明编译器利用 CPU 的特性所能进行的优化。请给出 5 种不同类型的示例，并说明优化原理。（10分）

- ①流水线的
- ②串操作
- ③cache
- ④优化的机器指令
- ⑤多核CPU

- 1.立即数
- 2.直接
- 3.寄存器
- 4.寄存器间接
- 5.变址
- 6.基+变址

2、除了利用 CPU 特性进行优化，编译器还可以优化减少在 CPU 上执行指令。举出三种不同类型的优化示例。（6 分）

删除语句 避免
减少运算

3、为了提高程序的安全性，编译器可以在程序中生成一些安全检查代码。举出两种不同的安全检查示例并说明其原理。（4 分）

解答
内容
不得
超过
装订
线

分 数	
评卷人	

五、（20 分）链接和异常控制流

1、设有如下程序（10 分）

```
#include <stdio.h>
int global = 20;
int main()
{
    int x, y;
    x = 25;
    global = 25;
    y = global;
    printf("good %d %d\n", x, y);
    return 0;
}
```

在编译生成可重定位目标文件中，会有数据节(.data)、只读数据节(.rodata)、代码节(.text)、
代码节的重定位节(.rel.text)、符号表节(.symtab)、字符串节(.strtab)等。

请结合给出的程序，回答如下问题。

所有符号的名字

(1) 上述六个节中分别包含哪些信息? (6分)

(2) 需要重定位的符号有哪几个? (2分)

global, main, printf

(3) 哪些C语句和语句中的哪些部分需要重定位? (不用给出具体的机器指令)? (2分)

2、异常控制流 (10分)

(1) 产生中断的原因有哪些?

(2) 产生异常的原因有哪些?

指向指令时的异常

(3) 中断和异常的差别什么?

外|内

(4) 什么是中断描述符表?

(5) 中断和异常的响应过程是什么?

24秋A卷

2026年5月28日 9:59



24秋A卷



华中科技大学 2024~2025 第一学期
“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试时间 2024-11-26 晚上 考试时长 150 分钟

院 (系) 专业班级

学 号 姓 名

题号	一	二	三	四	五			总分	总分人	核对人
分值	20	20	20	20	20			100		
得分										

分 数	
评卷人	

一、(20 分) 数据的表示和访问。

1. 有一个 C 语言程序如下。(10 分)

```
typedef struct {  
    char    id[10];  
    int     age;  
    float   height;  
} person_info;  
void main() {  
    person_info p1;  
    strcpy(p1.id, "12350"); #字符 0 的 ASCII 为 0x30  
    p1.age = 15;  
    p1.height = 1.75f;  
    unsigned short x1 = *(unsigned short*)(p1.id + 1);  
    int y1 = -100; -----①  
}
```

将上述程序编译链接生成可执行程序, 用调试工具运行完语句①后, 观察变量地址如下: p1 的地址是 0xffffd038, p1.age 的地址是 0xffffd044, p1.height 的地址是 0xffffd048, x1 的地址是 0xffffd032, y1 的地址是 0xffffd034。以字节形式观察这些变量在内存中的存放形式。以 16 进制的形式填写程序用到的内存单元内容 (无关的字节中的内容填写 XX, person_info 的大小为 20 字节)。

0xffffd032: _____
0xffffd03a: _____
0xffffd042: _____
0xffffd04a: _____。

提示: 浮点数 float 的编码规则: float 用 32 位表示, 最高位 (第 31 位) 为符号位, 23-30 位为用移码表示的指数 (移码值为 0x7F), 0-22 位为尾数位。

2. 调试一个 C 程序时,看到源程序语句与对应的反汇编语句如下。根据观察到的信息填空。(10 分,每空 1 分)

```
int    arr1[5];
short arr2[3][2];
void   main() {
    int i, j;
    i = 2;    // 执行该语句时, ebp 的值为 0xffffd068, eax 的值为 0x56558fdc
0x5655619d <+16>:  c7 45 ec 02 00 00 00    movl    $0x2, -0x14(%ebp)
    变量 i 的地址是 0x_____, 该变量存放在_____段中。
    arr1[i] = 34;
0x565561a4 <+23>:  8b 55 ec    mov    -0x14(%ebp), %edx
0x565561a7 <+26>:  c7 84 90 30 00 00 00 22 00 00 00    movl    $0x22, 0x30(%eax,%edx,4)
    此时, %edx 存放的是_____的值, arr1[i]的地址是 0x_____。
    i = 1;
0x565561b2 <+37>:  movl    $0x1, -0x14(%ebp)
    j = 1;
0x565561b9 <+44>:  movl    $0x1, -0x10(%ebp)
    arr2[i][j] = 27;
0x565561c0 <+51>:  8b 55 ec    mov    -0x14(%ebp), %edx
0x565561c3 <+54>:  8d 0c 12    lea    (%edx,%edx,1), %ecx
    执行这条指令后, %ecx 中的值是_____, arr2 的每行有两个元素, %ecx 表示的含
    义是_____。
0x565561c6 <+57>:  8b 55 f0    mov    -0x10(%ebp), %edx
0x565561c9 <+60>:  01 ca    add    %ecx, %edx
0x565561cb <+62>:  66 c7 84 50 44 00 00 00 1b 00    movw    $0x1b, 0x44(%eax,%edx,2)
    arr2[i][j]的地址是 0x_____。
    int *p1 = &arr1[2];
0x565561d5 <+72>:  8d 90 30 00 00 00    lea    _____(%eax), %edx
0x565561db <+78>:  89 55 f4    mov    %edx, -0xc(%ebp)
    *(p1 + 1) = 20;
0x565561de <+81>:  mov    -0xc(%ebp), %edx
0x565561e1 <+84>:  add    _____, %edx
0x565561e4 <+87>:  movl    $0x14, (%edx)
    short *p3 = &arr2[1][1];
0x565561e7 <+90>:  8d 80 4a 00 00 00    lea    _____(%eax), %eax
0x565561ed <+96>:  89 45 fc    mov    %eax, -0x4(%ebp)
}
```

分 数	
评卷人	

二、(20 分) C 程序的转换

1. 阅读下面的程序段，回答问题（10 分，每空 1 分）

```
void main() {
    short s = -5;
    unsigned short us = 0xfffb;
    int i = s;           // ①
    unsigned int ui = us; // ②
    if (ui < 5)           // ③
        s = (int)(s + s) >> 1;
    else
        us = (unsigned int)(us + us) >> 1; // ④
}
```

变量 s、us、i、ui，对应的地址表达式为：-0xc(%ebp)、-0xa(%ebp)、-0x8(%ebp)、-0x4(%ebp)。

(1) 执行完①处语句后，i = 0x_____；

该语句对 s 变量的处理指令为：_____ -0xc(%ebp),%eax。

(2) 将②处的运算指令序列补充完整：

0x565561b0 <+35>: _____ -0xa(%ebp),%eax

0x565561b4 <+39>: mov _____ %eax,-0x4(%ebp)

执行完②处语句后 ui = _____；

(3) 对于③处语句，补充完整这条语句的处理序列：

0x565561b7 <+42>: cmpl \$0x5, -0x4(%ebp)

0x565561bb <+46>: _____ 0x565561cb <main+62>

③ 处 if-else 语句，执行分支_____。

(4) ④处语句对应的机器指令是：

0x565561cb <+62>: _____ -0xa(%ebp),%eax

0x565561cf <+66>: _____

0x565561d1 <+68>: _____ %eax

0x565561d3 <+70>: mov _____, -0xa(%ebp)

2. 阅读如下程序及其反汇编代码，回答问题（10 分）。

```
int max(int a, int b) {
    if (a > b) return a;
    else return b;
}

void main() {
    int x = 100;
    int y = -10;
    int m = max(x, y);
}
```

main 函数中，int m=max(x, y) 对应的反汇编代码如下：

```
0x565561ca <+30>:  push  -0x8(%ebp)          ----- ①
0x565561cd <+33>:  push  -0xc(%ebp)
0x565561d0 <+36>:  call  0x5655618d <max>
0x565561d5 <+41>:  add   $0x8,%esp          ----- ②
0x565561d8 <+44>:  mov   %eax, -0x4(%ebp)

max 函数的反汇编代码如下：
```

```
0x5655618d <+0>:  push  %ebp          ----- ③
0x5655618e <+1>:  mov   %esp, %ebp
0x5655619a <+13>:  mov   0x8(%ebp), %eax
0x5655619d <+16>:  cmp   0xc(%ebp), %eax
0x565561a0 <+19>:  jle   0x565561a7 <max+26>
0x565561a2 <+21>:  mov   0x8(%ebp), %eax
0x565561a5 <+24>:  jmp   0x565561aa <max+29>
0x565561a7 <+26>:  mov   0xc(%ebp), %eax
0x565561aa <+29>:  mov   %ebp, %esp
0x565561ac <+31>:  pop   %ebp          ----- ④
0x565561ad <+32>:  ret
```

设执行①处语句前，(esp)= 0xffffd058，(ebp)= 0xffffd068。

填写执行指令③后堆栈的示意图，包括main函数的局部变量x, y的位置和内容，无关的位置填写XX。

指令④的作用是_____。若缺少指令④，子程序（能/不能）_____正常返回。

执行指令②前，esp=_____。指令②的作用是_____。

	0xffffd048
	0xffffd04c
	0xffffd050
	0xffffd054
	0xffffd058
	0xffffd05c
	0xffffd060
	0xffffd064
	0xffffd068

分 数		三、（20 分）程序的运行的基本原理
评卷人		

1. C 程序的部分反汇编代码如下。完成填空（10 分，每空 1 分）

```
void main() {
    int x = -1;
0x565561b9 <+28>:  movl  $0xffffffff,-0x14(%ebp)
    int y = -2;
0x565561c0 <+35>:  movl  $0xffffffe,-0x10(%ebp)
    int z = y - x;
0x565561c7 <+42>:  mov   -0x10(%ebp),%edx
0x565561ca <+45>:  sub   -0x14(%ebp),%edx          ----- ①
0x565561cd <+48>:  mov   %edx,-0xc(%ebp)
    if (y == z)
```

```

0x565561d0 <+51>:    mov     -0x10(%ebp),%edx
0x565561d3 <+54>:    cmp     %edx,-0xc(%ebp)
0x565561d6 <+57>:    jne     0x565561ee <main+81> -----②
    printf("equal\n");
0x565561d8 <+59>:    sub     $0xc,%esp
    .....
0x565561ec <+79>:    jmp     0x56556202 <main+101> ----- ③
    else printf("not equal\n");
0x565561ee <+81>:    .....
0x56556202<+101>    .....
}

```

内存地址 0x565561c0—0x565561c6 的内存单元中, 存放的是汇编语句 `mov $0xffffffff, 0x10(%ebp)` 的_____。

① 处的指令执行后, CF=_____, OF=_____, ZF=_____, SF=_____。

程序在运行时, 寄存器 EIP 中存放的是_____。当②处的指令预取和解码后, EIP 的值为_____。由于此时 ZF = _____, 转移条件成立, ②处的指令执行后, EIP 的值为_____。

③ 处的 `jmp` 指令, 其在实现 `if-else` 语句时的作用是_____。

2. 已知函数 `swap` 的 C 语言框架如下。回答问题 (10 分)

```

int a = 200, b = 100;
void swap(int *x, int *y) {
    0x5655618d <+0>:    push    %ebp
    0x5655618e <+1>:    mov     %esp,%ebp
    0x56556190 <+3>:    sub     $0x10,%esp
    0x56556193 <+6>:    call    0x565561ee <__x86.get_pc_thunk.ax>
    0x56556198 <+11>:   add     $0x2e44,%eax
    int tmp;
    if (*x > *y) {
    0x5655619d <+16>:   mov     0x8(%ebp),%eax
    0x565561a0 <+19>:   mov     (%eax),%edx
    0x565561a2 <+21>:   mov     0xc(%ebp),%eax
    0x565561a5 <+24>:   mov     (%eax),%eax
    0x565561a7 <+26>:   cmp     %eax,%edx
    0x565561a9 <+28>:   jle     0x565561c5 <swap+56>
        tmp = *x;
        *x = *y;
        *y = tmp;
    }
}
void main() {
    swap(&a, &b);
=> 0x565561d5 <+13>:   lea     0x30(%eax), %edx

```

```

0x565561db <+19>:    push    %edx
0x565561dc <+20>:    lea     0x2c(%eax), %eax
0x565561e2 <+26>:    push    %eax
0x565561e3 <+27>:    call   0x5655618d <swap>
(0x565561e3 <+27>:    e8 a5 ff ff ff  call   0x5655618d <swap>)
0x565561e8 <+32>:    add     $0x8, %esp
}

```

(1) 根据反汇编指令， swap(&a, &b)函数调用时，如何传递参数的地址？（2 分）

(2) 在 swap 函数中，如何根据地址，获取 a, b 的值？（2 分）

(3) 反汇编指令：call 0x5655618d <swap>，对应的机器指令为：e8 a5 ff ff ff，其中 a5 ff ff 的含义是什么？（3 分）。

(4) 请写出 0x565561c5<swap+56> 处的机器指令，使得程序能正常返回。（3 分）

分 数	
评卷人	

四、（20 分）程序分析与优化

1、根据反汇编代码，填补 C 语言语句缺失的部分（7 分，每空 1 分）

```

void main() {
    char str1[13] = _____;
    0x565561d5 <+40>:    movl    $0x64636261, -0x26(%ebp)    #字符 a 的 ASCII 为 0x61
    0x565561dc <+47>:    movl    $0x68676665, -0x22(%ebp)
    0x565561e3 <+54>:    movl    $0x6c6b6a69, -0x1e(%ebp)
    0x565561ea <+61>:    movb    $0x0, -0x1a(%ebp)

    char str2[13];
    for (int i = _____; _____; _____) {
    0x565561ee <+65>:    movl    $0x0, -0x2c(%ebp)
    0x565561f5 <+72>:    jmp     0x56556215 <main+104>
        _____ = _____;
    0x565561f7 <+74>:    lea     -0x26(%ebp), %ecx
    0x565561fa <+77>:    mov     -0x2c(%ebp), %edx

```


解答内容不得超过装订线

```
0x565561fd <+80>:    add    %ecx, %edx
0x565561ff <+82>:    movzbl (%edx), %edx
0x56556202 <+85>:    mov    %edx, %ecx
0x56556204 <+87>:    and    $0xf, %ecx
0x56556207 <+90>:    lea    -0x19(%ebp), %ebx    # -0x19(%ebp)代表 str2 数组首字符
0x5655620a <+93>:    mov    -0x2c(%ebp), %edx
0x5655620d <+96>:    add    %ebx, %edx
0x5655620f <+98>:    mov    %cl, (%edx)
    }
0x56556211 <+100>:   addl    $0x1, -0x2c(%ebp)
0x56556215 <+104>:   cmpl    $0xb, -0x2c(%ebp)
0x56556219 <+108>:   jle     0x565561f7 <main+74>
    _____;
0x5655621b <+110>:   movb    $0x0, -0xd(%ebp)
}
```

- 2. 上述程序的功能是什么？（2 分）
- 3. 上述反汇编程序，效率较低的原因有哪些？（3 分）
- 4. 常见的程序优化方法有哪些？（3 分）
- 5. 请用汇编语言程序片段给出功能等价的优化方案。（5 分）

分 数	
评卷人	

五、（20 分）程序的链接和异常控制流

1、有两个源文件: main.c 和 test.c，它们的内容如下，将这两个文件分别编译后再链接成一个执行程序。回答下列问题。（10 分）

```

/*main.c*/
int mul( );
int a[5]={1,2,3,4,5};
int main( ) {
    int val;
    val=mul();
    printf("sum_val=%d\n", val);
    return 0;
}

```

```

/*test.c*/
extern int a[];
static int b[5]={6,7,8,9,10};
int mul( ) {
    int i, sum_val=0;
    for (i=0; i<5; i++)
    {
        sum_val = sum_val + a[i] * b[i];
    }
    return sum_val;
}

```

(1) 可重定位目标文件中有哪些节？每个节中包含有哪些信息？至少写出 4 个。(4 分)

(2) 对于编译生成的可重定位目标文件 `main.o` 和 `test.o`，各自生成了哪些符号？并说明其类型（全局、外部、本地），以及这些符号出现在各自模块的哪个节中。(4 分)

(3) `main.c` 中哪些语句中的那些符号需要重定位？(2 分)

2. 回答与中断和异常相关的问题 (10 分)

(1) 可屏蔽中断和不可屏蔽中断有什么区别？(2 分)

(2) 举 3 个异常的例子。(3 分)

(3) 用户单击了一下鼠标，请描述计算机的处理过程。(3 分)

(4) 中断向量表在中断响应过程中起到什么作用？(2 分)



华中科技大学 2024~2025 第二学期
“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试时间 2025-5-13 晚上 考试时长 150 分钟
院 (系) 计算机科学与技术 专业班级 启明 2401
学 号 1202414869 姓 名 韩轩

题号	一	二	三	四	五			总分	总分人	核对人
分值	20	20	30	20	10			100		
得分										

分 数	
评卷人	

一、(20 分) 计算机工作的基本原理。

1、正常控制流下的程序执行 (10 分, 每空 1 分)

① 冯 诺 依 曼 体 系 结 构 的 基 本 思 想 是 存 储 程 序 和 自 动 执 行。程序计数器 的自动变化是实
现程序自动执行的核心机制之一。以下是 Intel CPU 中的 rip 自动变化的方法。在加载运行一个程
序时, 操作系统和 CPU 会协同工作, 将可执行文件中记载的入口地址送入 rip 中。设 rip 为
0x08001144, 该处指令为“c7 45 f4 05 00 00 00 movl \$0x5,-0xc(%rbp)”, 在取出该指令并进行
译码后, rip=0x800114b。在 0x0800114f 处, 有指令 not equal f “75 09 jne 0x800115a”, 若执行该指
令前, ZF=0, 指令执行后 rip=0x ; 若 ZF=1, 指令执行后 rip=0x08001145。
在 0x08001158 处, 有反汇编语句“eb xx jmp 0x8001161”, 这里机器码(1 个字节)xx = 07;
在执行 CALL 指令时, CPU 会将 rip 压入堆栈栈顶, 同时会将 子程序入口地址送入
rip 中。在子程序返回时, 执行 ret 指令, 会从 栈顶弹出 送入 rip 中。

2、异常控制流下的 rip 的变化 (10 分, 每空 1 分): 出 8 个字节

在执行一条指令后, CPU 检测到有按键操作, 此时会发生 中断, CPU 会依次保存标志寄
存器和 rip, 同时根据中断号从 中断向量表 中找到 中断处理程序的入口地址。
序保护恢复通用寄存器。iret 指令先将 栈顶弹出 弹出标志寄存器 从而恢复被打断程序的继
续执行。若 CPU 在执行除零指令或者指令中访问的地址超出了本程序的空间范围, 此时会产
生 异常。CPU 执行异常处理程序后, 若继续执行引起异常的指令, 则称之为 故障。若执行
引起异常的指令的下一条指令, 则称之为 陷阱, 其代表性的事件有 调试跟踪。

解
答
内
容
不
得
超
过
装
订
线

分 数	
评卷人	

二、(20 分) 数据的存储和地址表达式

1. 数据的存储形式 (10 分)

设有如下程序段，在运行时观察了各变量的地址如下

```
int x=10;    // &x = 0x08004010
int y=-10;   // &y = 0x8004014
char str[10]="20250513"; // &str[0] = 0x8004018
char a[4]={255,-1,257,-257}; // &a[0] = 0x8004022
float f=20.5; // &f = 0x8004028
short u=30;  // &u = 0x800402c
```

11110
⑭
E

以 16 进制的形式填写程序用到的内存单元内容(无关的字节中的内容填写 XX, 每行 2 分)。

```
0x08004010: 0a 00 00 00 1f 1f 1f 1f
0x08004018: 32 30 32 35 30 35 31 33
0x08004020: 00 xx 1f 1f 1f 1f 1f 1f
0x08004028: 00 00 28 41 1e 00 1f 1f
```

*(int *)(str+2) 的值是 0x 32353035

*(char *)(&x+1) 的值是 0x 1f

提示: '0' 的 ASCII 为 0x30; 浮点数 float 的编码规则: float 用 32 位表示, 最高位(第 31 位)为符号位, 23-30 位为用移码表示的指数(移码值为 0x7f), 0-22 位为尾数位。

2. 根据调试一个 C 程序时看到的如下信息填空。(10 分, 每空 1 分)

```
int a[5];
int i=3;
movl $0x3, -0x44(%rbp) // rbp = 0x7ffffffc040
```

此时, i 的地址是 0x 7ffffffc040, i 存放在 2 段中。i 的地址表达式是 -0x44(%rbp)。该表达式是在 relocation 就固定不变了。但是 i 的具体地址值在不同的执行时刻, 因 rbp 的变化而变化。编译程序写完 → 生成执行程序

查地寻地

```
int *p=&a[0];
lea -0x20(%rbp), %rax ; 执行后, rax = 0x 7ffffffc020
mov %rax, -0x38(%rbp) ; p 的地址是 -0x38(%rbp)
a[0]=1;
```

第 2 页 共 8 页

分 数	
评卷人	

三、(30 分) C 程序与机器语言的对应

```
movl $0x1, -0x38(%rbp)
*(a+1)=2;
movl $0x2, -0x1c(%rbp)
*(p+2)=3;
mov -0x38(%rbp), %rax
add 0x8, %rax
movl $0x3, (%rax)
a[i]=5;
mov -0x44(%rbp), %eax
cltq ; 将 rax 的高 32 位清零
movl $0x5, -0x20(%rbp), %rax, 4)
```

审题, 想明白再下笔!

分 数	
评卷人	

三、(30 分) C 程序与机器语言的对应

1、数据传送、算术运算、按位运算、移位运算 (10 分, 每小题 1 分)

(写汇编语句时, 要求有指令助记符、操作数和操作数地址, 用变量符号名表示操作数地址)

设有 `int x=0x12345678;`

`unsigned short u=0xffff; short v=0xffff;`

分别执行如下语句 (每条语句执行前, x 都是 0x12345678)

- ① `x=u;` 数据扩展传送指令的指令助记符是 _____, 执行后, `x=_____`。
- ② `x=v;` 数据扩展传送指令的指令助记符是 _____, 执行后, `x=_____`。
- ③ `u=u>0;` `u>0` _____ (是、否) 成立; 执行后 `u=_____`。
- ④ `v=v>0;` `v>0` _____ (是、否) 成立; 执行后 `v=_____`。
- ⑤ `x = x>>2` 对应的汇编语句是 shr 2, 执行后, `x=0x_____`。
- ⑥ `x = x<<1` 对应的汇编语句是 shl 1, 执行后, `x=0x_____`。
- ⑦ `x=x&0x0f` 对应的汇编语句是 and 0xf, 执行后, `x = 0x_____`。
- ⑧ `x = -x` 对应的汇编语句是 neg, 执行后, `x = 0x_____`。
- ⑨ `x = ~x` 对应的汇编语句是 not, 执行后, `x = 0x_____`。
- ⑩ `x=x+10` 对应的汇编语句是 add 10, 执行后, `x = 0x_____`。

2、分支转移程序（10 分，每空 1 分）

① if 语句与汇编语句的对应（5 分，每空 1 分）

（L1、L2、L3、L4 皆为标号，代表机器指令的地址）

```
int x=-1;    int y=1;

    movl    $0xffffffff, -0xc(%rbp)

    movl    $0x1, -0x8(%rbp)

int flag;

if (x>y)    flag=1;

    mov     -0xc(%rbp), %eax

    cmp     -0x8(%rbp), %eax

L1:  _____

    movl    $0x1, -0x4(%rbp)

L2:  _____

else    flag=0;

L3:  _____

L4:  .....
```

若将变量 x、y 定义为 unsigned int 类型，L1 处的汇编语句应为 _____。

若漏写了 L2 处的语句，与该 if 语句功能等价的 C 语句是 _____。

② switch 语句与汇编语句的对应（5 分，每空 1 分）

```
int x=3;

    movl    $0x3, -0x8(%rbp)

int y;

switch (x)

    cmpl    $0x1, -0x8(%rbp)

    _____

    cmpl    $0x2, -0x8(%rbp)

    _____

    _____

{
```

```

    case 1: y=1; break;

L1:    movl    $0x1, -0x4(%rbp)

L2:    jmp     L6

    case 2: y=2; break;

L3:    movl    $0x2, -0x4(%rbp)

L4:    jmp     L6

    default: y=3;

L5:    movl    $0x3, -0x4(%rbp)
}

L6: .....

```

若将 default 分支移到 case 1 之前，switch 语句执行后，y=_____。

若要使 default 语句放在 case 1 之前与放在最后的功能等价，default 语句应修改为：

_____。

3、函数调用与汇编语句的对应 （10 分）

有如下主程序：

```

int a = 1;
    0x000000000800116c <main+8>: movl    $0x1, -0x4(%rbp)

int b = 2;
    0x0000000008001173 <main+15>: movl    $0x2, -0x8(%rbp)

int sum = 0;
    0x000000000800117a <main+22>: movl    $0x0, -0xc(%rbp)

sum = fadd(a, b);
    0x0000000008001181 <main+29>: mov     -0x8(%rbp), %edx
    0x0000000008001184 <main+32>: mov     -0x4(%rbp), %eax
    0x0000000008001187 <main+35>: mov     %edx, %esi
    0x0000000008001189 <main+37>: mov     %eax, %edi
    0x000000000800118b <main+39>: callq   0x8001139 <fadd>
    0x0000000008001190 <main+44>: mov     %eax, -0xc(%rbp)

```

int fadd(int x, int y) 函数和反汇编代码如下：

```

{
    0x0000000008001139 <+0>:    push    %rbp
    0x000000000800113a <+1>:    mov     %rsp, %rbp
    0x000000000800113d <+4>:    mov     %edi, -0x14(%rbp)
    0x0000000008001140 <+7>:    mov     %esi, -0x18(%rbp)

    int u, v, w;
    u = 2*x ;

```

```

0x0000000008001143 <+10>:  mov    -0x14(%rbp), %eax
0x0000000008001146 <+13>:  add     %eax, %eax
0x0000000008001148 <+15>:  mov     %eax, -0x4(%rbp)
        v = y + 25;
0x000000000800114b <+18>:  mov     -0x18(%rbp), %eax
0x000000000800114e <+21>:  add     $0x19, %eax
0x0000000008001151 <+24>:  mov     %eax, -0x8(%rbp)
        w = u + v;
0x0000000008001154 <+27>:  mov     -0x4(%rbp), %edx
0x0000000008001157 <+30>:  mov     -0x8(%rbp), %eax
0x000000000800115a <+33>:  add     %edx, %eax
0x000000000800115c <+35>:  mov     %eax, -0xc(%rbp)
        return w;
0x000000000800115f <+38>:  mov     -0xc(%rbp), %eax
}
0x0000000008001162 <+41>:  pop     %rbp
0x0000000008001163 <+42>:  retq

```

刚进入 fadd 时（即 push %rbp 执行前），

rsp = 0x7ffffffe028；

rbp = 0x7ffffffe040

填写执行到 fadd 的 “pop %rbp” 语句时的堆栈内容。

要求：在图中标明 x, y, u, v, w 的位置；

每行是 4 个字节，其中的内容用 0x... 表示

具有随机性的单元内容填为 XXXXXXXX。

	0x7ffffffe008
	0x7ffffffe00c
xx	0x7ffffffe010
	0x7ffffffe014
	0x7ffffffe018
	0x7ffffffe01c
040.	0x7ffffffe020
7fff.	0x7ffffffe024
1190	0x7ffffffe028 ← rsp
	0x7ffffffe02c

w.
 v
 u
 rbp (rbp).
 1190
 0x1
 0x2
 x
 y
 ↓
 ↑ 栈顶.
 ← rsp

分 数	
评卷人	

四、(20 分) 程序编译与优化问答

1、程序优化 (16 分)

- ① 求二维数组中所有元素的和，在二重循环中，行序优先访问比列序优先访问快，利用了 CPU 的什么特性和二维数据存储的什么特性？ (2 分)

② 将有分支的语句转换成无分支的语句，或者将循环语句转换成无循环的语句，提高程序执行速度，利用了 CPU 的什么特性？为什么会提高速度？（2 分）

③ 将一大片空间中的内容置为 0，可以用循环语句一个字节一个字节的置零，也可以使用 `memset` 函数来完成，后者的执行速度更快，原因是什么？（2 分）

串指令

④ 写出与 `imul $2, %eax` 功能等价但执行速度更快的两种机器指令（2 分）

⑤ 举例说明，编译器还可以做哪些与 CPU 特性无关的优化？（2 分）

⑥ 设有如下程序段及其未优化的反汇编程序段（6 分）。

```
int a[5];
int sum=0, i;
    movl    $0x0, -0x28(%rbp)
for (i=0; i<5; i++)    sum+=a[i];
    movl    $0x0, -0x24(%rbp)
    jmp     L2
L1:  mov     -0x24(%rbp), %eax
    cltq    ; 将 rax 的高 32 位置 0
    mov     -0x20(%rbp,%rax,4), %eax
    add     %eax, -0x28(%rbp)
    addl    $0x1, -0x24(%rbp)
L2:  cmpl    $0x4, -0x24(%rbp)
    jle     L1
```

写出完成等价功能的尽可能优化的汇编程序段，并指出使用的优化策略。

2、程序的安全性（4分）

- ① 为什么不应将函数中局部变量的地址返回给调用函数来使用？（2分）

野指针.

- ② 编译器生成的执行程序中，可采取何种方法检测出缓冲区溢出攻击？（2分）

哨兵

分 数	
评卷人	

五、（10分）程序的链接回答

- 1、设一个程序中，有全局变量定义语句 `int global=10;` （2分）

在可重定位目标文件中的哪些节，会生成与该变量定义有关的哪些信息？

符号表节.
数据节.

- 2、在一个函数中有语句 `int temp=global;`（`global` 为前面定义的全局变量），在可重定位目标文件中的哪些节，会生成与该语句有关的哪些信息？（2分）

代码节.
重定位节.

- 3、设函数中有语句 `printf("very good\n");`；字符串“very good\n”存放在什么节中？该 `printf` 语句对应的代码中有几处需要重定位？分别是什么位置需要重定位？（2分）

只读数据节.
2处. `{` 和 `printf.`

- 4、简述链接的执行过程，以及在链接时能够检测到的错误。（4分）

重定义. 未定义

合并. 检错.
合并后. 定址. 重定位.

计算机系统基础_202404_模拟题

2026年5月28日 10:00



计算机系统
基础_202...

“ 计算机系统基础 ” 模拟试卷

一、 计算机工作基本原理填空

1、在对某程序进行调试时，看到如下一段信息：

004116DA	83 7D FC 00	cmpl	
004116DE	75 08	jne	004116E8
004116E0	8B 45 FC	mov	
004116E3	89 45 F8	mov	
004116E6	EB 07	jmp	004116EF
004116E8	C7 45 F8 01 00 00 00	movl	
004116EF	E8 2D FB FF FF	callq	func (00411221) ; func 为函数名
004116F4	C7 45 F4 21 12 41 00	movl	\$0x411221, -0x0c(%ebp)
004116FB	FF 55 F4	callq	-0x0c(%ebp)
.....			

① 观察指令在内存中的存放形式(每个空对应一个 16 进制字节数据)：

004116DA _____
004116E2

② 设当前 eip 为 0x004116DE，在取出 eip 指向的指令并进行译码后，eip = _____；

③ 在 0x004116DE 处的指令为：75 08 jne 004116E8，直观上是在 zf = _____ 时，会将 0x004116E8 → eip，计算出该地址值的方法 _____；当转移条件不成立时，则该指令执行完成，eip _____（会、不会）改变。

④ 004116EF 处的指令中有 0xFFFFFB2D，计算出该值的方法是 _____；

⑤ 004116FB 处的子程序调用指令，调用的子程序的入口地址是 _____，执行该处语句时，CPU 会将 0x_____ 压入堆栈中。

⑥ 执行子程序中的 RET 指令时，CPU 会 _____ → eip。

2、设一个 C 语言程序中定义有全局数组 int array[5]；array 的起始地址(即 &array[0])为 0x420100，每个元素占 4 个字节，现要将数组的第 2 个元素(即 array[2])置为 20。按指定的寻址方式写出实

现该功能的机器指令段（汇编语句段）。

注：完成该功能的 C 语句有：“array[2]=20; *(array +2) =20; int i=2; array[i]=20; int *p=array; p[i] = 20;”等等。不同的 C 语句写法编译后，生成的机器指令不同，会出现多种寻址方式访问同一单元。

① 使用直接寻址方式，机器指令中含有操作数的偏移地址：

② 使用寄存器间接寻址方式，先将操作数的地址送 ebx，再访存：

_____；给 ebx 赋值

③ 使用变址寻址方式，先将元素的下标（即 2）送 eax，再访存：

_____；给 eax 赋值

④ 使用基址加变址寻址方式，先将数组的起始地址送 eax，第 2 个元素在数组中的偏移字节数送 ebx，再访存：

_____；给 eax 赋值

_____；给 ebx 赋值

⑤ 对于全局变量，其空间分配在_____。而对于非静态的局部变量，其空间分配在_____，对应的地址表达方式一般为 disp(%ebp)、disp(%esp) 等。

二、 数据存储及 C 语句转换填空

在Linux环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fadd(int a, int b)
{
    int temp;
    temp = a + b;
    return temp;
}

void main( )
{
```

```
int x = 0x1234;

int y = -32;

int result = 0;

char msg[6] = "abc12"; // '1'的ASCII是 0x31, 'a'的ASCII是 0x61

result = fadd(x, y);

result = *(int *) (msg+1);

}
```

调试时，设变量 x 的地址（即 $\&x$ ）为 $0xffffd508$ ； y 的地址(即 $\&y$) 为 $0xffffd50c$, $result$ 的地址为 $0xffffd510$ ，数组 msg 的起始地址为 $0xffffb516$ 。

1、执行到“ $result = fadd(x, y);$ ”时，以字节为单位观察内存内容（用16进制数的形式填空，最左边是内存地址）。

$0xffffd508$	_____	_____	_____	_____	_____	_____	_____
$0xffffd510$	_____	_____	_____	_____	XX	XX	_____
$0xffffd518$	_____	_____	_____	_____	XX	XX	XX XX

2、数据传送指令解读

“ $int\ x = 0x1234;$ ”对应的机器指令为： $movl\ \$0x1234,\ -0x20(\%ebp)$ ，执行该语句时， $ebp = 0x$ _____。

“ $int\ result = 0;$ ”对应的反汇编指令为_____。

执行“ $result = *(int *) (msg+1);$ ”后， $result$ 中的值为 $0x$ _____。

3、函数调用语句解读

语句“ $result = fadd(x, y);$ ”对应的反汇编代码（最左边的是机器指令的地址）如下。

0x56556210 <+72>:	push -0xc(%ebp)		0xffffd4ec
0x56556213 <+75>:	push -0x20(%ebp)		0xffffd4f0
0x56556216 <+78>:	call 0x5655619d <fadd>		0xffffd4f4
0x5655621b <+83>:	add \$0x8, %esp		0xffffd4f8
0x5655621e <+86>:	mov %eax, -0x18(%ebp)		0xffffd4fc
		XXXXXXXX	0xffffd500

设执行“ $result = fadd(x, y);$ ”之前， esp 的值为 $0xffffd500$ 。

- ① 在表格的适当位置填写刚进入函数 $fadd$ 内部时，堆栈中存放的相关数据（6分）；
- ② 刚进入函数 $fadd$ 内部时， $esp = 0x$ _____；
- ③ 执行“ $add\ \$0x8,\ \%esp$ ”之后， $esp = 0x$ _____。

4. 函数 fadd 的指令解读

函数体对应的反汇编代码有：

```
0x5655619d <+0>:    push  %ebp
0x5655619e <+1>:    mov   %esp, %ebp
0x565561a0 <+3>:    sub   $0x10, %esp
0x565561a3 <+6>:    mov   0x8(%ebp), %edx
0x565561a6 <+9>:    mov   0xc(%ebp), %eax
0x565561a9 <+12>:   add   %edx, %eax
0x565561ab <+14>:   mov   %eax, -0x4(%ebp)
0x565561ae <+17>:   mov   -0x4(%ebp), %eax
0x565561b1 <+20>:   .....
```

- ① 函数参数 a 的地址（即&a）是 0x_____。
- ② 局部变量 temp 的地址（即&temp）是 0x_____。
- ③ 执行 “add %edx, %eax” 后， CF=____, SF=____, ZF=____, OF=_____。
- ④ 在函数的结束处，有程序段

```
_____  
_____  
ret
```

执行后可返回到调用函数的断点处。

三、 程序阅读与理解

1、阅读下面的程序，回答问题 。

```
void func ()  
{  unsigned short us = 65535;  
  unsigned int ui;  
  short s = -1;  
  int i;  
  int x = 0, y = 0;          ; ①  
  ui = us;  
  i = s;                      ; ②  
  if (ui > 0) x = 1;  
  if (i > 0) y = 1;           ; ③  
  ui = ui >> 1;               ; >>1 右移一个二进制位  
  i = i >> 1;                 ; ④  
  ui = ~ui;                   ; 按位取反  
  i = -i;                     ; ⑤
```

}

(1) 执行完 ① 处语句后,

us = 0x _____ s = 0x _____ (2 个字节的 16 进制数)

(2) 执行完 ② 处语句后,

ui = 0x _____ i = 0x _____ (4 个字节的 16 进制数)

ui=us; 对应的机器指令: _____ us, %eax 、 mov %eax, ui

i=s; 对应的机器指令: _____ s, %eax 、 mov %eax, i

(3) 执行完 ③ 处语句后,

x = _____ y = _____

if (ui>0) x=1; 对应的机器指令: cmp \$0, ui、 _____ lp1、 mov \$1, x、 lp1:...

if (i >0) y=1; 对应的机器指令: cmp \$0, i、 _____ lp2、 mov \$1, y、 lp2:...

(4) 执行完 ④ 处语句后,

ui = 0x _____ i = 0x _____ (4 个字节的 16 进制数)

ui = ui >> 1; 对应的机器指令: _____ \$1, ui

i = i >> 1; 对应的机器指令: _____ \$1, i

(5) 执行完 ⑤ 处语句后,

ui = 0x _____ i = 0x _____ (4 个字节的 16 进制数)

ui = ~ui; 对应的机器指令: _____ ui

i = -i; 对应的机器指令: _____ i

2、调试一个 C 程序时, 在函数的反汇编中看到源程序语句与对应的反汇编语句如下。根据观察到的信息填空。

int a[2][5];

int i = 1;

movl \$1, -0x2c(%ebp)

i 的地址为 0x0019feb8, ebp = 0x _____

int j = 3;

movl \$3, -0x30(%ebp) &j = 0x _____

a[i][j] = 10;

imul \$0x14, -0x2c(%ebp), %eax

执行后, eax = _____

该值的含义是 _____

lea -0x28(%ebp, %eax, 1), %ecx

ecx 中存放的值的含义是 _____

mov -0x30(%ebp), %edx

movl \$0xa, (%ecx, %edx, 4)

指令中的 edx 乘 4 的原因是 _____

a[0] 的地址表达形式是 _____;

global[i] = 18; global 为一个全局整型数组

mov -0x2c(%ebp), %eax

movl \$0x12, 0x417120(, %eax, 4)

global 数组的起始地址是_____，

```
int* p = &global[1];
```

```
mov    $4, %eax
```

```
shl    $0, %eax
```

```
add    $0x417120, %eax
```

```
mov    %eax, -0x34(%ebp)
```

变量 p 中的值是 0x_____

```
*p = 19;
```

```
mov    -0x34(%ebp), %eax
```

```
movl   $0x13, (%eax)
```

第二条汇编语句访存的寻址方式是_____

3、 以下是结构 test 的声明。假设在 32 位 Windows 平台上编译（采用缺省的自然对齐方式），问结构成员 d 和 v 的偏移量是多少？ 结构体所占存储空间是多少字节？ 如何调整成员的先后顺序使得结构所占存储空间最小？

```
struct{
    char c;
    int i;
    double d;
    short s;
    double *v;
    long l;
} test;
```

4、 阅读下面的程序，回答问题 。

```
.section .data
array: .long 10, -20, 30, -40, 50
length = (. - array)/4      # length 为 array 中元数的个数，= 5
format: .ascii "%d\n\0"

.section .text
.global _start
_start:
mov    $0, %eax
mov    $length, %ecx
lea    array, %edi          # ①
lp_1:
cmpl   $0, (%edi)
jl     lp_2                 # ②
inc    %eax
lp_2:
add    $4, %edi
sub    $1, %ecx             # ③
jne    lp_1
push   %eax
```

第 6 页 共 17 页

```
push $format
call printf
mov $1, %eax    # 程序正常退出
mov $0, %ebx
int $0x80
```

- 1) 上述程序的功能是什么？运行后，屏幕上显示的是什么？
 - 2) 若标号 lp_1 写到 ①处语句前，程序运行的结果是什么？为什么？
 - 3) 若将 ② 处的语句改为 “jb lp_2”，程序运行的结果是什么？
 - 4) 若漏写了 ③ 处的语句，程序运行会出现什么现象？为什么？
- 5、已知 **function_test** 的 C 语言代码框架如下，根据对应的汇编代码填写 C 代码中缺失部分。

```
int function_test( unsigned x)    // unsigned int
{
    int result=0;
    int i;
    for ( ① ; ② ; ③ ) {
        ④ ;
        ⑤ ;
    }
    return result;
}
```

上述函数过程体对应的汇编代码如下：

```
1  movl    8(%ebp), %ebx
2  movl    $0, %eax
3  movl    $0, %ecx
4  .L12:
5  leal    (%eax,%eax), %edx
6  movl    %ebx, %eax
7  andl    $3, %eax
8  orl     %edx, %eax
9  shrl    %ebx    // shr $1, %ebx
10 addl    $1, %ecx
11 cmpl    $64, %ecx
12 jne     .L12
    .....
ret
```

参见教材 P138 -P139 例 3-13 的分析

四、 程序优化

1、 举例说明编写 C 程序或者编译器优化时，利用 CPU 特性的做法（包括优化前的方法，优化后的方法）。

- ① 提高 CPU 中 cache 的命中率
- ② 提高 CPU 中指令流水线的利用率
- ③ 使用 CPU 中处理速度更快的指令
- ④ 使用 CPU 中单指令多数据流指令或串操作指令
- ⑤ 使用多核 CPU 中多线程处理能力

2、 在编写程序（也包括编译器）可以做哪些 与 CPU 无关的优化工作？（给出 5 种不同类型的示例）。

以下优化不依赖特定 CPU 架构，适用于通用编译器和程序开发：

(1) 常量传播 (Constant Propagation)

原理：在编译时计算表达式中已知常量的值，直接替换结果。

示例：

```
int a = 10;
int b = a * 2; // 优化为 int b = 20;
```

(2) 死代码消除 (Dead Code Elimination)

原理：移除永远不会执行的代码（如不可达分支或未使用的变量）。

示例：

```
if (false) {
    printf("This will never run"); // 直接删除
}
```

(3) 循环不变量外提 (Loop Invariant Code Motion)

原理：将循环内不变的计算移到循环外，减少重复计算。

示例：

```
for (int i = 0; i < n; i++) {
    int x = y * z; // y 和 z 在循环内不变，外提到循环外
    arr[i] = x;
}
```

(4) 函数内联 (Function Inlining)

原理：将小函数调用替换为函数体本身，减少调用开销。

示例：

```
inline int square(int x) { return x * x; }  
int a = square(5); // 优化为 int a = 25;
```

(5) 公共子表达式消除 (Common Subexpression Elimination, CSE)

原理：识别并重用重复计算的表达式。

示例：

```
int a = x * y + z;  
int b = x * y + w; // x*y 只计算一次
```

解答内容不得超过装订线

3、为了提高程序运行的安全性，编译器有多种编译开关（选项），增强发现程序漏洞的能力。试举出二种编译开关的例子，并说明增强安全性的实现原理。【堆栈帧检查、未初始化变量使用检查】

(1) 堆栈帧检查 (Stack Frame Protection)

编译选项：GCC/Clang: `-fstack-protector` 或 `-fstack-protector-all`

原理：

在函数栈帧中插入 金丝雀值 (Canary)，在函数返回前检查该值是否被修改。

若检测到溢出（如缓冲区溢出覆盖了返回地址），立即终止程序。

效果：防止栈溢出攻击（如 ROP 攻击）。

(2) 未初始化变量检查 (Uninitialized Variable Check)

编译选项：

GCC/Clang: `-Wuninitialized`（警告）或 `-O2`（自动优化时报错）

MSVC: `/RTC1`

原理：

静态分析或运行时检查变量是否未初始化即被使用。

对局部变量（尤其是栈上的变量）进行标记，访问时验证其初始化状态。

示例：

```
int x;
printf("%d", x); // 编译时报错或运行时终止
```

(3) 其他常见安全选项

地址随机化（ASLR）：

原理：随机化代码和数据的内存地址，增加攻击者预测难度。

格式化字符串检查：

原理：禁止直接使用用户输入作为 `printf` 的格式化字符串。

五、 链接和异常控制流问答

1、 设一个函数中有语句 `int temp=global;` 其中 `global` 是一个初值为 35 的 `int` 类型全局变量。编译器对 `global` 的定义(`int global=35;`)和 `temp=global` 编译时，分别会在可重定位目标文件中的哪些节生成哪些信息？

对于 `int global=35;` 在 数据节(.data) 节存放 35 (0x00000023)；在 字符串节(.strtab)，存放字符串 “global”；在符号表节 (.symtab)，存放有关符号 `global` 的信息，包括定义该符号的节号、在相应节 (.data) 节的地址、数据长度（4 个字节）、属性信息等。

对于 `int temp=global;` 在代码节(.text)中，生成相应的语句，其中 `global` 的地址（偏移量）用占位符 `0x00000000`。在代码节的重定位节(.rel.text)，要记录 代码节的相应位置（`global` 占位符的起始地址）要被什么符号（符号表的哪一项）、用什么定位方式（地址相对程序计算器 PC 的 32 位偏移）所代替，还包括计算地址时的附加项等。

2、 可重定位目标文件中有哪些节？各节中主要有什么信息？什么是符号解析？链接的过程是什么？

1. 可重定位目标文件中的节（Sections）及各节信息

可重定位目标文件（如 .o 文件）通常包含以下核心节（以 ELF 格式为例）：

节名	作用	存储内容
.text	代码节	程序的机器指令（函数实现、逻辑代码）。
.data	已初始化数据节	已初始化的全局变量和静态变量（如 <code>int global = 42;</code> ）。
.bss	未初始化数据节	未初始化的全局变量和静态变量（如 <code>int buffer[100];</code> ），仅记录大小，不占空间。
.rodata	只读数据节	常量数据（如字符串字面量 "Hello"、 <code>const</code> 变量）。
.symtab	符号表	所有符号（函数、全局变量）的名称、类型、大小、所在节等信息。
.strtab	字符串表	符号名称字符串（如 "main"、"global"）。

解答内容不得超过装订线

.rel.text	代码重定位表	.text 节中需要重定位的指令（如函数调用、全局变量引用）。
.rel.data	数据重定位表	.data 节中需要重定位的数据（如全局指针 <code>int* p = &global;</code> ）。
.debug	调试信息（可选）	调试符号(如行号、变量类型)，需 <code>-g</code> 选项生成。
.line	行号信息（可选）	源代码行号与机器指令的映射。
.comment	注释信息（可选）	编译器版本信息（如 GCC: (Ubuntu 11.3.0)）。

2. 符号解析 (Symbol Resolution)

定义
符号解析是链接器的核心任务之一，目的是将每个符号（如函数名、变量名）的引用与其定义关联起来。

符号类型：

全局符号：定义在当前文件（如 `int global;`）或外部文件（`extern int global;`）。
局部符号：静态函数/变量（如 `static int x;`），仅当前文件可见。
外部符号：未定义的引用（如 `printf`）。

解析规则

强符号 vs 弱符号：
强符号：函数名或已初始化的全局变量（如 `int x = 5;`）。
弱符号：未初始化的全局变量（如 `int x;`）。
规则：强符号优先，重复强符号会报错；弱符号可被覆盖。

解析过程：

链接器遍历所有目标文件的 `.symtab`，为每个符号找到唯一的定义。
若找不到定义，报 `undefined reference` 错误（如未链接 `-lm` 时调用 `sqrt`）。

3. 链接的过程 (Linking)

链接器将多个可重定位目标文件（.o）合并为可执行文件（a.out），主要步骤如下：

（1）符号解析 (Symbol Resolution)

将所有目标文件的符号表（.symtab）合并，解决符号引用。
示例：

main.o 调用 `foo()`，链接器在 lib.o 中找到 `foo` 的定义。

（2）重定位 (Relocation)

地址分配：为每个节（.text、.data 等）分配运行时内存地址。
修正引用：根据重定位表（.rel.text、.rel.data）修改代码和数据中的占位符。
绝对地址修正：如全局变量 `global` 的地址。
相对地址修正：如 `call foo` 指令的偏移量。

（3）合并节 (Section Merging)

将所有输入文件的 `.text` 节合并到输出文件的 `.text` 节，其他节同理。

示例：

`main.o` 和 `lib.o` 的 `.text` 合并为可执行文件的 `.text`。

(4) 生成可执行文件

写入 ELF 头、程序头（描述内存布局）、节头表等元信息。

最终文件：包含可直接加载到内存执行的代码和数据。

3、什么是中断和异常？两者有何差别？什么是中断描述符表？中断和异常的响应过程是什么？

中断和异常：一个进程在执行过程中，正常的逻辑控制流被特殊的事件所打断，CPU 转到处理这些事件的内核程序去执行，从而引发一个异常控制流。

中断一般是指 CPU 之外的事件发生，如按键盘，鼠标等，是与当前正在执行的指令无关的异步事件。异常是 CPU 正在执行的指令引发的事件，如除 0、访问数据的地址超出程序空间范围等等，是与正在执行的指令相关的同步事件。异常分为故障、陷阱和终止。

中断描述表（中断矢量表）：中断服务程序和异常处理程序的入口地址构成的一个表，每个表项占 8 个字节，存入口偏移地址及相应代码段的描述信息；这些表项是按照中断和异常的编号（即中断和异常的类型号）顺序排列的。

中断和异常的响应过程：CPU 的控制逻辑确定检测到的中断和异常类型号，从中断描述表取出对应的表项，计算相应的处理程序的入口地址，保存当前的 CS、EIP、EFLAGS 等信息，然后转到处理程序的入口地址对应的位置去执行。在中断处理程序中有 `IRET` 指令，执行该指令从堆栈中恢复 CS、EIP、EFLAGS 等，从而回到被中断的位置继续执行。对于三类异常：故障、陷阱和终止的处理策略有所差异。故障表示能够修复，引起故障的指令会被再次执行；陷阱则是执行引发陷阱的下一条指令；而终止就要结束程序的运行。

4、在一个程序的运行过程中（如正在执行一个二维数组求累加和），用户按了键盘上的某个键，计算机系统会做出哪些响应（即一系列的处理过程）？中断分哪几类？请举例说明各类中断在何种情况下产生。

六、 已知以下关于 Lab3 Bang 阶段的信息，请完成填空，注意涉及数值全部使用 16 进制。

解答内容不得超过装订线

08048e6d <test>:

8048e6d:	55	push	%ebp
8048e6e:	89 e5	mov	%esp,%ebp
8048e70:	53	push	%ebx
8048e71:	83 ec 24	sub	\$0x24,%esp
8048e74:	e8 6e ff ff ff	call	8048de7 <uniqueval>
8048e79:	89 45 f4	mov	%eax,-0xc(%ebp)
8048e7c:	e8 6b 03 00 00	call	80491ec <getbuf>
8048e81:	89 c3	mov	%eax,%ebx
8048e83:	e8 5f ff ff ff	call	8048de7 <uniqueval>
.....			

080491ec <getbuf>:

80491ec:	55	push	%ebp
80491ed:	89 e5	mov	%esp,%ebp
80491ef:	83 ec 38	sub	\$0x38,%esp
80491f2:	8d 45 d8	lea	-0x28(%ebp),%eax
80491f5:	89 04 24	mov	%eax,(%esp)
80491f8:	e8 55 fb ff ff	call	8048d52 <Gets>
80491fd:	b8 01 00 00 00	mov	\$0x1,%eax
8049202:	c9	leave	
8049203:	c3	ret	

08048d05 <bang>:

8048d05:	55	push	%ebp
8048d06:	89 e5	mov	%esp,%ebp
8048d08:	83 ec 18	sub	\$0x18,%esp
8048d0b:	a1 18 c2 04 08	mov	0x804c218,%eax
8048d10:	3b 05 20 c2 04 08	cmp	0x804c220,%eax
8048d16:	75 1e	jne	8048d36 <bang+0x31>
8048d18:	89 44 24 04	mov	%eax,0x4(%esp)
8048d1c:	c7 04 24 e4 a2 04 08	movl	\$0x804a2e4,(%esp)
8048d23:	e8 a8 fb ff ff	call	80488d0 <printf@plt>
.....			


```
Breakpoint 2, 0x80491f2 in getbuf ()
(gdb) info r
eax                0x6f50c1c5      1867563461
ecx                0xf7fbd068      -134492056
edx                0xf7fbd3cc      -134491188
ebx                0x0          0
esp                0x55683458      0x55683458 <_reserved+1037400>
ebp                0x55683490      0x55683490 <_reserved+1037456>
esi                0x55686018      1432903704
edi                0x1          1
eip                0x80491f2      0x80491f2 <getbuf+6>
.....
```

```
(gdb) x 0x804c218
0x804c218 <global_value>: 0x00000000
```

```
acd@ubuntu:~/Lab3$ ./makecookie U201414XXX
0x250d3ee8
```

```
int global_value = 0;

void bang(int val)
{
    if (global_value == cookie) {
        printf("Aha Bang!: You set global_value to 0x%x.\n", global_value);
        validate(2);
    } else
        printf("Oh Misfire: global_value = 0x%x\n", global_value);
    exit(0);
}
```

- 1)调用 `getbuf` 后的返回地址是: (1) **0x08048d05**
- 2)调用 `getbuf` 时, 存放 `getbuf` 返回地址的单元是: (2)
- 3)调用 `getbuf` 时, 执行到断点 2 时, `getbuf` 的栈帧栈顶地址是: (3)
- 4)`getbuf` 中缓冲区 `buf` 的起始地址是: (4)
- 5)攻击字符串的长度为: (5) 字节
- 6)Cookie 值 `0x250d3ee8` 所在的存储单元地址是: (6)
- 7)以下是所设计的一个带有注释的攻击字符串文件的内容, 请填空:

c3 c3 c3 c3

```

b8      (7)                /* mov      (8)                ,%eax */
a3 18 c2 04 08             /* mov      %eax, (9)                */
68 05 8d 04 08             /* push    (10)                */
c3                        /* ret      */

```

90 90

- 14) (11) (12) (13) 55
 15)上述攻击字符串可使 bang 输出: (14)

七、假设一个 c 语言程序有两个模块: main.c 和 swap.c, 对它们单独编译, 分别生成可重定位目标文件 main.o 和 swap.o。 main.c 和 swap.c 代码如下:

main.c

```
int buf[2] = {1, -4};
extern void swap( );
int sum=0;
int main( )
{
    swap( );
    return 0;
}
int ave( ) {
    int a;
    static int count=2;
    sum=buf[0]+buf[1];
    a=sum/count;
    return a;
}
```

swap.c

```
extern int buf[];
extern int ave();
int *bufp0 = &buf[0];
static int *bufp1;
int sum;
void swap( ) {
    int temp;
    bufp1 = &buf[1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
    ave( );
}
```

解答内容不得超过装订线

请指出 main.o 和 swap.o 中所有强符号和弱符号分别有哪些?

请指出 swap.o 的符号表中分别有哪些符号?

请指出 swap.o 的符号表中各符号分别是什么类型的符号(全局符号, 本地符号或外部符号)? 请指出 swap.o 的符号表中各符号分别出现在 swap.o 中的哪个节(.text, .data 或.bss)?

(2) 假定 swap.o 的反汇编代码如下：

```
00000000 <swap>:
 0: 55                push    %ebp
 1: 89 e5             mov     %esp, %ebp
 3: 83 ec 18          sub     $0x18, %esp
 6: c7 05 00 00 00 04 movl    $0x4, 0x0      # 重定位位置①
 d: 00 00 00
    8: R_386_32 .bss
    c: R_386_32 buf
10: a1 00 00 00 00    mov     0x0, %eax      # 重定位位置②
    11: R_386_32 bufp0
15: 8b 00             mov     (%eax), %eax
17: 89 45 f4          mov     %eax, -0xc(%ebp)
1a: a1 00 00 00 00    mov     0x0, %eax      # 重定位位置③
    1b: R_386_32 bufp0
1f: 8b 15 00 00 00 00 mov     0x0, %edx
    21: R_386_32 .bss      # 重定位位置④
25: 8b 12             mov     (%edx), %edx
27: 89 10             mov     %edx, (%eax)
29: a1 00 00 00 00    mov     0x0, %eax      # 重定位位置⑤
    2a: R_386_32 .bss
2e: 8b 55 f4          mov     -0xc(%ebp), %edx
31: 89 10             mov     %edx, (%eax)
33: e8 fc ff ff ff    call    34 <swap+0x34> # 重定位位置⑥
    34: _____ ⑥
38: 90                nop
39: c9                leave
3a: c3                ret
```

假定可执行目标文件里 swap 函数代码的起始地址是 0x00808100，swap 函数代码紧接在 ave 函数代码的后面，且 ave 函数代码占 0x2e 个字节。对 swap.o 中的重定位位置⑥进行重定位。回答：此处待重定位的符号是什么？重定位类型是什么？重定位前的值是什么？该值的含义是什么？重定位后的值是什么（需要给出计算过程）？重定位后最终指向的虚拟地址是多少？

问题	答案
待重定位的符号	ave
重定位类型	R_386_PC32（PC 相对偏移）
重定位前的值	0xffffffffc（即 -4）

解答内容不得超过装订线

该值的含义	未重定位时无效的占位符偏移量 (跳转到 <code>call</code> 自身)。
重定位后的值	<code>0xffffffffc6</code> (计算: <code>ave</code> 地址 - <code>PC</code> = - <code>0x3a</code>)
最终指向的虚拟地址	<code>0x008080d1</code> (<code>ave</code> 的入口地址)

25春A卷答案

2026年5月28日 10:00



25春A卷答
案



华中科技大学 2024~2025 第二学期
“ 计算机系统基础 ” 考试试卷 (A 卷)

考试方式 闭卷 考试时间 2025-5-13 晚上 考试时长 150 分钟

院 (系) _____ 专业班级 _____

学 号 _____ 姓 名 _____

题号	一	二	三	四	五			总分	总分人	核对人
分值	20	20	30	20	10			100		
得分										

分 数	
评卷人	

一、(20 分) 计算机工作的基本原理。

1、正常控制流下的程序执行 (10 分, 每空 1 分)

① 冯·诺依曼体系结构的基本思想是存储程序和自动执行。程序计数器的自动变化是实现程序自动执行的核心机制之一。以下是 Intel CPU 中的 rip 自动变化的方法。在加载运行一个程序时, 操作系统和 CPU 会协同工作, 将可执行文件中记载的入口地址送入 rip 中。设 rip 为 0x08001144, 该处指令为“c7 45 f4 05 00 00 00 movl \$0x5,-0xc(%rbp)”, 在取出该指令并进行译码后, rip = 0x800114b。在 0x0800114f 处, 有指令 “75 09 jne 0x800115a”, 若执行该指令前, ZF=0, 指令执行后 rip = 0x800115a; 若 ZF=1, 指令执行后 rip = 0x 8001151。

在 0x08001158 处, 有反汇编语句“eb xx jmp 0x8001161”, 这里机器码(1 个字节)xx = 07。

在执行 CALL 指令时, CPU 会将 rip 压入堆栈栈顶, 同时会将 子程序入口地址 送入 rip 中。在子程序返回时, 执行 ret 指令, 会从 栈顶弹出 8 个字节 送入 rip 中。

2、异常控制流下的 rip 的变化 (10 分, 每空 1 分):

在执行一条指令后, CPU 检测到有按键操作, 此时会发生 中断。CPU 会依次保存标志寄存器和 rip, 同时根据中断号从 中断矢量表 中找到 中断处理程序的入口地址 送给 rip。中断处理程序保护恢复通用寄存器。iret 指令先将 栈顶 8 个字节弹出送 RIP, 再 弹出标志寄存器, 从而恢复被打断程序的继续执行。若 CPU 在执行除零指令或者指令中访问的地址超出了本程序的空间范围, 此时会产生 异常。CPU 执行异常处理程序后, 若继续执行引起异常的指令, 则称之为 故障; 若执行引起异常的指令的下一条指令, 则称之为 陷阱, 其代表性的事件有 软中断或者调试跟踪。

分 数	
评卷人	

二、(20 分) 数据的存储和地址表达形式

1. 数据的存储形式 (10 分)
设有如下程序段，在运行时观察了各变量的地址如下

```
int x = 10; // &x = 0x08004010  
int y = -10; // &y = 0x8004014  
char str[10] = "20250513"; // &str[0] = 0x8004018  
char a[4] = {255, -1, 257, -257}; // &a[0] = 0x8004022  
float f = 20.5; // &f = 0x8004028  
short u = 30; // &u = 0x800402c
```

以 16 进制的形式填写程序用到的内存单元内容(无关的字节中的内容填写 XX，每行 2 分)。

0x08004010:	<u>0A</u>	<u>00</u>	<u>00</u>	<u>00</u>	<u>F6</u>	<u>FF</u>	<u>FF</u>	<u>FF</u>
0x08004018:	<u>32</u>	<u>30</u>	<u>32</u>	<u>35</u>	<u>30</u>	<u>35</u>	<u>31</u>	<u>33</u>
0x08004020:	<u>00</u>	<u>XX</u>	<u>FF</u>	<u>FF</u>	<u>01</u>	<u>FF</u>	<u>XX</u>	<u>XX</u>
0x08004028:	<u>00</u>	<u>00</u>	<u>a4</u>	<u>41</u>	<u>1E</u>	<u>00</u>	<u>XX</u>	<u>XX</u>

`*(int*)(str+2)` 的值是 0x 35303532
`*(char*)&x+1` 的值是 0x F6

提示：'0' 的 ASCII 为 0x30；浮点数 float 的编码规则：float 用 32 位表示，最高位（第 31 位）为符号位，23-30 位为用移码表示的指数（移码值为 0x7F），0-22 位为尾数位。

2. 根据调试一个 C 程序时看到的如下信息填空。(10 分，每空 1 分)

```
int a[5];  
int i=3;  
movl $0x3, -0x44(%rbp) // rbp = 0x7ffffffe040
```

此时，i 的地址是 0x 7ffffffedffc，i 存放在 堆栈 段中。i 的地址表达形式是 -0x44(%rbp)，该表达形式是在 生成执行程序 时就固定不变了。但是 i 的具体地址值在不同的执行时刻，因 rbp 的变化而变化。

```
int *p=&a[0];  
lea -0x20(%rbp), %rax ; 执行后，rax = 0x 7ffffffe020  
mov %rax, -0x38(%rbp) ; p 的地址是 0x7ffffffe008  
a[0]=1;
```

```

movl    $0x1, -0x20 (%rbp)
*(a+1)= 2 ;
movl    $0x2, -0x1c(%rbp)
*(p+2)=3;
mov     -0x38(%rbp), %rax
add     $0x08, %rax
movl    $0x3, (%rax)
a[i]=5;
mov     -0x44(%rbp), %eax
cltq    ; 将 rax 的高 32 位清零
movl    $0x5, -0x20(%rbp,%rax, 4 )

```

分 数	
评卷人	

三、(30 分) C 程序与机器语言的对应

1、数据传送、算术运算、按位运算、移位运算（10 分，每小题 1 分）

(写汇编语句时，要求有指令助记符、操作数和操作数地址，用变量符号名表示操作数地址)

设有 `int x=0x12345678;`

`unsigned short u=0xffff; short v=0xffff;`

分别执行如下语句（每条语句执行前，x 都是 0x12345678）

- ① `x=u;` 数据扩展传送指令的指令助记符是 movzwl，执行后，`x=` 0x0000ffff。
- ② `x=v;` 数据扩展传送指令的指令助记符是 movswl，执行后，`x=` 0xffffffff。
- ③ `u=u>0;` `u>0` 是（是、否）成立；执行后 `u=` 1。
- ④ `v=v>0;` `v>0` 否（是、否）成立；执行后 `v=` 0。
- ⑤ `x = x>>2` 对应的汇编语句是 sar \$2, x，执行后，`x=0x` 048d159e。
- ⑥ `x = x<<1` 对应的汇编语句是 shl \$1, x，执行后，`x=0x` 2468acf0。
- ⑦ `x=x&0x0f` 对应的汇编语句是 and \$0x0f, x，执行后，`x = 0x` 00000008。
- ⑧ `x = -x` 对应的汇编语句是 neg x，执行后，`x = 0x` edcba988。
- ⑨ `x = ~x` 对应的汇编语句是 not x，执行后，`x = 0x` edcba987。
- ⑩ `x=x+10` 对应的汇编语句是 addl \$0x0a, x，执行后，`x = 0x` 12345682。

2、分支转移程序（10 分，每空 1 分）

① if 语句与汇编语句的对应（5 分，每空 1 分）

（L1、L2、L3、L4 皆为标号，代表机器指令的地址）

```
int x= -1;    int y=1;

    movl    $0xffffffff, -0xc(%rbp)

    movl    $0x1, -0x8(%rbp)

int flag;

if (x>y)    flag=1;

    mov     -0xc(%rbp), %eax

    cmp     -0x8(%rbp), %eax

L1:  jle    L3

    movl    $0x1, -0x4(%rbp)

L2:  jmp    L4

else    flag=0;

L3:  movl    $0, -0x4(%rbp)

L4:  .....
```

若将变量 x、y 定义为 unsigned int 类型，L1 处的汇编语句应为 jbe L3。

若漏写了 L2 处的语句，与该 if 语句功能等价的 C 语句是 flag=0。

② switch 语句与汇编语句的对应（5 分，每空 1 分）

```
int x=3;

    movl    $0x3, -0x8(%rbp)

int y;

switch (x)

    cmpl    $0x1, -0x8(%rbp)

    je      L1

    cmpl    $0x2, -0x8(%rbp)

    je      L3

    jmp     L5

{
```

```

        case 1: y=1; break;

L1:     movl    $0x1, -0x4(%rbp)

L2:     jmp     L6

        case 2: y=2; break;

L3:     movl    $0x2, -0x4(%rbp)

L4:     jmp     L6

        default: y=3;

L5:     movl    $0x3, -0x4(%rbp)

}

L6: .....

```

若将 default 分支移到 case 1 之前，switch 语句执行后，y= 1。

若要使 default 语句放在 case1 之前与放在最后的功能等价，default 语句应修改为：

default: y=3 ; break ;。

3、函数调用与汇编语句的对应 （10 分）

有如下主程序：

```

int a = 1;
0x000000000800116c <main+8>: movl    $0x1, -0x4(%rbp)

int b = 2;
0x0000000008001173 <main+15>: movl    $0x2, -0x8(%rbp)

int sum = 0;
0x000000000800117a <main+22>: movl    $0x0, -0xc(%rbp)

sum = fadd(a, b);
0x0000000008001181 <main+29>: mov     -0x8(%rbp), %edx
0x0000000008001184 <main+32>: mov     -0x4(%rbp), %eax
0x0000000008001187 <main+35>: mov     %edx, %esi
0x0000000008001189 <main+37>: mov     %eax, %edi
0x000000000800118b <main+39>: callq   0x8001139 <fadd>
0x0000000008001190 <main+44>: mov     %eax, -0xc(%rbp)

```

int fadd(int x, int y) 函数和反汇编代码如下：

```

{
0x0000000008001139 <+0>:    push    %rbp
0x000000000800113a <+1>:    mov     %rsp, %rbp
0x000000000800113d <+4>:    mov     %edi, -0x14(%rbp)
0x0000000008001140 <+7>:    mov     %esi, -0x18(%rbp)

    int u, v, w;
    u = 2*x ;

```

```

0x0000000008001143 <+10>:  mov    -0x14(%rbp), %eax
0x0000000008001146 <+13>:  add     %eax, %eax
0x0000000008001148 <+15>:  mov     %eax, -0x4(%rbp)
    v = y + 25;
0x000000000800114b <+18>:  mov    -0x18(%rbp), %eax
0x000000000800114e <+21>:  add     $0x19, %eax
0x0000000008001151 <+24>:  mov     %eax, -0x8(%rbp)
    w = u + v;
0x0000000008001154 <+27>:  mov    -0x4(%rbp), %edx
0x0000000008001157 <+30>:  mov    -0x8(%rbp), %eax
0x000000000800115a <+33>:  add     %edx, %eax
0x000000000800115c <+35>:  mov     %eax, -0xc(%rbp)
    return w;
0x000000000800115f <+38>:  mov    -0xc(%rbp), %eax
}
0x0000000008001162 <+41>:  pop     %rbp
0x0000000008001163 <+42>:  retq

```

刚进入 fadd 时（即 push %rbp 执行前），

rsp = 0x7ffffffe028；

rbp = 0x7ffffffe040

填写执行到 fadd 的 “pop %rbp” 语句时

的堆栈内容。

要求：在图中标明 x, y, u, v, w 的位置；

每行是 4 个字节，其中的内容用 0x...表示

具有随机性的单元内容填为

XXXXXXXX。

y	0x00000002	0x7ffffffe008
x	0x00000001	0x7ffffffe00c
	XXXXXXXXXX	0x7ffffffe010
w	0x0000001d	0x7ffffffe014
v	0x0000001b	0x7ffffffe018
u	0x00000002	0x7ffffffe01c
	0x7fff0040	0x7ffffffe020
	0x00007fff	0x7ffffffe024
	0x08001190	0x7ffffffe028
	0x00000000	0x7ffffffe02c

分 数	
评卷人	

四、(20 分) 程序编译与优化问答

1、程序优化（16 分）

① 求二维数组中所有元素的和，在二重循环中，行序优先访问比列序优先访问快，利用了 CPU 的什么特性和二维数据存储的什么特性？（2 分）

答：CPU 访问数据时使用 CPU 中的 cache，若数据不在 cache 中，就要将内存中的数据块调入 cache，有时间开销。二重数组按行序优先的顺序优存放，数据块调入 cache 时，把即将要

访问的数据也就调入了 **cache**。按行序访问，**cache** 的命中率高，速度快。

② 将有分支的语句转换成无分支的语句，或者将循环语句转换成无循环的语句，提高程序执行速度，利用了 CPU 的什么特性？为什么会提高速度？（2 分）

答：CPU 对指令进行处理是用指令流水线方式。提高指令流水线的利用率可加快程序执行速度。在发生转移时，已在流水线上进行加工了的“半成品”要作废，使得指令流水线的利用率低。

③ 将一大片空间中的内容置为 0，可以用循环语句一个字节一个字节的置零，也可以使用 **memset** 函数来完成，后者的执行速度更快，原因是什么？（2 分）

答：**memset** 中封装了串操作指令。CPU 中的串操作指令执行速度更快。

④ 写出与 **imul \$2, %eax** 功能等价但执行速度更快的两种机器指令（2 分）

答：
add %eax, %eax
shl \$1, %eax

⑤ 举例说明，编译器还可以做哪些与 CPU 特性无关的优化？（2 分）

答：去掉无作用的废语句。能在编译时，直接计算出结果的表达式或者函数调用，直接用计算出的结果，而不需要对中间过程产生语句，节省了程序运行时的指令数。

⑥ 设有如下程序段及其未优化的反汇编程序段（6 分）。

```
int a[5];
int sum=0, i;
    movl    $0x0, -0x28(%rbp)
for (i=0; i<5; i++)    sum+=a[i];
    movl    $0x0, -0x24(%rbp)
    jmp     L2
L1:  mov     -0x24(%rbp), %eax
    cltq    ; 将 rax 的高 32 位置 0
    mov     -0x20(%rbp,%rax,4), %eax
    add     %eax, -0x28(%rbp)
    addl    $0x1, -0x24(%rbp)
L2:  cmpl    $0x4, -0x24(%rbp)
    jle     L1
```

写出完成等价功能的尽可能优化的汇编程序段，并指出使用的优化策略。

答：优化策略有：循环展开、变量与寄存器绑定，**sum** 与 **eax** 绑定

```
mov     a[0], %eax
add     a[1], %eax
add     a[2], %eax
add     a[3], %eax
add     a[4], %eax
mov     %eax, sum
mov     $5, i
```

2、程序的安全性（4分）

① 为什么不应将函数中局部变量的地址返回给调用函数来使用？（2分）

答：局部变量分配在栈中。在一个函数执行完成后，该栈处于“释放”状态，可供下面执行的函数使用。即意味着相应单元中的内容被其他函数改变，而不再是从函数返回时的内容。

② 编译器生成的执行程序中，可采取何种方法检测出缓冲区溢出攻击？（2分）

答：在保存的断点地址之前，加一个金丝雀（哨兵），该值是一个随机值。在函数返回前，检查该值是否发生变化，若变化，则说明有溢出。

分 数	
评卷人	

五、（10分）程序的链接回答

1、设一个程序中，有全局变量定义语句 `int global=10;` （2分）

在可重定位目标文件中的**哪些节**，会生成与该变量定义有关的**哪些**信息？

答：字符串节，存放有 `global` 的 ASCII 串；
符号表节，记录 `global` 定义在数据节，`global` 的地址，符号是已定义状态；
数据节，存放 `global` 的具体值，有 `0A 00 00 00`。

2、在一个函数中有语句 `int temp=global;`（`global` 为前面定义的全局变量），在可重定位目标文件中的**哪些节**，会生成与该语句有关的**哪些**信息？（2分）

答：代码节，有 `temp=global` 对应的机器指令（非优化状态下 2 条指令）。
代码节的重定位节，有需要定位的信息，包括机器指令的什么位置、用什么符号、以何种重定位方式来重定位，还有计算重定位值时需要的附加值。

3、设函数中有语句 `printf(“very good\n”);` 字符串“very good\n”存放在什么节中？该 `printf` 语句对应的代码中有几处需要重定位？分别是什么位置需要重定位？（2分）

答：只读数据节，存放“very good\n”的 ASCII 串。有 2 个地方要重定，一是字符串的地址，二是 `printf` 函数的地址。

4、简述链接的执行过程，以及在链接时能够检测到的错误。（4分）

答：可以检测到符号重定义错误、符号未定义错误。
链接是将多个目标文件（.o 或 .obj）和库文件合并成一个可执行文件或共享库的过程。其核心目标是解析符号引用、分配内存地址，并最终生成可运行的程序。

将各个可重定位目标文件中代码节、数据节、只读节等分别合并到一个文件中，在合并时，要检查各个符号是否重复定义，若有重复定义报错；同时要记录哪些符号没有定义（只是引用了外部符号），在文件合并后，对未定义的符号要判断是否出现在链接库中，若还存在未定义符号，也要报错。

根据合并后的文件，确定符号的地址。根据符号地址以及重定位节信息，对程序中需要重定位的地方进行修改，完成重定位工作。

23秋B卷答案

2026年5月28日 9:59



23秋B卷答
案



华中科技大学计算机科学与技术学院 2023~2024 第一学期

“ 计算机系统基础 ” 考试试卷(B 卷)

考试方式 闭卷 考试日期 2024-3-18 考试时长 150 分钟

专业班级 学 号 姓 名

题号	一	二	三	四	五	总分	核对人
分值	20	20	20	20	20	100	
得分							

分 数	
评卷人	

一、数据的存储表示和访问（共 20 分）

1、数据的存储表示（10 分）

```
void func1()
{
    int x = 10;
    int y[2] = { 20, 32 };
    char msg[6] = "abcde";
    int* p = &x;
    short z = *(short*)(msg + 1);
    int u = *(int*)(msg + 1);
}
```

+1字节

2字节

设 x 的地址(&x) 为: 0xffffd508; y[0]的地址为: 0xffffd50c; msg[0] 的地址为: 0xffffd51a;
p 的地址(&p) 为: 0xffffd514; z 的地址为: 0xffffd518; u 的地址为: 0xffffd520。

在函数 func1 执行结束返回前, 以字节形式观察内存内容。以 16 进制的形式填写程序用到的内存单元内容（无关的字节中的内容不要填写, 打 XX）。

0xffffd508	0A	00	00	00	14	00	00	00
0xffffd510	20	00	00	00	08	D5	FF	FF
0xffffd518	62	63	61	62	63	64	65	00
0xffffd520	62	63	64	65	XX	XX	XX	XX

解答内容不得超过装订线

2、调试一个 C 程序时，在函数的反汇编中看到源程序语句与对应的反汇编语句如下。根据观察到的信息填空。（10 分）

```
int a[2][5];
```

```
int i = 1;
```

```
movl $1, -0x2c(%ebp)
```

i 的地址为 0x0019feb8, ebp = 0x 0019FEE4

```
int j = 3;
```

```
movl $3, -0x30(%ebp) &j = 0x 0019FEB4
```

```
a[i][j] = 10;
```

```
imul $0x14, -0x2c(%ebp), %eax
```

执行后, eax = 0x14

eax 中值的含义是 数组 a 中, 前 i-1 行元素的字节长度

```
lea -0x28(%ebp, %eax, 1), %ecx
```

ecx 中存放的值的含义是 数组元素 a[i][0] 的地址

```
mov -0x30(%ebp), %edx
```

```
movl $0xa, (%ecx, %edx, 4)
```

指令中的 edx 乘 4 的原因是 数组元素类型为 int, 字节长度为 4

a[0][0] 的地址表达形式是 -0x28(%ebp) (或 -0x14(%ecx));

```
global [i] = 18; global 为一个全局整型数组
```

```
mov -0x2c(%ebp), %eax
```

```
movl $0x12, 0x417120(, %eax, 4)
```

global 数组的起始地址是 0x417120,

```
int* p = &global[1];
```

```
mov $4, %eax
```

```
shl $0, %eax
```

```
add $0x417120, %eax
```

```
mov %eax, -0x34(%ebp)
```

变量 p 中的值是 0x 417124

```
*p = 19;
```

```
mov -0x34(%ebp), %eax
```

```
movl $0x13, (%eax)
```

第二条汇编语句访存的寻址方式是 寄存器间接寻址

解答内容不得超过装订线

分 数	
评卷人	

二、程序运行的基本过程（共 20 分）

- 1、在对某程序进行调试时，看到如下一段信息（10 分，每空 1 分）：
- ① 设当前 eip 为 0x00411650，该处指令为“83 7D FC 00 cmpl \$0, -4(%ebp)”，在取出该指令并进行译码后，eip = 0x00411654；
- ② 在 0x00411654 处，有指令“7E 08 jle 0041165E”，综合前一条指令，就是在 -4(%ebp) 小于等于 0 条件下，将 0x0041165E → eip，计算出该地址值的方法是 直接寻址；
- ③ 在 0x0041165C 处，有 jmp 指令“EB 08”，该指令对应的反汇编语句是：jmp 0x00411666；执行该指令时，会 无（有、无）条件跳转到 0x00411666 处；
- ④ 在 0x00411666 处，有一子程序直接调用指令“E8 65 FC FF FF callq 004112D0”。由此推断子程序的入口地址是 0x4112D0；指令中 FFFFC65 的含义是是 子程序入口地址和当前地址的相对位移；执行该 call 指令时，CPU 会将 0x0041166B 压入堆栈中；
- ⑤ 在 0x00411672 处，有指令“FF 55 F4 callq -0xc(%ebp)”，得到子程序入口地址的方法是 在一个局部变量中保存有子程序入口地址；
- ⑥ 执行 RET 指令时，CPU 会 将栈顶元素 → eip。

2、函数调用（10 分）

在Linux环境下，对一个C语言程序进行编译、链接、调试运行，程序片段如下。

```
int fmax(int a, int b) {
    004115A5 push    %ebp
    004115A6 mov     %esp, %ebp
    004115A8 sub     $0x10, %esp

    int result;
    if (a > b)
        004115A9 mov     0x8(%ebx), %eax
        004115AC cmp     0xc(%ebp), %eax
        004115AF jle     fmax+19h (04115B9h)
        result = a;
    004115B1 mov     0x8(%ebp), %eax
    004115B4 mov     %eax, -0x4(%ebp)
    004115B7 jmp     fmax+1Fh (04115BFh)
    else result = b;
    004115B9 mov     0xc(%ebp), %eax
```

```

004115BC  mov          %eax, -0x4(%ebp)
return result;
004115BF  mov          -0x4(%ebp), %eax
}

004115C2  mov          %ebp, %esp
004115C4  pop          %ebp
004115C5  ret

void main( ) { .....
int  x = fmax(10, -10);
00413863  push        $0xffffffff6h
00413865  push        $0xa
00413867  call        004115A5
0041386C  add         $8, %esp
0041386F  mov         %eax, -0x4(%ebp)
.....
}

```

设执行 main 函数中的 call 004115A5 之前，esp= 0xffffd500; ebp= 0xffffd510.

填写刚进入函数 fmax 时（执行 push %ebp 之前），堆栈中相关单元的内容。

函数参数 a 的地址（即&a）是 0x ffffd4f8。

局部变量 result 的地址是 0x ffffd4ec。

	0xffffd4f0
0x0041386c	0xffffd4f4
0xa	0xffffd4f8
0xffffffff6	0xffffd4fc
XXXXXXXXXX	0xffffd500

分 数		三、C 语句的转换 （共 20 分）
评卷人		

1、阅读下面的程序，回答问题 （10 分，每小题 2 分）。

```

void func ()
{
    unsigned short  us_x = 0x8000;
    unsigned int    ui_x;
    short          s_x = -0x8000;
    int            i_x;
    int            u = 0, v = 0;           // ①
    ui_x = us_x;
    i_x = s_x;                             // ②
    if (ui_x > 0)  u = -1;
    if (i_x > 0)  v = -1;                   // ③
    ui_x = ui_x >> 1;
    i_x = i_x >> 1;                         // ④
    ui_x = ~ui_x;
}

```

}

(1) 执行完 ① 处语句后,

us x = 0x 8000 s x = 0x 8000 (2个字节的16进制数)

(2) 执行完 ② 处语句后,

ui x = 0x 00008000 i x = 0x ffff8000 (4个字节的16进制数)

ui x=us x; 对应的机器指令: **movzwl** us x,%eax 、 mov %eax, ui x

i x=s x; 对应的机器指令: **movswl** s x,%eax 、 mov %eax,i x

(3) 执行完 ③ 处语句后,

$$u = -1 \quad v = 0$$

if (ui_x > 0) u=-1; 对应的机器指令: `cmp $0, ui_x, jbe(jna) process2,`
`mov $-1, u, process2;...`

```
if (i_x > 0) v=-1; 对应的机器指令: cmp $0, i_x、jle(jng) process3、
                                mov $-1, v、    process3:...
```

(4) 执行完 ④ 处语句后,

ui x=0x 00004000 i x=0x FFFC000 (4个字节的16进

ui x = ui x >> 1; 对应的机器指令: **shrl** \$1, ui x

i x = i x >> 1; 对应的机器指令: sarl \$1, i x

(5) 执行完 ⑤ 处语句后,

ui_x = 0x FFFFBFFF i_x = 0x 00004000 (4个字节的 16)

ui x = ~ui x; 对应的机器指令: notl ui x

$i \ x = -i \ x;$ 对应的机器指令: **negl** $i \ x$

2、程序优化（10分）。

设有如下一段C语言程序，请写出完成相同功能、执行速度尽可能快的汇编语言程序代码片段，并指出优化中利用了CPU中的哪些特性。

```
int  a[5], i, sum=0;
```

• • • • •

```
for (i = 0; i < 5; i++) sum += a[i];
```

#eax: 存放 sum 的值

#ecx: 存放 i 的值

```
mov    $0,    %eax
```

```
add    a(0,%ecx, 4)
```

```

        mov    $0, %ecx
label_a:
        add    a(, %ecx, 4), %eax
        add    a+4(, %ecx, 4), %eax
        add    a+8(, %ecx, 4), %eax
        add    a+12(, %ecx, 4), %eax
        add    a+16(, %ecx, 4), %eax
        mov    %eax, sum

```

将变量和寄存器绑定，利用寄存器代替内存变量，提高速度

利用 CPU 中的指令流水线，将循环展开，消除循环，提高速度

分 数	
评卷人	

四、程序阅读与理解（20 分）

1、阅读下面的程序，回答问题（10 分）。

```

.section .data
array: .long -1, -2, 11, -9, 5, 100
length = (. -array)/4          # length 为array中元数的个数, = 6
format: .ascii "%d\n\0"

.section .text
.global _start
_start:
    mov    $0, %eax
    mov    $length, %ecx
    mov    $0, %esi            # ①
lp_1:
    cmpl   $10, array(, %esi, 4)
    jg     lp_2                # ②
    inc    %eax
lp_2:
    inc    %esi
    sub    $1, %ecx            # ③
    jne    lp_1
    push   %eax
    push   $format
    call   printf
    mov    $1, %eax            # 程序正常退出
    mov    $0, %ebx

```

```
int $0x80
```

1) 上述程序的功能是什么? 运行后, 屏幕上显示的是什么? (2 分)

统计并显示数组 `array` 中, 不大于 10 的元素个数。

运行后, 屏幕上显示 4。

2) 若标号 `lp_1` 写到 ①处语句前, 程序运行的结果是什么? 为什么? (3 分)

程序运行结果为 6, 因为始终在比较 `array` 的第一个元素-1, 比较了 6 次, 每次都不大于 10。

3) 若将 ② 处的语句改为 “`ja lp_2`”, 程序运行的结果是什么? (2 分)

程序运行结果为 1。

4) 若漏写了 ③ 处的语句, 程序运行会出现或什么现象? 为什么? (3 分)

则程序进入死循环, 最终异常终止。因为 `%ecx` 的值不变, 程序会一直循环。但 `%esi` 的值会不断增大, 最终 `cmpl $10, array(, %esi, 4)` 访问内存超出范围, 会引起异常退出。

2. 程序填空 (10 分, 每空 1 分)

将一个以 0 结束的字符串中的所有的大写字母转换为对应的小写字母, 并将转换后的字符串、以及修改的字母个数输出。

```
.....
.section .data
    format: .ascii "%s %d\n\0"
    buffer: .ascii "Computer System Foundation \n_\0_____"

.section .text
.global _start
_start:
    mov  _$0_____, %ebx    # ebx 存放 buffer 数组元素的下标
    mov  _$0_____, %ecx    # ecx 存放修改字母的个数

loop_begin:
    mov  buffer(%ebx), %al
    cmp  $0, %al
    je   loop_over
    cmp  $'A', %al
    jb   next_char
```

```

        cmp    '$Z', %al
        ja next_char
        add    'a' - 'A', %al
        mov    %al, buffer(%ebx)
        inc    %ecx
next_char:
        inc    %ebx
        jmp    loop_begin
loop_over:
        push   %ecx
        pushl   $buffer
        pushl   $format
        call    printf
        mov     $1, %eax # 程序正常退出
        mov     $0, %ebx
        int     $0x80

```

分 数	
评卷人	

五、问答题（20 分）

- 1、可重定位目标文件中有哪些节？各节中主要有什么信息？什么是符号解析？链接的过程是什么？（10 分）

可重定位目标文件中有：

.text 节：编译汇编后的代码

.data 节：已初始化的全局变量、静态局部变量

.rodata 节：只读数据

.bss 节：未初始化的全局变量、静态局部变量；初始化为 0 的全局变量、静态局部变量。

此外还有文件头、节头表。

符号解析，是将每个模块中引用的符号，与某个目标模块中的定义符号建立关联。

链接的过程包括符号解析和重定位两方面。符号解析如上所述。重定位则是分别合并代码和数据，并根据代码和数据在虚拟地址空间中的位置，确定每个符号的最终存储地址，然后根据符号的确切地址来修改符号的引用处的地址。

- 2、在一个程序的运行过程中（如正在执行一个二维数组求累加和），用户按了键盘上的某个键，计算机系统会做出哪些响应（即一系列的处理过程）？中断分哪几类？请举例说明各类中断在何种情况下产生。（10 分）

实模式下，键盘产生一个**中断请求**。

系统中断逻辑电路，将中断请求转为**中断号**，发送给 CPU。

CPU 响应中断请求。首先保存正在执行程序当前 **eip** 和其他寄存器的内容；然后根据中断号**查询中断向量表**，获取该中断相应的中断处理子程序的入口地址；然后中断当前程序，**跳转到该中断处理子程序中**，处理键盘输入；执行完中断处理子程序后，取回保存的 **eip** 和其他寄存器，**返回到原程序**中继续执行。

中断类型包括：

不可屏蔽外部中断：例如电源掉电。

可屏蔽外部中断：例如外部设备产生的中断请求。

CPU 检测的内部中断：例如除法出错。

程序检测的内部中断：例如程序中的软中断调用。

笔记

2026年5月28日 10:01

CF: 无符号数进/借位标志

ZF: 为零吗? SF: 结果为负吗? OF: 有溢出吗?

add: 增; sub: 减

inc: 自增; dec: 自减

neg: 求补(取反+1). cmp: 后-前.

imul/mul: 乘法. i: 有符号; r: 操作数受限

idiv/div: 除法.

and: 按位与. or: 按位或. xor: 按位异或

sar/shl: 左移. 源要么是立即数, 要么是 %cl.

sar: 算术右移. shr: 逻辑右移.

test: 按位与, 不存结果, 只改标志 ✗

jmp: 无条件跳转. js: 负 jns: 非负

je: 相等. jz: 为零 jne: 不相等. jnz: 不为零

jg: 大于 jge: 大于等于. jl: 小于. jle: 小于等于

ja: 无符号大于

jb: 无符号小于

jo: 溢出. jno: 不溢出

e: equal. n: not. $\begin{cases} g: \text{greater} & l: \text{less} \\ a: \text{above} & b: \text{below} \end{cases}$

链接: 1. Symbol Resolution: 扫描. 0 文件. 库文件.

将符号引用(函数名, 外部变量) $\rightarrow$ symtable

2. Relocation: .text 和 .data 合并. 为指令与全局变量分配地址.

根据 .rel.text 和 .rel.data. 分配真实地址.

Undefined reference: 未定义的引用.

Multiple definition: 多重定义.

中断分类:

代码段 (.text): 存指令代码

数据段 (.data): 存已初始化的全局变量和静态变量.

未初始化 $\rightarrow$ (.bss) $\uparrow$

只读数据段 (.rodata): 存只读常量字符串.

符号表 (.symtable): 存符号表信息.

(.dynamic): 动态链接信息.

(.dynsym): 动态符号表

中断分类:

中断

外部 I/O 输入信号

异常

主动请求操作系统
的服务

故障

指令发现错误

终止

致命硬件错误



华中科技大学计算机科学与技术学院 2023-2024 第一学期

“计算机系统基础”考试试卷(B卷)

考试方式 闭卷 考试日期 2024-3-18 考试时长 150 分钟

专业班级 学号 姓名

题号	一	二	三	四	五	总分	核对人
分值	20	20	20	20	20	100	
得分							

分数	
评卷人	

一、数据的存储表示和访问 (共 20 分)

1、数据的存储表示 (10 分)

void func1()

{

int x = 10;

int y[2] = { 20, 32 };

char msg[6] = "abcde";

int* p = &x;

short z = *(short*)(msg + 1);

int u = *(int*)(msg - 1);

}

设 x 的地址(&x) 为: 0x0000508; y[0] 的地址为: 0x000050c; msg[0] 的地址为: 0x000051a;

p 的地址(&p) 为: 0x0000514; z 的地址为: 0x0000518; u 的地址为: 0x0000520;

在函数 func1 执行结束返回前, 以字节形式观察内存内容, 以 16 进制的形式填写程序用到的内存单元内容 (无关的字节中的内容不要填写, 打 XX)。

0x0000508 0A 00 00 00 14 00 00 00

0x0000510 20 00 00 00 08 05 FF FF

0x0000518 62 63 64 65 66 67 68 69

0x0000520 6A 6B 6C 6D XX XX XX XX

2、调试一个 C 程序时, 在函数的反汇编中看到源程序语句与对应的反汇编语句如下, 根据

第 1 页 共 8 页

寻址方式: ① 立即寻址

② 寄存器寻址. → 寄存器里的数字.

③ 直接寻址

④ 寄存器间接寻址. → 寄存器里放指向内存位置.

⑤ 变址寻址

⑥ 基址加变址.

此处: 整片压栈, 而数组内元素向上生长.

做此类题应当厘顺思路. ✓

目的 = 源, lea 是
赋值操作

观察到的信息填空。(10 分)

lea 指令:

进行运算并存储

int a[2][3];

int i = 1;

movl \$1, -0x2c(%ebp)

i 的地址为 0x0019feb8, ebp = 0x0019feb4

int j = 3;

movl \$3, -0x30(%ebp) &j = 0x0019feb8

a[i][j] = 10;

iml \$0x1a, -0x2c(%ebp), %eax

执行后, eax = 0x14

eax 中值的含义是 行号乘以 4, 得到的偏移量.

lea -0x28(%ebp, %eax, 1), %ecx

ecx 中存放的值的含义是 0x11111111

mov -0x30(%ebp), %edx

movl \$0xa, (%ecx, %edx, 4)

指令中的 edx 乘 4 的原因是 int 字节长度 4, 使用达 0x11111111 位置.

a[0][0] 的地址表达式形式是 0x417120 + 0x20 * (%ebx, %ecx, 1)

global [i] = 18; global 为一个全局整型数组

mov -0x2c(%ebp), %eax

movl \$0x12, -0x1120(%eax, 4)

global 数组的起始地址是 0x417120

int* p = &global[i];

mov \$4, %eax

shl \$0, %eax

add \$0x417120, %eax

mov %eax, -0x34(%ebp)

变量 p 中的值是 0x417124

*p = 18;

mov -0x34(%ebp), %eax

movl \$0x13, (%eax)

第二条汇编语句访存的寻址方式是 寄存器间接寻址.

第 2 页 共 8 页

EIP: Extended Instruction Pointer

EIP: Extended Instruction Pointer

下一条将被取出的指令的地址。

一会复出过程。

0, ebp-4.
0x00411654 => +4即可, EIP的作用。

局部变量中保存入口地址。
return.

从右向左传参。
→ E-10 (b)
→ E-10 (a)
→ E-00413866 → 返回地址。

esp: 栈顶指针。

ebp: 基址指针。

0xa.
0xfffffff6h.

asp.

分 数	
评卷人	

二、程序运行的基本过程 (共 20 分)

1、在对某程序进行测试时,看到如下一段信息 (10 分, 每空 1 分):

① 设当前 eip = 0x00411650, 该处指令为 "83 7D F0 00 cmpl \$0, -4(%ebp)". 在取指

该指令并进行译码后, eip = 0x00411654.

② 在 0x00411654 处, 有指令 "7E 08 jle 0041165F", 综合前一条指令, 就是在

条件下, 将 0x0041165E -> eip, 计算前该地址值的方法是

③ 在 0x0041165E 处, 有 jmp 指令 "EB 00", 该指令对应的反汇编语句是 "jmp 0x00411666". 执行该指令时, 会 (有、无) 条件跳转到 0x00411666 处。

④ 在 0x00411666 处, 有一子程序直接调用指令 "E8 65 FC FF call 004117D0". 由此推断子程序的入口地址是 0x004117D0.

call 指令时, CPU 会将 0x0041166B 压入堆栈中。

⑤ 在 0x00411672 处, 有指令 "FF 55 F4 callq -0xc(%ebp)", 得到子程序入口地址的方法是指针在栈。

⑥ 执行 RET 指令时, CPU 会恢复 eip。

return

2、函数调用 (10 分)

在Linux环境下, 对一个C语言程序进行编译、链接、调试运行, 程序片段如下。

```
int fmax(int a, int b) {
    00411540 push    %ebp
    00411541 mov     %esp, %ebp
    00411548 sub     $0x10, %esp

    int result;
    if (a > b)
        00411549 mov     0x8(%ebx), %eax
        0041154C cmp     0xc(%ebp), %eax
        0041154F jle     fmax+19h (04115B9h)

        result = a;
    00411551 mov     0x8(%ebp), %eax
    00411554 mov     %eax, -0x4(%ebp)
    00411557 jmp     fmax+1Fh (04115BFh)
    else result = b;
    00411559 mov     0xc(%ebp), %eax
    0041155C mov     %eax, -0x4(%ebp)

    return result;
}
```

第 3 页 共 8 页

```
    004115BF mov     -0x4(%ebp), %eax
}

    004115C2 mov     %ebp, %esp
    004115C4 pop     %ebp
    004115C5 ret

void main() {
    int x = fmax(10, -10);
    00413863 push    $0xfffffff6h
    00413865 push    $0xa
    00413867 call    00411540
    0041386C add     $8, %esp
    0041386F mov     %eax, -0x4(%ebp)
    .....
}
```

设执行 main 函数中的 call 00411545 之前, esp = 0xffffd500; ebp = 0xffffd510.

填写刚进入函数 fmax 时 (执行 push %ebp 之前), 堆栈中相关单元的内容。

函数参数 a 的地址 (即 &a) 是 0x_____.

局部变量 result 的地址是 0x_____.

分 数	
评卷人	

三、C 语句的转换 (共 20 分)

1、阅读下面的程序, 回答问题 (10 分, 每小题 2 分)。

```
void fune ()
{
    unsigned short us_x = 0x8000;
    unsigned int ui_x;
    short s_x = -0x8000;
    int i_x;
    int u = 0, v = 0;           // ①
    ui_x = us_x;
    i_x = s_x;                 // ②
    if (ui_x > 0) u = -1;
    if (i_x > 0) v = -1;        // ③
    ui_x = ui_x >> 1;
    i_x = i_x >> 1;             // ④
    ui_x = "ui_x";
    i_x = -i_x;                 // ⑤
}
```

① 执行完 ① 处语句后,

第 4 页 共 8 页

2. 程序填空（10 分，每空 1 分）

将一个以 0 结束的字符串中的所有的大写字母转换为对应的小写字母，并将转换后的字符串，以及修改的字母个数输出。

```
.....
.section .data
format: .ascii  "%s  %d\n0"
buffer: .ascii  "Computer System Foundation 'n\_____"
.section .text
.global _start
_start:
    mov     _____, %ebx    # ebx 存放 buffer 数组元素的下标
    mov     _____, %ecx    # ecx 存放修改字母的个数
loop_begin:
    mov     buffer(%ebx), %al
    cmp     $0, %al
    _____
    cmp     $'A', %al
    _____
    cmp     $'Z', %al
    _____
    add     _____, %al
    mov     %al, _____
    inc     _____
next_char:
    _____
    jmp     loop_begin
loop_over:
    push    %ecx
    pushl   $buffer
    pushl   $format
    call    printf
    mov     $1, %eax    # 程序正常退出
    mov     $0, %ebx
    int     $0x80
```

解
答
内
容
不
得
超
过
装
订
线

分 数		五、问答题（20 分）
评卷人		

1、可重定位目标文件中有哪些节？各节中主要有什么信息？什么是符号解析？链接的过程是什么？（10 分）

2、在一个程序的运行过程中（如正在执行一个二维数组求累加和），用户按了键盘上的某个键，计算机系统会做出哪些响应（即一系列的处理过程）？中断分哪几类？请举例说明各类中断在何种情况下产生。（10 分）

零散补充

2026年5月29日 20:43

1. 哨兵

2. 串指令

牢兰的补充

2026年5月29日 23:04

第一部分：程序链接（链接、ELF、符号、重定位）1. 链接的基本概念定义：将多个可重定位目标文件（.o）合并成一个可执行文件（或共享库）的过程。两个核心任务：符号解析

（Symbol Resolution）：将符号引用与定义关联。重定位（Relocation）：给符号分配最终虚拟地址，并修改指令中的引用。考点填空：链接器的输入是_____，输出是_____。（答：可重定位目标文件；可执行目标文件/共享库）

2. ELF 文件（重点）2.1 两种视图视图使用者关键表核心单位链接视图链接器节头表（Section Header Table）节（Section）执行视图加载器程序头表（Program Header Table）段（Segment）简答：为什么同一个 ELF 文件有两种视图？（答：链接器需要细粒度信息（节）进行合并和重定位；加载器只需要粗粒度的段来映射内存权限。）

2.2 常见节（Section）及其作用（必考填空）节名存储内容是否在磁盘占空间示例.text机器指令是mov %eax, %ebx.data已初始化的全局/静态变量（初值非0）是int g=10; → 0x0a 00 00 00.bss未初始化或初始化为0的全局/静态变量否（仅占位符）static int s;.rodata只读数据（字符串常量、跳转表）是"hello"、printf 格式串.symtab符号表（变量名、函数名等）是每个符号的名称、类型、节索引.strtab字符串表（存放符号名字符串）是.symtab 中的 st_name 指向此处.rela.text.text 节中需要重定位的位置信息是记录偏移、符号索引、重定位类型.rela.data.data 节中需要重定位的位置信息是如全局指针变量的初始值

常考点：未初始化的全局变量放在 .bss 节，不占用磁盘空间。static 局部变量放在 .data 或 .bss，而不是栈。字符串字面量放在 .rodata，修改它会导致段错误。重定位信息存在 .rela.text / .rela.data 中。2.3 ELF 头（ELF Header）中的关键字段e_ident（魔数 0x7f 45 4c 46）、e_type（文件类型：可重定位、可执行、共享库）、e_shoff（节头表偏移）、e_shnum（节头表项数）、e_entry（入口地址，可执行文件有）。填空：readelf -h 可以查看_____。（ELF头信息）

3. 符号（Symbol）符号分类（非常重要）：全局符号（Global）：本模块定义、可被其他模块引用的符号。例：非 static 的函数名、非 static 的全局变量。外部符号（External）：本模块引用、其他模块定义的符号。例：extern int g;。局部符号（Local）：带 static 的函数或全局变量，仅本模块可见。例：static int s;。注意：函数的局部变量（auto 变量）不是链接器符号，它们只在栈上。强弱符号规则（旧版 GCC ≤8）：强符号：函数名、已初始化的全局变量。弱符号：未初始化的全局变量。规则：强符号不能重复定义。一强多弱 → 选强。多弱 → 任选一个（可能引发错误）。新版 GCC（≥9）：所有全局变量默认强符号，重复定义直接报错。填空/判断：int x; 是_____符号（弱/强）。（旧：弱；新：强）static int y = 10; 是_____符号（全局/局部/外部）。（局部）多个目标文件中有同名的非 static 全局变量且都未初始化，链接器会_____。（任选一个，无警告但危险）

4. 符号解析过程（链接器扫描算法）维护三个集合：E（已加入的目标文件）、U（未解析的符号）、D（已定义的符号）。扫描顺序：命令行中.o 和 .a 的顺序至关重要！静态库（.a）只用来解析当前 U 中未定义的符号；如果放在前面（U 为空），库中所有模块都会被丢弃，导致后面的 .o 找不到符号。简答：为什么链接时静态库一般放在所有目标文件之后？（答：为了让前面 .o 产生的未解析符号能够从库中找到定义。）5. 重定位（Relocation）重定位条目（Elf64_Rela）包含：r_offset：在 .text 或 .data 中的偏移位

置。r_info：高24位是符号表索引，低8位是重定位类型。r_addend：附加常数。常见重定位类型（x86-64）：类型计算方式用途R_X86_64_64S + A绝对地址（如全局变量地址）R_X86_64_PC32S + A - PPC相对寻址（如函数调用）S = 符号的实际地址，A = addend，P = 重定位位置的地址（r_offset）。考题形式：给定 objdump -r 输出或汇编指令中的偏移，计算最终地址。关键点：为什么 R_X86_64_PC32 的 addend 常为 -4？答：因为 rip 相对寻址的偏移量是相对于下一条指令的地址，而重定位位置往往位于指令中偏移量字段的起始（距离下一条指令恰好4字节），因此公式 $S - (P+4) = S + (-4) - P$ ，即 $A = -4$ 。

6. 静态链接 vs 动态链接特性

静态链接	动态链接
（加载时）	（运行时）
时机生成可执行文件	前程序加载时
程序运行中	文件大小大小
内存共享否是（代码段共享）	是（按需加载）
库更新需重新链接替换	.so 即可替换
.so 即可依赖缺失无法生成可执行文件	程序无法启动
程序可启动，调用相关功能时报错	使用场景嵌入式、静态工具常规应用（libc）
插件系统、dlopen	简答：动态链接的优点？（共享内存、更新方便、可执行文件小）
简答：动态链接的缺点？（启动稍慢、依赖环境、兼容性问题）	

7. 位置无关代码（PIC）与 PLT/GOT

PIC：使用相对寻址（如 rip 相对），使得代码可以加载到任意地址。PLT（过程链接表）：位于代码段，用于外部函数调用的延迟绑定。第一次调用时，PLT 跳转到动态链接器解析函数地址，填入 GOT；后续调用直接通过 GOT 跳转。GOT（全局偏移表）：位于数据段，存放全局变量和外部函数的实际地址。考点：为什么需要 PLT/GOT？（实现 PIC 和延迟绑定，提高启动速度）

第二部分：异常控制流（ECF）

1. 异常（Exception）的概念定义：控制流中出现的突变，由处理器检测到的某种事件（除零、缺页、系统调用等）触发。异常处理流程：处理器保存当前上下文（CS、EIP、EFLAGS）。根据异常编号在中断描述符表（IDT）中查找处理程序入口。跳转到处理程序执行。处理完毕后，IRET 恢复上下文，继续执行（或终止）。填空：异常处理程序执行完后，通过 _____ 指令返回。（答：iret 或 iretd）

2. 异常的分类（必考）

类别	原因示例	返回行为	故障（Fault）	可恢复的错误
除零（#DE）	可能重新执行当前指令	陷阱（Trap）	主动请求（系统调用）	int 0x80、syscall
返回下一条指令	终止（Abort）	致命错误	硬件故障、非法指令	不返回中断（Interrupt）
外部硬件事件	键盘中断、时钟中断	返回下一条指令	简答：缺页异常属于哪一类？（故障，因为处理后重新执行产生异常的指令）	填空：Linux 系统调用通过 _____ 指令（32位）或 _____ 指令（64位）实现。（int 0x80；syscall）

3. 中断（Interrupt）与异常的区别

中断	异步，来自外部硬件（可屏蔽/不可屏蔽）；由 INT n 指令产生的称为 软中断。异常：同步，由执行指令引发。可屏蔽中断（Maskable Interrupt）：可通过 CLI / STI 控制是否响应。不可屏蔽中断（NMI）：必须响应，用于紧急事件（如电源失效）。简答：用户单击鼠标，计算机如何处理？（外部硬件中断 → CPU 保存上下文 → 查找 IDT → 执行中断服务程序 → 读取鼠标数据 → 恢复上下文）
中断向量表（IDT）作用：存储每个中断/异常的处理程序入口地址（或门描述符）。实模式：中断向量表（IVT）固定在地址 0x0000 处，每项4字节（CS:IP）。保护模式：IDT 位置由 IDTR 寄存器指向，每个门描述符8字节（32位）或16字节（64位）。填空：CPU 响应中断时，根据中断号在 _____ 中查找处理程序入口。（IDT）	5. 系统调用（System Call）用户态程序请求内核服务的机制。Linux 32位：int 0x80，系统调用号放在 %eax，参数依次放在 %ebx、%ecx、%edx、%esi、%edi。Linux 64位：syscall 指令，系统调用号放在 %rax，参数放在 %rdi、%rsi、%rdx、%r10、%r8、%r9。简答：系统调用与普通函数调用的区别？（系统调用陷入内核，权限切换；普通函数调用不切换特权级。）
6. 上下文切换（Context Switch）由调度器触发，保存当前进程的上下文（寄存器、页表等），加载新进程	

的上下文。触发条件：时间片用完、I/O 阻塞、中断等。7. 信号 (Signal) —— 高级异常控制流软件中断，用于通知进程发生了异步事件 (如 SIGSEGV、SIGINT)。进程可以：忽略、捕获 (执行信号处理函数)、默认处理 (终止等)。第三部分：CPU 特性 (指令流水线、冒险、乱序等)

1. 指令流水线 (Pipelining) 基本思想：将一条指令的执行过程分解为多个阶段，每个阶段由独立的硬件处理，从而实现多条指令并行执行。经典五级流水线：阶段操作 IF (取指) 从指令缓存中取出指令，更新 PCID (译码) 解析指令，读取寄存器 EX (执行) 算术逻辑运算或地址计算 MEM (访存) 读/写数据存储器 WB (写回) 将结果写回寄存器

填空：流水线中，影响性能的主要障碍是 ____。(冒险 Hazards)

2. 冒险 (Hazards) 与解决方案

2.1 结构冒险 (Structural Hazard) 原因：硬件资源冲突 (如同一个存储器同时被取指和访存使用)。解决：分离指令缓存和数据缓存 (哈佛结构)；资源复制。

2.2 数据冒险 (Data Hazard) 原因：指令间的数据依赖。写后读 (RAW)：真依赖读后写 (WAR)：反依赖写后写 (WAW)：输出依赖

解决：插入气泡 (stall)：暂停流水线，等待数据准备。转发 (Forwarding/Bypassing)：将执行阶段的结果直接送往后继指令的输入，避免写回再读。编译器调度：重排指令，插入无依赖指令。

2.3 控制冒险 (Control Hazard) 原因：分支、跳转指令改变 PC 值，导致流水线可能取错指令。解决：分支预测：静态 (总是取分支/不取分支)、动态 (2-bit 饱和计数器、全局历史)。延迟分支：在分支指令后插入一条一定会执行的指令。双端口指令缓存：同时取分支两侧的指令。

简答：什么是分支预测错误惩罚？(预测错误时需要清空流水线，重新取正确路径的指令，损失多个时钟周期)

3. 乱序执行 (Out-of-Order Execution) 原理：在保证程序逻辑正确的前提下，不按程序顺序执行指令，以提高流水线效率。关键技术：寄存器重命名 (消除 WAR/WAW)、保留站、重排序缓冲区 (ROB)。填空：乱序执行中，指令提交 (commit) 必须按 ____ 顺序。(程序顺序)

4. SIMD (单指令多数据流) 目的：一条指令同时处理多个数据元素 (如 SSE、AVX)。寄存器：XMM (128 位)、YMM (256 位)、ZMM (512 位)。典型指令：addps (打包单精度加法)、movaps (对齐移动)、paddb (打包字节加法)。优化场景：图像处理、矩阵运算、多媒体编码。

5. 常见优化技术 (考点) 循环展开 (Loop unrolling) 使用 SIMD 指令减少分支 (条件传送代替分支) 内存对齐 (提高缓存命中) 编译器优化选项 (-O2、-O3、-fPIC 等) 简答：为什么 memset 比手写循环快？(内部使用 rep stos 或 SIMD，一次处理多个字节，减少循环次数)

三部分综合复习建议链接部分：重点掌握 ELF 节的作用、符号分类、重定位计算、静态/动态链接区别。做几道 objdump + readelf 的练习题，熟悉重定位条目。异常控制流：熟记异常分类 (故障/陷阱/终止/中断)，理解 IDT 和系统调用流程。简答题常考“鼠标中断过程”“缺页异常处理”。CPU 特性：理解流水线五级、三类冒险的解决方案。简答题可能让你画出流水线状态或分析冲突。记住分支预测、转发、乱序的基本思想。

24春B卷答案

2026年5月28日 9:59



24春B卷答
案



华中科技大学 2023~2024 第二学期

“ 计算机系统基础 ” 考试试卷 (B 卷)

考试方式 闭卷 考试时间 2024 年 5 月 25 日 考试时长 150 分钟

院 (系): 专业班级:

学 号: 姓 名:

题号	一	二	三	四	五	总分	总分人	核对人
分值	20	20	20	20	20	100		
得分								

分 数	
评卷人	

一、(20 分) 数据的存储和访问

1、 设有如下程序,调试时在函数 f 结束前观察了变量的地址以及内存单元内容。请将程序、以及内存单元中的内容补充完整。(10 分)。

```
struct person {
    char    id[6];
    int     height;
    double  salary;
    char    grade;
};
void f()
{
    int     x=25;           // &x 为 0xffffd4e0
    int     *px = &x;       // &px 为 0xffffd4e4
    short   sa[2]={-2, 5};  // &sa (即 &sa[0]) 为 0xffffd4e8
    char    msg[6]="abcd";  // &msg[0]为 0xffffd506, 'a' 的 ASCII 为 0x61
    struct person p;        // &p 为 0xffffd4ec, &(p.salary)为 0xffffd4f8
                                // p 中各字段的地址被对应类型长度整除

    strcpy(p.id, "**");      ① ** 应被替换为 1234
    p.height = $$;          ② $$ 应被替换为 0x64
    p.grade='@@';           ③ @@ 应被替换为 b
    p.salary = 12000;        ④ p.salary 的表示为 0x40 c7 70 00 00 00 00 00
                                ⑤ person 的大小为 24 字节。
}
```

下面是以 16 进制数的形式显示内存内容,每空都需要填一个字节,最左边是内存地址。
注:数据是小端存储方式;部分内存单元内容与结构变量中的字段无关,具有随机性。

0xffffd4e0	19	00	00	00	e0	d4	ff	ff
0xffffd4e8	fe	ff	05	00	31	32	33	34
0xffffd4f0	00	e4	fb	f7	64	00	00	00
0xffffd4f8	00	00	00	00	00	70	c7	40
0xffffd500	62	d5	ff	ff	6c	e6	61	62
0xffffd508	63	64	00	XX	XX	XX	XX	XX

2、 根据调试时观察到的信息，填写空行处的 C 语句或将语句补充完整（10 分）。

```
int global[5]; // 全局变量
void data_access()
{
    ..... // 此处有一些机器指令设置了 %esp, %eax 等
    int i=0;
        movl    $0x0, (%esp)
    int local[5];
    int *p=NULL;
        movl    $0x0, 0x4(%esp)
    global[0] = 1;
        movl    $0x1, 0x34(%eax)
    global[ 1 ] = 2;
        movl    $0x2, 0x38(%eax)
    i = 2;
        movl    $0x2, (%esp)
    i++;
        addl    $0x1, (%esp)
    global [ i ] = 3;
        mov     (%esp), %edx
        movl    $0x3, 0x34(%eax,%edx,4)
    *(global + 3) = 10;
        movl    $0xa, 0x40(%eax)
    local[0] =1;
        movl    $0x1, 0x8(%esp)
    local [ 3 ] = 20;
        movl    $0x14, 0x14(%esp)
    p = &local [0];
        lea     0x8(%esp), %eax
        mov     %eax, 0x4(%esp)
    p += 4;
        addl    $0x10, 0x4(%esp)
    *p = 0x32;
        mov     0x4(%esp), %eax
        movl    $0x32, (%eax)
    i = local [2];
        mov     0x10(%esp), %eax
        mov     %eax, (%esp)
    .....
}
```

分 数	
评卷人	

二、(20 分) C 语言常用运算及函数调用的机器级实现

1、写出各条 C 语句执行后相应变量的值，并将机器指令补充完整 (10 分)。

```

void myfunc( )
{
    .....
    int      x,  y;
    unsigned int  u, v;
    unsigned short s = 0xfffe;
    0x0000119d :   movw   $0xfffe, -0x14(%ebp)
    short t = -1;
    0x000011a3 :   movw   $0xffff, -0x12(%ebp)
    x = s;      // ① x = 0x_0000fffe_
    0x000011a9 :   movzwl  -0x14(%ebp), __%eax__
    0x000011ad :   mov     %eax, -0x10(%ebp)
    y = t;      // ② y = 0x_ffffff_
    0x000011b0 :   _movswl__ -0x12(%ebp), %eax
    0x000011b4 :   mov     %eax, -0xc(%ebp)
    u = s;      // ③ u = 0x_0000fffe_
    0x000011b7 :   _movzwl__ -0x14(%ebp), %eax
    0x000011bb :   mov     %eax, -0x8(%ebp)
    v = t;      // ④ v = 0x_ffffff_
    0x000011be :   _movswl__ -0x12(%ebp), %eax__
    0x000011c2 :   mov     %eax, -0x4(%ebp)
    if (x > 0) t = 1; // ⑤ t = 0x_1_
    0x000011c5 :   cmpl    $0x0, -0x10(%ebp)
    0x000011c9 :   _jle__  0x11d1 <myfunc+68>
    0x000011cb :   movw   $0x1, -0x12(%ebp)
    if (u > 0) t = 2; // ⑥ t = 0x_2_
    0x000011d1 <+68>: cmpl    $0x0, -0x8(%ebp)
    0x000011d5 :   _jbe__  0x11dd <myfunc+80>
    0x000011d7 :   movw   $0x2, -0x12(%ebp)
    u = u >> 2;    // ⑦ u = 0x_3fff_
    0x000011dd <+80>:   _shrl   $0x2__, -0x8(%ebp)
    x = x >> 2;    // ⑧ x = 0x_3fff_
    0x000011e1 <+84>:   _sarl   $0x2__, -0x10(%ebp)
    y = -y;        // ⑨ y = 0x_1_
    0x000011e5 <+88>:   _negl__  -0xc(%ebp)
    v = ~v;        // ⑩ 按位取反; v = 0x_0_
    0x000011e8 <+91>:   _notl__  -0x4(%ebp)
    .....
}

```

解答内容不得超过装订线

2、函数调用的机器级实现 （10 分）

语句 “int r = fsub(25,10);” 对应的反汇编代码（最左边的是机器指令的地址）如下。

```
0x565561bb: 6a 0a push $0xa
0x565561bd: 6a 19 push $0x19
0x565561bf: e8 c9 ff ff call 0x5655618d <fsub>
0x565561c4: 83 c4 08 add $0x8,%esp
0x565561c7: 89 45 fc mov %eax,-0x4(%ebp)
```

设执行 “int r = fsub(25,10);” 之前，esp 的值为

0xffffd4f8，ebp 值为 0xffffd508。

0x0f	0xffffd4e4
0xffffd508	0xffffd4e8
0x565561c4	0xffffd4ec
0x19	0xffffd4f0 x
0xa	0xffffd4f4 y
XXXXXXXXXX	0xffffd4f8

函数 fsub 的反汇编代码如下：

```
int fsub(int x, int y) {
    0x5655618d <+0>: 55 push %ebp
    0x5655618e <+1>: 89 e5 mov %esp,%ebp
    0x56556190 <+3>: 83 ec 10 sub $0x10,%esp
    0x56556193 <+6>: e8 35 00 00 00 call 0x565561cd <__x86.get_pc_thunk.ax>
    0x56556198 <+11>: 05 44 2e 00 00 add $0x2e44,%eax

    int result;
    result = x - y;

    0x5655619d <+16>: 8b 45 08 mov 0x8(%ebp),%eax
    0x565561a0 <+19>: 2b 45 0c sub 0xc(%ebp),%eax
    0x565561a3 <+22>: 89 45 fc mov %eax,-0x4(%ebp)

    return result;

    0x565561a6 <+25>: 8b 45 fc mov -0x4(%ebp),%eax
}
0x565561a9 <+28>: c9 leave
0x565561aa <+29>: c3 ret
```

在上面的表格中，填写执行到 leave 时相应内存单元中的值（表格中的 5 行都需要填写，每行均用 0x 作为前缀）。

- ① 函数参数 x 的地址（即&x）是 0x_ffffd4f0_。
- ② 局部变量 result 的地址（即&result）是 0x_ffffd4e4_。
- ③ 执行 “sub 0xc(%ebp), %eax” 后，CF=_0_, SF=_0_, ZF=_0_, OF=_0_。
- ④ 执行 “ret” 后，%esp = 0x_0xffffd4f0_

分 数	
评卷人	

三、(20 分) 计算机系统的基本原理

1、程序计数器 (eip) 的自动变化 (10 分)

设有 void myfunc() {

int main() {

int x = -20;

0x565561ca <+29>: c7 45 f0 ec ff ff movl \$0xffffffff,-0x10(%ebp)

// 在取出该指令并进行译码后, eip = 0x_565561d1_

int y;

void (*fp)();

if (x >= 0) y = 1;

0x565561d1 <+36>: 83 7d f0 00 cmpl \$0x0,-0x10(%ebp)

0x565561d5 <+40>: 78 09 js 0x565561e0 <main+51>

// 上面语句中的 0x565561e0, 是_(eip) + 09_ 计算得到的

// 当 js 转移条件成立, 即 sf = _1_ 时, 执行 (eip) + 指令中位移量 → eip

// 若 js 转移条件不成立, 该指令也就执行完成, 此时 eip = 0x_565561d7_

0x565561d7 <+42>: c7 45 ec 01 00 00 00 movl \$0x1,-0x14(%ebp)

0x565561de <+49>: eb 07 jmp 0x565561e7 <main+58>

// 上面语句中的机器码 07, 表示的含义是_目标地址与当前 eip 之前的位移量_

else y = -1;

0x565561e0 <+51>: c7 45 ec ff ff ff ff movl \$0xffffffff,-0x14(%ebp)

myfunc();

0x565561e7 <+58>: e8 b1 ff ff ff call 0x5655619d <myfunc>

// 执行 call 指令时, 会将_将 eip 压栈_, 将_0x5655619d_ → eip

fp = myfunc;

0x565561ec <+63>: 8d 83 c5 d1 ff ff lea -0x2e3b(%ebx),%eax

// 执行 lea 指令时, ebx = 0x_56558fd8_ 或 0x5655619d + 0x2e3b _

0x565561f2 <+69>: 89 45 f4 mov %eax,-0xc(%ebp)

fp();

0x565561f5 <+72>: 8b 45 f4 mov -0xc(%ebp),%eax

0x565561f8 <+75>: ff d0 call *%eax

// 执行 call 指令时, 采用_寄存器_寻址方式, 得到函数的入口地址

0x565561fa <+77>:

}

执行函数 myfunc 中的 ret 指令时, CPU 会_从栈顶弹出一个双字_ → eip。

解答内容不得超过装订线

2、程序执行的基本原理（10 分）。

(1) 程序运行的基本过程是什么？

A、取指令，cpu 根据 eip 的值从内存中读入指令； B、指令译码， eip 自增指令的长度；
C、取操作数，计算操作数的地址并根据地址取得操作数； D、执行，在算术逻辑运算部
件执行相应的操作； E、结果写回。重复上述步骤，使得程序自动地一条一条执行。

(2) 举例说明什么是地址类型转换？什么是数据类型转换？

地址类型转换：int v; char *p = (char *)&v; 相同的内存单元，视为存放不同类型的
数据。 数据类型转换：int v=1; float f=(float)v; 相同的值 1，整型、浮点型有不同的
表现形式。

(3) 全局变量与非静态的局部变量各分配在什么段中？ 为什么一个函数不应返回局部变量
的地址？

全局变量在数据段中分配空间，非静态局部变量在堆栈段中分配空间。

返回（非静态）局部变量的地址，也就是返回栈中一个单元的地址。堆栈是各个函数共用的
空间；函数在返回后，函数之前占用的堆栈空间被其他函数所用，根据该地址进行后续操作
时，访问到的数据可能并非原来函数结束前的值，存在不确定性。

(4) 指令集体系结构与计算机系统的什么部件有关？它是哪两个部分之间的接口？

指令集体系结构（ISA）与 CPU 有关，CPU 决定了 ISA。

ISA 是 软件和硬件之间的接口。各种程序都是由指令构成， 而指令交给 CPU 去执行。

(5) 什么是缓冲区溢出？缓冲区溢出会有哪些危害？

缓冲区溢出：读/写缓冲区时，超出了缓冲区定义的范围。

缓冲区溢出可能会造成程序出现逻辑错误（如非预期修改了其他变量的值）、异常、甚至崩溃
（如越出本程序的空间分配范围）。

分 数	
评卷人	

四、（20 分）程序编译及优化

1、举例说明编译器利用 CPU 的特性所能进行的优化。请给出 5 种不同类型的示例，并说明
优化原理。（10 分）

- (1) 利用指令流水线：循环展开、分支语句向无分支转换等等。当遇到转移时，指令流水线上所
做的一些工作将被抛弃，流水线的效率受到影响。
- (2) 利用 cache：尽量按顺序访问内存单元（例如将按列访问二维数组调整为按行访问）， 否则
会产生 cache 抖动，即一个数据块的频繁换入换出 cache，增加时间开销。
- (3) 利用更快的机器指令：例如 lea -0x7(%eax,%eax,8), %eax 实现 $9*(eax) - 7 \Rightarrow eax$ 。不同指
令的执行速度不同。
- (4) 利用串操作指令：代替对串中的数据一个一个循环操作（如将一个存储区中内容拷贝到另一
个存储区中）。
- (5) 使用 SIMD：一条指令可以同时操作多个操作数，对于批量数据减少操作次数，节约时间。
- (6) 寄存器与变量绑定：不需要访问内存单元（变量）。
- (7) 使用多核 CPU：多线程编程。

2、除了利用 CPU 特性进行优化，编译器还可以优化减少在 CPU 上执行指令。举出三种不同类型的优化示例。（6 分）

(1) 去除“废”语句（废语句段）：

`int x = 1; for(x = 2; x < 10; x++) { ... } =>`
`int x; for(x = 2; x < 10; x++) { ... }` 有这些语句与没有这些语句的运行结果是等同。

(2) 可以计算出的结果：

`int x = 1; int y = x + 2; int z = f(x); int f(int x) { return x + 1; } =>`
`int x = 1; int y = 3; int z = 2;`

(3) 将函数调用转换成直接使用函数体的形式：

减少函数参数的传递，减少 CALL 与 RET 指令。

(4) 减少部分运算

`int f(int x,int y) {int u=2*x; int v=3*y+5; return u+v; }`
 当函数调用为 `w=f(10,a);` 可以省掉 `int u=2*x,` 直接使用 `return 25+3*y;`

3、为了提高程序的安全性，编译器可以在程序中生成一些安全检查代码。举出两种不同的安全检查示例并说明其原理。（4 分）

堆栈帧检查：访问存储空间是否发生溢出，例如 `int a[2]; a[2]=10;` 发生了溢出。

为数组 `a` 的前后，各预留一个 4 字节的空间，初值置为 `0xffffffff`；在程序结束前，检查预留的空间中的内容是否还是 `0xffffffff`；若不是，则说明发生了溢出。

未初始化变量使用检查：检查一个变量在使用之前是未初始化。

示例：`int x; if (flag==1) x=10; int y=x;` 注：编译器可以肯定未初始化就使用的变量，在编译时就会报错；对于有条件初始化变量（如 `flag==1` 时），则编译器可能无法判定是否初始化。

对于有条件初始化变量 `x`，编译器为其生成一个影子变量，初值为 0。对于 `x` 的赋值语句，同时生成成为影子变量置 1 的指令；对与使用 `x` 的语句，则判断影子变量是否为 1，确定是否使用了未初始化的变量。

分 数	
评卷人	

五、（20 分）链接和异常控制流

1、设有如下程序（10 分）

```
#include <stdio.h>
int global =20;
int main()
{
    int x , y;
    x = 25;
    global = 25;
    y = global;
    printf("good %d %d\n",x, y);
    return 0;
}
```

在编译生成可重定位目标文件中，会有数据节(.data)、只读数据节(.rodata)、代码节(.text)、代码节的重定位节(.rel.text)、符号表节(.symtab)、字符串节(.strtab)等。

请结合给出的程序，回答如下问题。

(1) 上述六个节中分别包含哪些信息？(6分)

.data: 为 global 分配 4 个字节，被设置为 0x00000014

.rodata: "good %d %d\n", 0

.text: main()函数的机器指令

.rel.text: 对于每一个需要重定位的条目，都会有如下重定位信息，什么位置需要重定位、由哪一个符号来重定位、用什么方式来重定位，以及重定位位置计算的附加量。

.symtab: 所有符号(global、main、printf)的相关信息，如符号所在的节号、在节(.data)节的地址、所占空间的大小，等等。

.strtab: 所有符号的名字，如"global"、"main"、"printf"

(2) 需要重定位的符号有哪几个？(2分)

3 个符号: global、main、printf

(3) 哪些 C 语句和语句中的哪些部分需要重定位？(不用给出具体的机器指令)？(2分)

global = 25; global

y = global; global

printf("good %d %d\n", x, y); printf 和 "good %d %d\n" 都需要重定位

2、异常控制流 (10分)

(1) 产生中断的原因有哪些？

不可屏蔽事件(如掉电)、可以屏蔽的外部事件(如敲击键盘、鼠标、时钟中断等)。

(2) 产生异常的原因有哪些？

CPU 执行指令时引起的异常事件(如软中断、除法出错、单步中断、缺页、访问单元地址越界等)。

(3) 中断和异常的差别什么？

中断是由外部设备触发、与正在执行的指令无关的异步事件。异常是与正在执行的指令相关的同步事件(与外部设备无关)。

(4) 什么是中断描述符表？

存放中断处理程序相关信息(入口地址、类别、权限等)的表。

(5) 中断和异常的响应过程是什么？

CPU 捕捉到中断事件，CPU 暂停正在进行的程序，根据中断描述符表获取中断服务程序的入口地址，然后执行中断服务程序，执行完后返回到程序暂停的位置继续执行。异常的响应过程是类似的，异常处理后，可以重新执行引起异常的指令(故障)，或者引起异常指令的下一条指令(陷阱)，或者终止。